

UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Master Degree in Computer Science

Curriculum Artificial Intelligence

Hebbian Co-Design of Morphing Winged Drones

Supervisors:

Prof. Andrea Cossu

Andrea Vicari, PhD

Dr. Fuda Van Diggelen

Prof. Dario Floreano

Candidate:

Salvatore Mastrangelo

ACADEMIC YEAR 2025/2026

Abstract

The body and the controller of a flying robot depend on each other, yet the two are usually designed in sequence. Designing them together, or co-design, evaluates every candidate body through the flight of a controller suited to that body. Training a controller for every body is too expensive. A single shared controller is cheap, but it biases the search towards the bodies that resemble its training distribution.

This thesis proposes a controller that adapts during flight to the body it is given. The setting is a morphing winged drone that flies through a forest. A recurrent generalist controller is trained once by reinforcement learning on 256 random bodies, and then frozen. The generalist is never told which body it flies. Evolved Hebbian rules change the 224 output weights of the generalist during every flight. Plasticity is not used to learn the task, but to adapt to the body a solution of the task that was learned for all bodies. The rules are evolved by the Covariance Matrix Adaptation Evolution Strategy, nested inside a search over the bodies by the Non-dominated Sorting Genetic Algorithm II. The two objectives are the progress along the forest and the cost of transport. No network is trained inside the pipeline, and a run evaluates up to 1 600 distinct bodies in about four days on one computing node.

Eight runs of the pipeline are compared with eight control runs of the same body search under the frozen generalist. The co-design fronts are up to 17 % cheaper at equal progress and fly up to 12 % farther at equal cost of transport. On both indicators, every co-design run is better than every control run. *An adaptation that costs no more than the flights is enough to change the outcome of the search over bodies.* The rules specialize to the bodies that their search retains: they add 9 to 19 m of progress to those bodies, and only 1 to 6 m to the bodies of a control front. The price is generality: on random bodies, the rules of a run end 18 % below the frozen generalist. All results were obtained in simulation.

Glossary

- **BPTT:** Backpropagation Through Time
- **CI:** Confidence Interval
- **CMA-ES:** Covariance Matrix Adaptation Evolution Strategy
- **CoT:** Cost of Transport
- **EA:** Evolutionary Algorithms
- **ELU:** Exponential Linear Unit
- **EPANN:** Evolved Plastic Artificial Neural Network
- **ES:** Evolution Strategy
- **GAE:** Generalized Advantage Estimation
- **IR:** Intermediate Representation
- **KL:** Kullback-Leibler (divergence)
- **LSTM:** Long Short-Term Memory Networks
- **MAML:** Model-Agnostic Meta-Learning
- **MDP:** Markov Decision Process
- **NSGA-II:** Non-dominated Sorting Genetic Algorithm II
- **PCA:** Principal Component Analysis
- **POMDP:** Partially Observable Markov Decision Process
- **PPO:** Proximal Policy Optimization
- **RL:** Reinforcement Learning
- **RNN:** Recurrent Neural Network
- **SBX:** Simulated Binary Crossover
- **TD:** Temporal Difference
- **TRPO:** Trust Region Policy Optimization
- **URDF:** Unified Robot Description Format

Contents

1 Introduction **11**

1.1 Co-Design and Its Cost 11
1.2 Plasticity as Adaptation 12
1.3 Contributions 13
1.4 Structure of the Thesis 14

2 Technical Background **15**

2.1 Deep Reinforcement Learning 15
2.1.1 Markov Decision Processes 15
2.1.2 Actor-Critic Reinforcement Learning 17
2.1.3 Proximal Policy Optimization 19
2.2 Long Short-Term Memory Networks 20
2.2.1 Recurrent Neural Networks 20
2.2.2 Vanishing Gradient Problem 20
2.2.3 Long Short-Term Memory 21
2.3 Evolutionary Algorithms 23
2.3.1 CMA-ES 23
2.3.2 NSGA-II 25
2.4 Hebbian Learning 27
2.4.1 Hebb's Postulate 28
2.4.2 The Generalized Hebbian Rule 28
2.4.3 Plasticity Rules as the Object of Optimization 30

3 Related Work & Challenges **31**

3.1 Related Work 31
3.1.1 Co-Design of Morphology and Control 31
3.1.2 Evolved and Learned Plasticity 33
3.1.3 Positioning of This Thesis 35
3.2 The Sequential Design Challenge 36
3.3 The Morphology Optimization Challenge 36
3.4 The Computational Feasibility Challenge 37

4 Simulation Environment **41**
4.1 Genesis: The Simulator . . . 41
4.1.1 Parallel Simulation on the GPU . . . 41
4.1.2 Physics Simulation . . . 43
4.1.3 Aerodynamics Simulation . . . 44
4.1.4 Simulation Noise . . . 45
4.2 Flight Environment . . . 46
4.2.1 Growing Density Forests . . . 47
4.3 Drone Model . . . 48
4.3.1 Drone Shape and Actuators . . . 48
4.3.2 Sensors . . . 51
4.3.3 Drone Genome . . . 53
4.3.4 The Standard Drone . . . 53

5 Methodology **57**
5.1 Base Controllers . . . 57
5.1.1 Architecture of the Controller . . . 57
5.1.2 Training Environment . . . 59
5.1.3 Reward Function . . . 61
5.1.4 Training the Generalist . . . 64
5.1.5 Specialist Controllers . . . 67
5.2 The Hebbian Controller . . . 67
5.2.1 Hebbian Rules on the Output Layer . . . 67
5.2.2 Why the Output Layer . . . 69
5.2.3 Weight Decay and the Lifetime of the Weights . . . 70
5.2.4 Parameters of the Plastic Layer . . . 72
5.3 The Co-Design Pipeline . . . 74
5.3.1 Evaluation Flights . . . 75
5.3.2 Inner Loop: Evolving the Hebbian Rules . . . 75
5.3.3 Outer Loop: Evolving the Drone Morphology . . . 78
5.3.4 The Course of a Run . . . 79
5.3.5 The Control Condition . . . 80

6 Experiments And Results **83**
6.1 Inner Loop Experiments . . . 83
6.1.1 Recovering Performance on Generalist Controllers . . . 84
6.1.2 Improving Performance on Specialist Controllers . . . 87
6.2 Full Co-Design Experiments . . . 90
6.2.1 Co-Design vs. Morphology-Only Evolution . . . 90
6.2.2 Swapping the Pareto Fronts . . . 95

6.2.3 PCA Analysis of the Evolved Morphologies 97
6.2.4 Morphology Choices of The Evolution 99
6.2.5 Generalization Capabilities of Evolved Controllers 102

7 Conclusions & Future Work 105
7.1 Conclusions 105
7.2 Future Work 108
7.2.1 Meta-Learning Techniques 108
7.2.2 Testing on Real Drones 109
7.2.3 Testing on Different Systems 109

A Appendix A: Simulation Details 117
A.1 Genesis Architecture and Taichi Kernels 117
A.2 Rigid Body Dynamics 119
A.3 Aerodynamic Model 120
A.4 Aerodynamic Noise 124
A.5 Collision Test and Forest Statistics 124
A.6 Uniform and Latin Hypercube Forests 125
A.6.1 Uniform Density Forests 125
A.6.2 Latin Hypercube Forests 126
A.7 Mass Model and Propeller Power of the Drone 128

Introduction

A flying robot is a body and a controller. The body decides what the robot can do: how much lift its wing produces, how much energy it spends per metre, how quickly it can turn. The controller decides what the robot does with that body. The two choices depend on each other. The best controller for one body is not the best controller for another body, and the fitness of a body can only be evaluated through the flight using such controller. Despite this coupling, flying robots are usually designed in sequence. The body comes first, from an engineer or from a commercial product, and the controller is designed afterwards for that body. This thesis studies the alternative, in which the body and the controller are designed together. It does so for a winged drone that has to fly through a forest, and it proposes a controller that adapts to the body it is given while it flies.

The drone of this thesis is a fixed-wing glider with a propeller at its nose and a wing in two panels. Each panel sweeps backwards and twists about its axis under a servomotor, and the elevator and the rudder move as a whole. The drone has no ailerons: it steers by changing the shape of its wing. Its body is described by fifteen parameters: the span, the aspect ratio and the airfoil section of the wing, the length of the fuselage, the position of the centre of mass and of the wing along the fuselage, the size of the elevator and of the rudder, the dihedral, and the gear ratios of the sweep and twist joints. These parameters span a wide space of bodies, from light and agile drones of about half a kilogram to gliders of two kilograms with a wing of almost a square metre. The controller is a neural network. It reads the altitude, the attitude and the velocity of the drone and a coarse depth image of the trees ahead, and it sets the throttle and the six joints twenty-five times per second. The task is to fly as far as possible through a corridor of trees whose density grows along the way, and to spend as little energy as possible in doing so. A flight is evaluated by two quantities. The *progress* is the distance covered along the corridor. The *cost of transport* is the energy spent per unit of weight and of distance. Every flight of the thesis takes place in simulation.

1.1 Co-Design and Its Cost

Designing the body and the controller together is a form of *co-design*. As an optimization problem, co-design is nested. An outer search runs over the bodies, and it scores every candidate body by the flight of the best controller for that body. An inner search finds that controller. The inner search is what makes the problem expensive. The co-design works that use learned controllers train a new controller from scratch for every body the search visits [27, 51]. In the setting of this thesis, a controller trained on many bodies takes fourteen hours on a computing node with two graphics cards. A controller trained on a single body takes less than two hours on one of the cards. One search over bodies evaluates 1 600 distinct bodies. A training for each of those bodies would take two and a half years in the first case. In the second case it would take almost two months, with two trainings running at

a time. The works that take this route stay under a thousand bodies in total, or they require more than a thousand processors for a single run.

The cheap alternative is to train one *generalist* controller on many bodies at once, and to let that single controller score every candidate body. This alternative has a cost of another kind. The generalist is a compromise between all the bodies it was trained on. It flies well the bodies that resemble its training distribution, and it flies poorly the bodies that differ from that distribution. Two bodies can therefore come in one order under the generalist and in the opposite order under their own controllers. Therefore a search under the generalist keeps the body that suits the generalist, and it discards the body with the greater potential. Training a controller for the chosen body afterwards does not repair this choice, because the body that was discarded cannot be recovered.

These facts are presented in chapter 3 as three challenges. The sequential design challenge: a body chosen before any search can never be improved by the search, whatever controller is trained on it. The morphology optimization challenge: a search under one fixed controller finds the bodies that suit that controller, not the bodies with the greatest potential. The computational feasibility challenge: training a controller for every body the search visits costs more than any search can afford. Together, the three challenges fix the shape of any workable solution. *The controller must be trained once, for the whole space of bodies, and must adapt cheaply to the bodies thare are being scored.* Whether an adaptation that cheap can change which bodies the search finds is the empirical question of this thesis.

1.2 Plasticity as Adaptation

The answer proposed in this thesis is a controller whose weights change during flight under evolved Hebbian rules. The starting point is a generalist: a recurrent neural network, with a Long Short-Term Memory layer followed by two feed-forward layers and a linear output layer, trained by reinforcement learning with Proximal Policy Optimization on 256 random bodies at once. The controller is never told which body it is flying. It receives no description of the body, so that it learns to fly any morphology without hints on what the flight dynamics could be. Once trained, the network is frozen. Only the weights of its output layer are allowed to change: 224 gains that translate the 32 features of the last hidden layer into the seven commands. At every control step each gain moves by an amount that depends on the activation of the *pre-synaptic* (the feature) and the *post-synaptic* (the command) neurons. The adaptation mechanism is the generalized form of Hebb's postulate, with four coefficients for each gain. Each gain also decays back towards its trained value. The 896 coefficients are evolved with the Covariance Matrix Adaptation Evolution Strategy (CMA-ES), on the mean reward of the flights of a whole population of bodies. The rules don't know the body morphology too, but it influences the evolutions since the body is inside the control loop. The commands act on the body, the body answers according to its wing, its tail and its mass, and the frozen layers turn that answer into the activity that the rules read. Two bodies flown with the same rules therefore end the flight with different weights.

This use of plasticity differs from the one found in the literature on evolved

plastic networks. There, the weights are drawn at random at the beginning of every episode, and the rules have to build a working controller from nothing [20, 52, 11]. In other works the fixed weights and the rules are co-trained by the same gradient [48]. Here the controller already exists and already flies. *Plasticity is not used to learn the task, but to adapt to the body a solution of the task that was learned for all bodies.* The rules start every flight from the trained weights and move them by a bounded amount. When every coefficient is zero, the Hebbian controller is the frozen generalist. This property gives the search of the rules a natural starting point, and it gives the experiments an exact control condition.

The Hebbian controller is placed inside a co-design pipeline made of two nested evolutionary searches. The inner loop evolves the rules with CMA-ES. At every generation the fitness of each set of rules is the mean reward of many flights. The outer loop evolves the bodies with the Non-dominated Sorting Genetic Algorithm II (NSGA-II). Every four generations of the inner loop, the eight best sets of rules fly every body on a longer course, the exam. The progress and the cost of transport measured in the exam are the two objectives of NSGA-II, which keeps half of the bodies and replaces the other half with offspring. The search of the rules is not restarted when the bodies change. A run of 200 generations examines 3 136 bodies, of which at most 1 600 are distinct, and takes about four days on one computing node, and no network is trained inside the run. The control condition is the same pipeline with the learning rate of the rules set to zero. The control starts from the same bodies, uses the same operators, the same exam and the same admission gate, and flies every body with the frozen generalist. The two conditions differ in the 896 coefficients, and the control is given the larger budget in computing time and in examined bodies.

1.3 Contributions

The contributions of this thesis are the following.

1. **A plastic controller on top of a frozen policy.** Evolved Hebbian rules act on the output layer of a policy that was pretrained by reinforcement learning and then frozen (section 5.2). No previous work places evolved rules on top of a frozen pretrained policy. Plasticity has been co-evolved with bodies before [18], with a separate set of rules in the genome of each body. Here the same set of rules flies every body of the population inside the co-design loop.

2. **A co-design pipeline that scores bodies by flights alone.** An evolution strategy over the rules is nested inside a multi-objective search over the bodies, and no controller is trained inside the search (section 5.3). The pipeline evaluates 1 600 distinct bodies in four days on one computing node. A controller training per body would take months on the same node.

3. **A simulation setting for the co-design of winged drones.** The task, the procedurally generated forests, the parametric space of bodies and the measurement of the two objectives were set up on the Genesis engine [23] and

on an aerodynamic solver developed in the Laboratory of Intelligent Systems of EPFL, where this work was carried out (chapter 4).

4. **A controlled comparison with morphology-only evolution.** Eight runs of the pipeline are compared with eight runs of the search under the fixed generalist, six of them started from the same bodies, with the larger budget given to the control (section 6.2). The co-design fronts are up to 17 % cheaper at equal progress and fly up to 12 % farther at equal cost of transport. On these indicators every co-design run is better than every control run. The evolved bodies' genome principal components are different too compared to the control.

5. **An analysis of what the rules do.** On a fixed body the rules move any base controller, the generalist and the specialists trained for that body alike, towards a slower and cheaper flight of the same course (section 6.1). Inside the search the rules specialize to the family of bodies that the outer loop retains. They fly those bodies farther than the frozen generalist, they do not transfer that gain to bodies selected under the generalist, and they lose their generality along the run (subsection 6.2.2 to subsection 6.2.5).

1.4 Structure of the Thesis

The thesis is organized as follows. The technical background is given in chapter 2: reinforcement learning and Proximal Policy Optimization, recurrent networks and the Long Short-Term Memory, evolutionary algorithms with CMA-ES and NSGA-II, and Hebbian learning with the plasticity rules that evolution can search. chapter 3 opens with the related work on co-design and on evolved plasticity, and with the position of this thesis with respect to both. It then states the three challenges, and the requirement they impose on the method. The simulation is described in chapter 4: the engine, the forest and the flight that is evaluated in it, and the drone with its space of bodies and its reference body. The method is described in chapter 5: the base controllers and their training, the Hebbian controller with the reasons for making the output layer plastic, and the co-design pipeline with its control condition. The experiments are reported in chapter 6: first the inner loop on fixed bodies, with the generalist and with the specialists as base, then the full pipeline against morphology-only evolution, with the analysis of the fronts, of the evolved bodies and of the evolved rules. The conclusions and three directions for future work are in chapter 7. The internal organization of the engine, the complete aerodynamic model and the derivations that the main text leaves out are collected in Appendix A.

Technical Background

This chapter introduces the main techniques used in this work. Each of the methods presented here, in addition to some novelties presented in the following chapters, is a building block of the final system.

2.1 Deep Reinforcement Learning

A common approach to train a deep neural network controller is by using Reinforcement Learning (RL): an agent interacts with an environment through a sequence of actions, and the only training signal it receives is a scalar *reward* that grades how well it is performing. RL rests on the *reward hypothesis*: that any goal can be expressed as the maximization of expected cumulative reward [70]. The presentation of this section follows the textbook of Sutton and Barto [70], to which the reader is referred for the proofs and for a broader treatment.

2.1.1 Markov Decision Processes

At each discrete time step t, the agent observes the state $s_t \in \mathcal{S}$ of the environment and selects an action $a_t \in \mathcal{A}$; the environment responds with a new state s_{t+1} and a scalar reward r_{t+1}, computed by the environment's own, generally unknown, transition and reward functions (Figure 2.1).

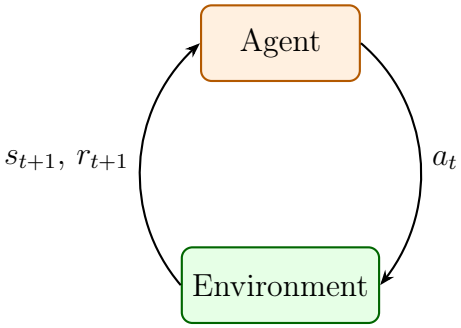


Figure 2.1: The agent-environment interaction loop. The reward r_{t+1} is a scalar with no fixed sign convention beyond larger being better: it is the only feedback the agent receives beyond a partial, possibly noisy observation of the environment.

This loop is assumed to be *Markovian*: the next state and reward depend only on the current state and action, not on the full history that produced them,

$$P(s_{t+1}, r_{t+1} \mid s_t, a_t) = P(s_{t+1}, r_{t+1} \mid s_0, a_0, r_1, \ldots, s_{t-1}, a_{t-1}, r_t, s_t, a_t). \tag{2.1}$$

The Markov property is what lets the current state summarize everything relevant about the past, and is what makes the following formalism, a Markov Decision Process (MDP), tractable.

Definition 2.1.1 (Markov Decision Process [59, 70])**.** *A Markov Decision Process is a tuple* $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$*,*
where:

- $\mathcal{S}$ *is the set of states,*
- $\mathcal{A}$ *is the set of actions,*
- $P(s' \mid s, a) = P(s_{t+1}{=}s' \mid s_t{=}s, a_t{=}a)$ *is the state-transition distribution,*
- $R(s, a) = \mathbb{E}[r_{t+1} \mid s_t{=}s, a_t{=}a]$ *is the expected immediate reward,*
- $\gamma \in [0, 1]$ *is a discount factor.*

The agent's behaviour is described by a policy π, a map from states to actions: a *deterministic* policy is a function $a = \pi(s)$, while a *stochastic* policy $\pi(a \mid s) = P(a_t{=}a \mid s_t{=}s)$ is a conditional distribution over actions. Stochasticity is the source of the exploration needed to discover good actions in the first place, since a purely deterministic agent can never try anything it does not already favour. The RL objective is to find a policy that maximizes the *return*, the total discounted future reward from a given time step,

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \tag{2.2}$$

The discount factor trades off immediate against delayed reward: $\gamma \approx 0$ makes the agent myopic, $\gamma \approx 1$ far-sighted; besides this qualitative role, $\gamma < 1$ is also what keeps G_t finite for a task with no fixed episode length and bounded per-step reward.

Given a policy π, the *state-value function* $v_\pi(s)$ and *action-value function* $q_\pi(s, a)$ are the expected return obtained from a state, or from a state-action pair, when following π thereafter:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid s_t{=}s], \qquad q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t{=}s, a_t{=}a]. \tag{2.3}$$

Because the return decomposes into an immediate reward and a discounted continuation, $G_t = r_{t+1} + \gamma G_{t+1}$, the value functions satisfy a recursive consistency condition, the *Bellman expectation equation*:

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \left(R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s, a)\, v_\pi(s') \right) = \sum_{a \in \mathcal{A}} \pi(a \mid s)\, q_\pi(s, a). \tag{2.4}$$

An *optimal* policy π_* is one whose action-value function $q_*(s, a) = \max_\pi q_\pi(s, a)$ dominates that of every other policy in every state; since some deterministic policy always achieves q_*, namely $\pi_*(s) = \arg\max_a q_*(s, a)$, finding q_* is by itself sufficient to solve the MDP.

When $\mathcal{S}$ and $\mathcal{A}$ are small and finite and (P, R) are fully known, v_π and q_* can be computed exactly, by dynamic-programming methods such as policy iteration or value iteration [3, 37], which apply the Bellman equations above as update rules until convergence (the *model-based* setting). In the *model-free* setting considered

for the rest of this section, which includes every task in this work, P and R are unknown, and value functions must instead be estimated from sampled transitions $(s_t, a_t, r_{t+1}, s_{t+1})$ obtained by actually interacting with the environment. Whenever $\mathcal{S}$, or $\mathcal{A}$, is continuous or otherwise too large to enumerate (as is the case for the sensor observations and continuous control commands used throughout this work), a lookup table, as in tabular Q-Learning [75], is no longer an option, and v_π, q_π, or π itself must instead be represented by parametric function approximators, typically deep neural networks; this is what is meant by *deep* reinforcement learning. The remainder of this section develops the *policy-based* branch of deep RL, which parametrizes the policy directly rather than deriving it from a value function, and which is the branch this work builds on.

2.1.2 Actor-Critic Reinforcement Learning

Policy-based methods [77, 71] parametrize the policy itself, $\pi_{\boldsymbol{\theta}}(a \mid s)$, a deep neural network with parameters $\boldsymbol{\theta}$, and optimize such parameters directly by gradient ascent on the expected return

$$J(\boldsymbol{\theta}) = \mathbb{E}_{\pi_{\boldsymbol{\theta}}}[G_0]\,. \tag{2.5}$$

Compared to deriving a policy from a value function, this tends to have better convergence properties and remains effective in high-dimensional or continuous action spaces. Its cost is that $J(\boldsymbol{\theta})$, and its gradient, can only be evaluated through samples, which tends to be high-variance and can converge to a local rather than a global optimum [70].

The *policy gradient theorem* [71] shows that, despite $J(\boldsymbol{\theta})$ depending on the environment's unknown transition dynamics through the states $\pi_{\boldsymbol{\theta}}$ itself visits, its gradient can be written without any explicit knowledge of those dynamics:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{\pi_{\boldsymbol{\theta}}}[\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a \mid s)\, q_{\pi_{\boldsymbol{\theta}}}(s, a)]\,. \tag{2.6}$$

Intuitively, $\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a \mid s)$ is the direction in parameter space that most increases the probability of action a in state s; weighting it by $q_{\pi_{\boldsymbol{\theta}}}(s, a)$ pushes $\boldsymbol{\theta}$ to raise the probability of actions that lead to high return and lower that of actions that lead to low return.

Actor-critic methods reduce this variance by learning a second, separate approximation of the value function (the *critic*, e.g. $V_{\boldsymbol{w}}(s) \approx v_{\pi_{\boldsymbol{\theta}}}(s)$) alongside the policy, or *actor*, $\pi_{\boldsymbol{\theta}}$ [42]. The critic is trained as in any value-based method, by a temporal difference (TD) update towards $r_{t+1} + \gamma V_{\boldsymbol{w}}(s_{t+1})$, while the actor is updated by policy gradient ascent, using the critic's own estimate of value in place of the true, unknown $q_{\pi_{\boldsymbol{\theta}}}$.

Figure 2.2 depicts both networks reading the same state s_t for simplicity, but nothing in the algorithm requires this symmetry: since only the actor $\pi_{\boldsymbol{\theta}}$ is kept for deployment while the critic $V_{\boldsymbol{w}}$ is discarded once training ends, the critic is free to observe a wider, *privileged* state that the actor never sees: quantities that are cheap to obtain in simulation but unavailable, or unreliable, on the deployed system. Because such an asymmetric critic only ever shapes the actor's parameter

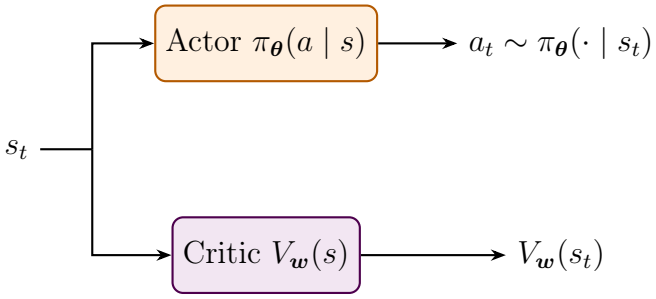


Figure 2.2: The actor-critic architecture: two networks read the same state s_t, the actor to sample an action, the critic to estimate its value $V_{\boldsymbol{w}}(s_t)$. The TD error between $V_{\boldsymbol{w}}(s_t)$ and $r_{t+1} + \gamma V_{\boldsymbol{w}}(s_{t+1})$, observed once the environment responds, is what trains the critic by regression and reweights the actor's policy-gradient update.

updates and never appears in its computation graph at test time, this cannot leak information the actor could not otherwise access; it simply gives the critic a less noisy value estimate to train against, which in turn lowers the variance of the advantage used to update the actor.

A further variance reduction method consists in subtracting a state-dependent baseline from the critic's estimate without introducing any bias: since, for any fixed s, $\mathbb{E}_{a\sim\pi_{\boldsymbol{\theta}}}[\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a \mid s)] = 0$, replacing $q_{\pi_{\boldsymbol{\theta}}}(s,a)$ in the policy gradient with the *advantage function*

$$A_\pi(s,a) = q_\pi(s,a) - v_\pi(s) \tag{2.7}$$

leaves $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ unchanged in expectation while typically reducing its variance substantially, since $A_\pi(s,a)$ measures how much better action a is than the policy's own average behaviour in s, rather than the absolute, and generally much larger, magnitude of the return itself. Computing $A_\pi(s,a) = q_\pi(s,a) - v_\pi(s)$ exactly would still require two separate value functions, but the single TD error $\delta_t = r_{t+1} + \gamma V_{\boldsymbol{w}}(s_{t+1}) - V_{\boldsymbol{w}}(s_t)$ is already, on its own, a one-sample estimate of the advantage: it substitutes the critic's own $V_{\boldsymbol{w}}$ for $v_\pi(s_t)$, and the sampled $r_{t+1} + \gamma V_{\boldsymbol{w}}(s_{t+1})$ for $q_\pi(s_t, a_t)$.

Relying on a single δ_t, however, looks only one step ahead, so the estimate inherits whatever bias $V_{\boldsymbol{w}}$ currently has; summing several consecutive TD errors looks further into the future and averages more of that bias away, at the cost of compounding sampling variance the further the sum reaches. Generalized Advantage Estimation (GAE) [65] controls this trade-off directly, combining successive TD errors into an exponentially-weighted sum governed by a decay parameter $\lambda \in [0,1]$:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}. \tag{2.8}$$

GAE interpolates between the high-bias, low-variance one-step estimate $\hat{A}_t = \delta_t$ (λ=0) and the low-bias, high-variance Monte Carlo estimate obtained by summing every future TD error (λ=1). This is the same bias-variance trade-off, controlled by the same kind of exponentially-decaying trace, that TD(λ) eligibility traces implement for the value function itself [70]. The actor and critic are typically two

networks, or two heads of a partly shared network, trained jointly end to end: the critic by regression towards its TD target, the actor by advantage-weighted policy gradient ascent.

2.1.3 Proximal Policy Optimization

The actor-critic parameter update above does not, by itself, say how large a step to take. Each batch of collected experience is only informative about the policy $\pi_{\boldsymbol{\theta}_{\text{old}}}$ that generated it, yet the gradient estimate computed from it is noisy; an update that moves $\boldsymbol{\theta}$ too far from $\boldsymbol{\theta}_{\text{old}}$ because of the strength of that noisy estimate can push $\pi_{\boldsymbol{\theta}}$ into a region of the parameter space the batch says nothing reliable about, collapsing performance in a way that plain gradient ascent has no mechanism to prevent. Trust Region Policy Optimization (TRPO) [64] addresses this directly, by constraining each update to a region, measured by KL divergence between $\pi_{\boldsymbol{\theta}}$ and $\pi_{\boldsymbol{\theta}_{\text{old}}}$, within which the current batch is trusted to be informative, at the cost of a second-order constrained-optimization step that is expensive computationally.

Proximal Policy Optimization (PPO) [66] achieves a similar effect with an unconstrained, first-order objective, optimizable directly by ordinary stochastic gradient ascent, which is what has made it, in practice, a default choice for on-policy deep RL. Let

$$r_t(\boldsymbol{\theta}) = \frac{\pi_{\boldsymbol{\theta}}(a_t \mid s_t)}{\pi_{\boldsymbol{\theta}_{\text{old}}}(a_t \mid s_t)} \tag{2.9}$$

be the probability ratio between the policy being optimized and the fixed, older policy that generated the batch (so that $r_t(\boldsymbol{\theta}_{\text{old}}) = 1$), and let $\hat{A}_t$ be an advantage estimate, e.g. GAE. The plain policy-gradient surrogate $\mathbb{E}_t[r_t(\boldsymbol{\theta})\hat{A}_t]$ has the correct gradient at $\boldsymbol{\theta} = \boldsymbol{\theta}_{\text{old}}$, but nothing stops $r_t(\boldsymbol{\theta})$ from drifting arbitrarily far from 1 once optimized repeatedly on the same batch. PPO instead maximizes the *clipped surrogate objective*

$$L^{\text{CLIP}}(\boldsymbol{\theta}) = \mathbb{E}_t\Big[\min\big(r_t(\boldsymbol{\theta})\,\hat{A}_t,\ \text{clip}(r_t(\boldsymbol{\theta}),\,1-\epsilon,\,1+\epsilon)\,\hat{A}_t\big)\Big], \tag{2.10}$$

where ϵ, typically between 0.1 and 0.2, sets the width of the trust region. When $\hat{A}_t > 0$, the clip caps the objective's growth once $r_t(\boldsymbol{\theta})$ exceeds $1+\epsilon$, removing any incentive to keep increasing $\pi_{\boldsymbol{\theta}}(a_t \mid s_t)$ further on this particular batch; symmetrically, when $\hat{A}_t < 0$, it caps the objective once $r_t(\boldsymbol{\theta})$ falls below $1-\epsilon$. The outer minimum makes L^{CLIP} a pessimistic (lower) bound on the unclipped surrogate, so clipping only ever activates in the direction that would otherwise make the update too aggressive, never in the direction that would make it too conservative. In practice, this first-order clipping behaves much like TRPO's explicit trust region, at a fraction of its implementation and computational cost.

The actor and critic are trained jointly, by combining L^{CLIP} with a value-function regression loss for the critic, $L^{\text{VF}}(\boldsymbol{\theta}) = \big(V_{\boldsymbol{\theta}}(s_t) - (\hat{A}_t + V_{\boldsymbol{\theta}_{\text{old}}}(s_t))\big)^2$, and an entropy bonus $S[\pi_{\boldsymbol{\theta}}](s_t)$ that discourages the policy from collapsing prematurely onto a deterministic one, into a single objective

$$L(\boldsymbol{\theta}) = L^{\text{CLIP}}(\boldsymbol{\theta}) - c_1 L^{\text{VF}}(\boldsymbol{\theta}) + c_2 S[\pi_{\boldsymbol{\theta}}](s_t), \tag{2.11}$$

maximized by a handful of epochs of minibatch stochastic gradient ascent over each freshly collected batch of trajectories [66]. Once these epochs are done the batch is discarded, $\boldsymbol{\theta}_{\text{old}} \leftarrow \boldsymbol{\theta}$ is refreshed, and a new batch is collected with the updated policy: an on-policy training loop that strictly alternates environment interaction under a fixed policy with optimization of that same policy on the data it just produced.

2.2 Long Short-Term Memory Networks

The material of this section is standard: a textbook treatment of recurrent networks, of the difficulty of training them over long sequences and of the gated architectures that address it is given by Goodfellow, Bengio and Courville [26].

2.2.1 Recurrent Neural Networks

Since the state-describing vectors are often inaccessible to a controller, and the only accessible knowledge is the sequence of observations, such settings are formally *Partially Observable Markov Decision Processes* (POMDPs) [40]. A recurrent neural network (RNN) bridges this gap by maintaining an internal state that captures information about the past. It processes a sequence $\boldsymbol{x}_1, \dots, \boldsymbol{x}_T$, that might represent a time series or a sequence of observations itself, and maintains a state $\boldsymbol{h}_t$ that is updated at every step from the current input and the previous state [15]:

$$\boldsymbol{h}_t = g(\boldsymbol{U}\boldsymbol{h}_{t-1} + \boldsymbol{W}\boldsymbol{x}_t + \boldsymbol{b}), \qquad \boldsymbol{h}_t \in \mathbb{R}^{d_h},\ \boldsymbol{x}_t \in \mathbb{R}^{d_{in}},\ \boldsymbol{U} \in \mathbb{R}^{d_h \times d_h},\ \boldsymbol{W} \in \mathbb{R}^{d_h \times d_{in}}, \tag{2.12}$$

with g a pointwise nonlinearity (typically tanh). Unrolled over time, this is a feed-forward computation with the same weights $\boldsymbol{U}, \boldsymbol{W}$ shared across every step, trained end-to-end by backpropagation through time (BPTT) [76]: a forward pass computes every $\boldsymbol{h}_t$ in temporal order, and a backward pass propagates gradients from the last step back to the first, through the same recurrent weight matrix $\boldsymbol{U}$ at every step.

2.2.2 Vanishing Gradient Problem

This repeated multiplication by $\boldsymbol{U}$ is also the vanilla RNN's main weakness. Informally, the gradient of the loss at step T with respect to an early state $\boldsymbol{h}_t$ scales, over the $T - t$ intervening steps, with a product of $T - t$ Jacobians of the state-transition function, each dominated by $\boldsymbol{U}$; if the dominant eigenvalue of $\boldsymbol{U}$ is below 1 this product shrinks geometrically with $T - t$ (*vanishing gradient*), and if it is above 1 it grows geometrically (*exploding gradient*) [54]. In the vanishing case, the influence of inputs far in the past disappears from the state before training can pick up on it, which makes vanilla RNNs difficult to train on long-range dependencies. A first fix is to make the state update a convex combination, $\boldsymbol{h}_t = \alpha\, \boldsymbol{h}_{t-1} + (1 - \alpha) \tanh(\boldsymbol{U}\boldsymbol{h}_{t-1} + \boldsymbol{W}\boldsymbol{x}_t)$, so that a fraction α of the state is carried over unchanged instead of being fully overwritten at every step; the Long Short-Term Memory (LSTM, below) [35] can be read as turning this fixed, hand-set α into a learned, input-dependent, per-dimension quantity.

2.2.3 Long Short-Term Memory

Long Short-Term Memory (LSTM) networks address the vanishing/exploding gradient problem by keeping two separate pieces of recurrent state instead of one, and by making the recurrence of the long-term one additive rather than a repeated matrix multiplication:

- the *hidden state* $\boldsymbol{h}_t \in \mathbb{R}^{d_h}$ plays the same role as the state of a vanilla RNN: it is what is read out for prediction or further processing at step t;
- the *cell state* $\boldsymbol{c}_t \in \mathbb{R}^{d_h}$ is an internal memory that carries information across time-steps, and is only ever combined with the rest of the network through an elementwise multiplication and an elementwise addition, never a matrix product. This is what keeps the gradient with respect to $\boldsymbol{c}_t$ from vanishing or exploding as it is backpropagated through time.

What is written into $\boldsymbol{c}_t$, what is dropped from it, and what part of it is exposed through $\boldsymbol{h}_t$ is decided at every step by three *gates*.

Definition 2.2.1 (Gate)**.** *A gate is a standard affine layer of $\boldsymbol{h}_{t-1}$ and $\boldsymbol{x}_t$ followed by a sigmoid activation, used to modulate a signal by elementwise multiplication:*

$$\boldsymbol{\gamma}_t = \sigma(\boldsymbol{U}_\gamma \boldsymbol{h}_{t-1} + \boldsymbol{W}_\gamma \boldsymbol{x}_t + \boldsymbol{b}_\gamma) \in (0,1)^{d_h}, \tag{2.13}$$

so that, for any signal $\boldsymbol{v} \in \mathbb{R}^{d_h}$, $\boldsymbol{\gamma}_t \odot \boldsymbol{v}$ lets a component of $\boldsymbol{v}$ through unchanged where the corresponding component of $\boldsymbol{\gamma}_t$ is close to 1 and suppresses it where that component is close to 0.

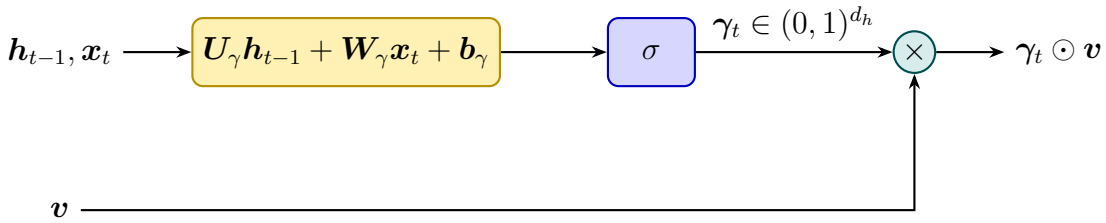


Figure 2.3: A gate: an affine map of the previous hidden state and the current input, squashed to $(0,1)$ by a sigmoid, used to modulate a signal $\boldsymbol{v}$ by elementwise multiplication.

The structure of a single gate is shown in Figure 2.3. An LSTM cell uses three such gates, each with its own parameters:

- the **forget gate** $\boldsymbol{f}_t$ decides which part of the old cell state $\boldsymbol{c}_{t-1}$ is kept;
- the **input gate** $\boldsymbol{i}_t$ decides which part of a candidate update $\boldsymbol{g}_t$ (computed exactly as a vanilla RNN state would be) is written into the cell state;
- the **output gate** $\boldsymbol{o}_t$ decides which part of the cell state is exposed as the new hidden state $\boldsymbol{h}_t$.

$$\boldsymbol{f}_t = \sigma(\boldsymbol{U}_f \boldsymbol{h}_{t-1} + \boldsymbol{W}_f \boldsymbol{x}_t + \boldsymbol{b}_f) \quad \text{forget gate} \tag{2.14}$$

$$\boldsymbol{i}_t = \sigma(\boldsymbol{U}_i \boldsymbol{h}_{t-1} + \boldsymbol{W}_i \boldsymbol{x}_t + \boldsymbol{b}_i) \quad \text{input gate} \tag{2.15}$$

$$\boldsymbol{g}_t = \tanh(\boldsymbol{U}_g \boldsymbol{h}_{t-1} + \boldsymbol{W}_g \boldsymbol{x}_t + \boldsymbol{b}_g) \quad \text{candidate cell content} \tag{2.16}$$

$$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \boldsymbol{g}_t \quad \text{cell state update} \tag{2.17}$$

$$\boldsymbol{o}_t = \sigma(\boldsymbol{U}_o \boldsymbol{h}_{t-1} + \boldsymbol{W}_o \boldsymbol{x}_t + \boldsymbol{b}_o) \quad \text{output gate} \tag{2.18}$$

$$\boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t) \quad \text{hidden state update} \tag{2.19}$$

where $\odot$ denotes the elementwise product. Each gate, and the candidate $\boldsymbol{g}_t$, has its own weight matrices $\boldsymbol{U}_\square, \boldsymbol{W}_\square$ and bias $\boldsymbol{b}_\square$, learned jointly with the rest of the network: a single LSTM layer of hidden size d_h therefore holds four sets of parameters shaped like a single vanilla-RNN layer's, one each for $\boldsymbol{f}_t, \boldsymbol{i}_t, \boldsymbol{g}_t, \boldsymbol{o}_t$.

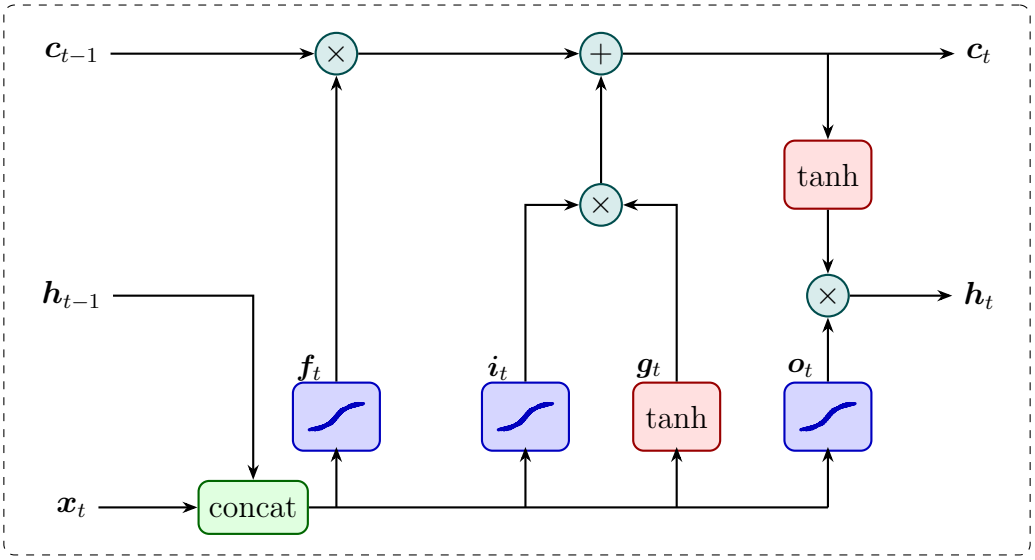


Figure 2.4: Structure of an LSTM cell. The top rail carries the cell state $\boldsymbol{c}_t$ across time-steps through only an elementwise multiplication (forget gate $\boldsymbol{f}_t$) and an elementwise addition (gated candidate $\boldsymbol{i}_t \odot \boldsymbol{g}_t$), with no matrix product in between: this is the property that keeps gradients from vanishing or exploding along this path. The hidden state $\boldsymbol{h}_t$ is a gated, squashed read-out of $\boldsymbol{c}_t$, and $\boldsymbol{h}_{t-1}, \boldsymbol{x}_t$ jointly drive all four gates/candidate at the bottom.

Figure 2.4 makes explicit why this design addresses the problem laid out in the previous section: the path carrying $\boldsymbol{c}_t$ across time-steps involves only elementwise operations gated by $\boldsymbol{f}_t$ and $\boldsymbol{i}_t \odot \boldsymbol{g}_t$, with no repeated multiplication by a shared weight matrix. Backpropagating through this path therefore does not force the gradient to contract or expand geometrically with the number of time-steps, which is what allows an LSTM to learn dependencies over much longer sequences than a vanilla RNN.

2.3 Evolutionary Algorithms

Evolutionary Algorithms (EAs) are a family of population-based, gradient-free optimization methods loosely inspired by natural selection [14]. Where a gradient method follows the slope of the objective from a single point, an EA maintains a whole *population* of candidate solutions, or *individuals*, each encoded by a vector of parameters called its *genome*, and repeats a simple loop over successive *generations*:

1. *Evaluation*: every individual is assigned a *fitness*, a scalar score obtained by actually running it on the task;
2. *Selection*: the fitter individuals are chosen as *parents*, so that good genomes get to reproduce more than bad ones;
3. *Variation*: new individuals, the *offspring*, are produced from the parents by *mutation* (a random perturbation of a single genome) and *recombination*, or *crossover*, which mixes the genomes of two parents;
4. *Replacement*: the offspring form the next population, possibly alongside the best individuals of the current one (*elitism*), so that a good solution, once found, is never lost.

Over many generations, selection pressure concentrates the population in high-fitness regions of the search space, while variation keeps exploring around it.

The appeal of this recipe is that it treats the objective as a black box: nothing is assumed about it beyond the ability to evaluate it. It needs no gradient, no differentiability, and no smoothness, and it is indifferent to whether the genome is continuous, discrete, or mixed. This makes EAs the natural tool whenever the quantity being optimized is only defined through a complete simulation and, more generally, whenever gradient-based training, such as the RL methods of the previous section, cannot reach the parameters of interest. The price is sample efficiency: every generation costs as many full evaluations as there are individuals, which is affordable mainly when evaluations can be run in parallel, as is the case for GPU-batched simulation. Two EAs are used in this work, one for each of the two settings they were designed for: CMA-ES for single-objective optimization over continuous parameters, and NSGA-II for problems with several competing objectives. The general presentation above follows the textbook of Eiben and Smith [14]; the two subsections below follow, respectively, the tutorial of Hansen [30] and the book of Deb [8].

2.3.1 CMA-ES

Evolution Strategies (ES) [61, 5] are the branch of EAs specialized to continuous search spaces, $\boldsymbol{x} \in \mathbb{R}^n$. In an ES, variation is not a discrete crossover but a Gaussian perturbation: at every generation, λ offspring are sampled around a single mean vector $\boldsymbol{m}$,

$$\boldsymbol{x}_k = \boldsymbol{m} + \sigma\, \boldsymbol{z}_k, \qquad \boldsymbol{z}_k \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{C}), \qquad k = 1, \ldots, \lambda, \tag{2.20}$$

where $\sigma > 0$ is a global *step size* and $\boldsymbol{C} \in \mathbb{R}^{n \times n}$ is a covariance matrix that shapes the cloud of samples. The offspring are evaluated and sorted by fitness, and the mean is moved to a weighted average of the best $\mu < \lambda$ of them,

$$\boldsymbol{m} \leftarrow \sum_{i=1}^{\mu} w_i \, \boldsymbol{x}_{i:\lambda}, \qquad w_1 \geq w_2 \geq \cdots \geq w_\mu > 0, \qquad \sum_{i=1}^{\mu} w_i = 1, \tag{2.21}$$

where $\boldsymbol{x}_{i:\lambda}$ denotes the i-th best of the λ samples. Note that only the *ranking* of the fitness values enters the update, never their magnitude: the algorithm is therefore invariant to any monotone rescaling of the objective, and requires no normalization of the reward. The flip side is that, when the fitness is a noisy estimate (as the outcome of a stochastic simulation is), what degrades the update is precisely noise large enough to reorder the candidates, and the standard remedies are to enlarge λ or to average each individual's fitness over repeated evaluations.

With σ and $\boldsymbol{C}$ fixed, this is a very simple algorithm, whose performance depends entirely on whether the cloud of samples happens to match the local shape of the fitness landscape: a narrow, tilted valley, for instance, forces an isotropic Gaussian to take tiny steps, since most large steps fall off the valley walls. The Covariance Matrix Adaptation Evolution Strategy (CMA-ES) [32, 30] removes this dependence by adapting both σ and $\boldsymbol{C}$ online, from the history of the steps that turned out to be successful, so that the sampling distribution itself learns the shape of the landscape; Figure 2.5 illustrates the effect.

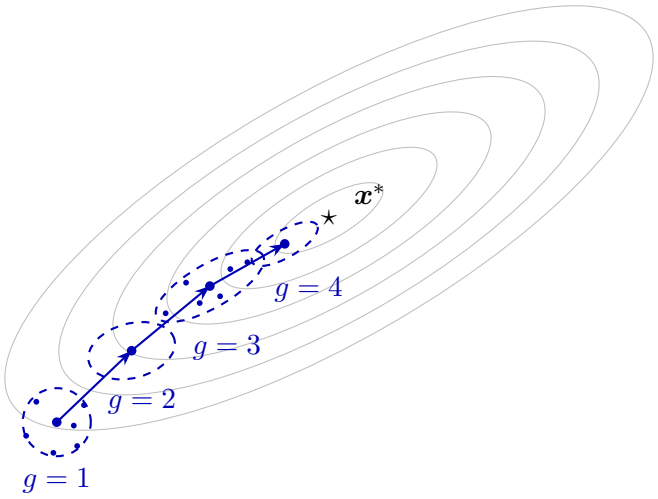


Figure 2.5: Covariance adaptation in CMA-ES on a fitness landscape shaped like a narrow, tilted valley (grey contours; the optimum is $\boldsymbol{x}^*$). Each dashed ellipse is the one-standard-deviation contour of the sampling distribution $\mathcal{N}(\boldsymbol{m}, \sigma^2 \boldsymbol{C})$ at generation g, and the dots are its samples. The search starts isotropic ($\boldsymbol{C} = \boldsymbol{I}$), but the successful samples of each generation lie preferentially along the valley, and the covariance update progressively stretches the distribution in that direction: by $g = 3$ the algorithm is taking long steps along the valley and short ones across it, which an isotropic search could never do.

The covariance is updated, after every generation, towards the directions in which the selected offspring were found. Writing $\boldsymbol{y}_{i:\lambda} = (\boldsymbol{x}_{i:\lambda} - \boldsymbol{m})/\sigma$ for the normalized

step that produced the i-th best sample, the update reads

$$\boldsymbol{C} \leftarrow (1 - c_1 - c_\mu)\,\boldsymbol{C} \;+\; c_1\,\boldsymbol{p}_c\,\boldsymbol{p}_c^{\top} \;+\; c_\mu \sum_{i=1}^{\mu} w_i\,\boldsymbol{y}_{i:\lambda}\,\boldsymbol{y}_{i:\lambda}^{\top}, \tag{2.22}$$

with small learning rates $c_1, c_\mu \in (0, 1)$. The last term is the weighted empirical covariance of this generation's successful steps: it stretches the distribution along the directions the good samples came from, and shrinks it along the others. The middle term does the same across generations, through the *evolution path* $\boldsymbol{p}_c$, an exponentially-decaying running sum of the successive displacements of the mean: a direction in which $\boldsymbol{m}$ keeps moving, generation after generation, is amplified, while directions in which it merely oscillates cancel out. On a smooth objective the net effect is that $\boldsymbol{C}$ comes to approximate the inverse Hessian near the optimum, so that CMA-ES behaves much like a second-order method, without ever computing a derivative.

The step size is adapted separately, by *cumulative step-size adaptation*: a second evolution path accumulates the displacements of the mean, rescaled by $\boldsymbol{C}^{-1/2}$ so that, if selection were purely random, its length would be that of a random walk. If consecutive steps point in a consistent direction the path grows longer than a random walk would, which signals that the steps are too short and σ is increased; if they largely cancel out, the path is shorter than expected and σ is decreased. This lets the algorithm take large steps while far from the optimum and contract progressively as it closes in on it. The learning rates, the weights w_i, and the population size $\lambda = 4 + \lfloor 3 \ln n \rfloor$ all have default values that work robustly across problems, so that the number of parameters the user must provide is reduced to just the initial mean $\boldsymbol{m}^{(0)}$ and step size $\sigma^{(0)}$: this near absence of tuning is a large part of why CMA-ES is the default choice for black-box optimization in moderate dimensions.

2.3.2 NSGA-II

Many design problems do not have a single objective. A plane should travel far *and* consume little energy; a structure should be light *and* stiff. Typically such objectives conflict, so that improving one worsens the other, and no single solution is best in every respect. *Multi-objective optimization* [50, 8] makes this explicit by asking to minimize a vector of M objectives at once, $\boldsymbol{f}(\boldsymbol{x}) = \big(f_1(\boldsymbol{x}), \ldots, f_M(\boldsymbol{x})\big)$, where an objective to be maximized is handled simply by negating it. Since there is no natural total order among vectors, the notion of "better" has to be replaced by a partial one:

Definition 2.3.1 (Pareto dominance)**.** *A solution* $\boldsymbol{x}$ dominates *a solution* $\boldsymbol{x}'$*, written* $\boldsymbol{x} \prec \boldsymbol{x}'$*, if it is no worse in every objective and strictly better in at least one:*

$$f_i(\boldsymbol{x}) \le f_i(\boldsymbol{x}') \;\; \forall\, i \in \{1, \ldots, M\} \qquad \textit{and} \qquad f_j(\boldsymbol{x}) < f_j(\boldsymbol{x}') \;\; \textit{for some } j. \tag{2.23}$$

The representation of such dominance can be visualized in a two-dimensional space in Figure 2.6(a). A solution that is dominated by no other is *Pareto-optimal*; the set of all such solutions is the *Pareto set*, and its image in objective space is the *Pareto front*. Every point on the front is a trade-off that cannot be improved in

one objective without giving something up in another, and which of them is "best" is not a question the optimizer can answer: it is left to the designer, once the front is known. The goal of a multi-objective optimizer is therefore not one point but an approximation of the whole front: a set of solutions that lies close to it (*convergence*) and is spread evenly along it (*diversity*). One could obtain this by fixing a weighted sum of the objectives and running a single-objective optimizer once per choice of weights, but that requires choosing the weights in advance, and a weighted sum cannot reach the points of a front where it bends inwards (its non-convex parts). A population-based method, in contrast, already maintains a set of solutions, and can be steered to spread that set along the front within a single run; this is why EAs are the dominant approach to multi-objective optimization [8].

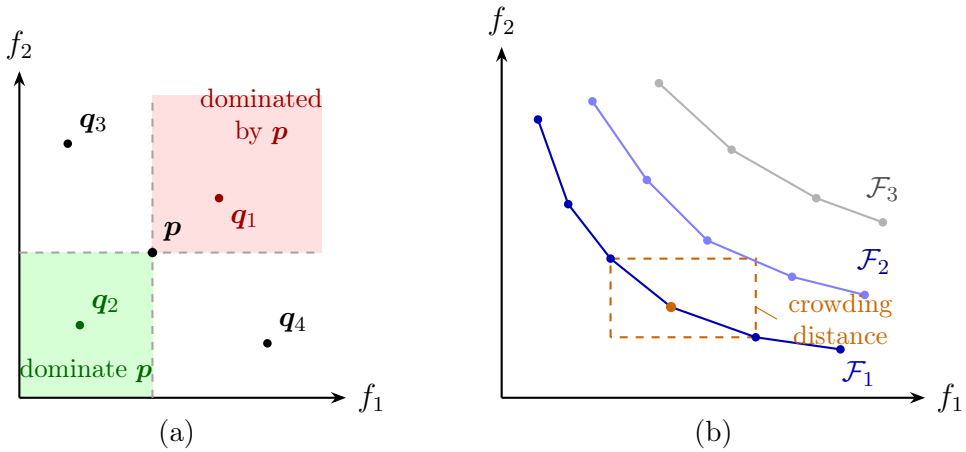


Figure 2.6: Two objectives, both to be minimized. (a) Pareto dominance: $\boldsymbol{p}$ dominates every point in the shaded red region ($\boldsymbol{q}_1$), is dominated by every point in the green one ($\boldsymbol{q}_2$), and is *incomparable* with points such as $\boldsymbol{q}_3$ and $\boldsymbol{q}_4$, each of which is better than $\boldsymbol{p}$ in one objective and worse in the other. (b) Non-dominated sorting of a population into fronts $\mathcal{F}_1 \prec \mathcal{F}_2 \prec \mathcal{F}_3$: $\mathcal{F}_1$ contains the points dominated by no other, $\mathcal{F}_2$ those dominated only by members of $\mathcal{F}_1$, and so on. The dashed box is the crowding cuboid of the orange point of $\mathcal{F}_1$: the rectangle spanned by its nearest neighbours along each objective, whose side lengths sum to its crowding distance. A point with a large cuboid sits in a sparsely populated part of its front, and is preferred over one in a crowded region.

The Non-dominated Sorting Genetic Algorithm II (NSGA-II) [10] is the most widely used such method. It is a genetic algorithm in the sense of section 2.3: for real-valued genomes, the standard variation operators are simulated binary crossover (SBX) and polynomial mutation [9], both of which produce offspring in the neighbourhood of their parents. What distinguishes it is the selection step, which is built on two ingredients, both illustrated in Figure 2.6(b).

The first is *non-dominated sorting*, which partitions the population into a sequence of *fronts* $\mathcal{F}_1, \mathcal{F}_2, \ldots$: $\mathcal{F}_1$ is the set of individuals that no other individual dominates, $\mathcal{F}_2$ the set of those that become non-dominated once $\mathcal{F}_1$ is removed, and so on until every individual has been assigned to a front. The index of an individual's front is its *rank*, and lower is better: $\mathcal{F}_1$ is the current approximation of the

Pareto front, $\mathcal{F}_2$ the second-best layer behind it. The "fast" procedure of NSGA-II computes this partition in $O(MN^2)$ time for a population of N individuals, which is negligible next to the cost of evaluating them.

The second is the *crowding distance*, which breaks ties among individuals of the same front in favour of diversity. For each objective, the front is sorted along that objective and each individual is assigned the distance between its two neighbours, normalized by the objective's range; the crowding distance is the sum of these values over all objectives, and the two extreme individuals of each front, which have only one neighbour, receive an infinite value so that they are always kept. Geometrically, it measures the size of the largest box around an individual that contains no other member of its front: a large value marks an isolated individual in a sparsely covered region, which is exactly where the front's approximation most needs improving.

The two are combined into a single *crowded-comparison* rule: between two individuals, the one with the lower rank wins, and if the ranks are equal, the one with the larger crowding distance does. A generation of NSGA-II then proceeds as follows. Parents are chosen from the population P of size N by binary tournaments under this rule, and produce N offspring Q by crossover and mutation. Parents and offspring are merged into a pool $P \cup Q$ of size $2N$, the pool is sorted into fronts, and the next population is filled with whole fronts in order, $\mathcal{F}_1$ first, until the next front no longer fits; that last front is truncated to the remaining slots by keeping its most isolated members, in descending order of crowding distance. Merging parents and offspring makes the algorithm elitist (a non-dominated individual can only be displaced by one that dominates it or by a more isolated peer), while the crowding distance keeps the population spread along the front without any additional parameter to tune, which is the combination that made NSGA-II the reference algorithm for problems with two or three objectives.

Since the output of a multi-objective run is a set rather than a number, comparing two runs, or tracking a single run across generations, requires a scalar measure of front quality. The most common is the *hypervolume* indicator [78]: the volume of objective space (an area, when $M = 2$) that is dominated by the front and bounded by a fixed *reference point*, chosen to be worse than every solution in every objective. A front that lies closer to the true Pareto front, or covers a wider portion of it, dominates a larger region, so the hypervolume rewards convergence and diversity at once; it is only meaningful, however, when the same reference point is used for every front being compared.

2.4 Hebbian Learning

Hebbian learning [33] is a different, and much older, view of what learning in a neural network can be: a synaptic weight changes as a consequence of the activity that flows through it, according to a *local* rule that involves no error signal and needs no supervision, and keeps doing so for as long as the network is running. The mathematical treatment of the first two subsections follows Gerstner and Kistler [24, 25].

2.4.1 Hebb's Postulate

The idea originates in neuropsychology. Hebb [33] proposed, as the mechanism by which associations form in the brain, that

> When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased,

a statement usually compressed into the slogan *"cells that fire together wire together"*. The postulate says nothing about what the strengthened connection is *for*: the synapse strengthens simply because the two neurons it links have repeatedly been active at the same time.

For a rate-based neuron model (the one used by artificial neural networks), the simplest formalization considers a single postsynaptic unit with scalar output y, receiving presynaptic activities $\boldsymbol{x} \in \mathbb{R}^n$ through a weight vector $\boldsymbol{w} \in \mathbb{R}^n$, $y = \boldsymbol{w}^\top \boldsymbol{x}$, and lets each weight change in proportion to the product of the activities at its two ends:

$$\Delta w_j = \eta \, x_j \, y, \qquad j = 1, \ldots, n, \tag{2.24}$$

with $\eta > 0$ a small *learning rate*. For a whole layer with input $\boldsymbol{x} \in \mathbb{R}^n$, output $\boldsymbol{y} \in \mathbb{R}^m$ and weight matrix $\boldsymbol{W} \in \mathbb{R}^{m \times n}$, the same rule reads, in matrix form, as an outer product:

$$\Delta \boldsymbol{W} = \eta \, \boldsymbol{y} \, \boldsymbol{x}^\top, \qquad \Delta W_{ij} = \eta \, y_i \, x_j. \tag{2.25}$$

The dominant property of (2.24), shared by the whole family of rules that descends from it, is *locality*. The update of W_{ij} depends only on x_j and y_i, the two activities physically present at the synapse, and on nothing else: not on the activity of other neurons, not on the state of other layers, and not on any measure of how well the network is doing. By contrast, with backpropagation the gradient of a loss L with respect to the same weight, $\partial L / \partial W_{ij} = \delta_i \, x_j$, has the same presynaptic factor x_j but replaces the postsynaptic activity y_i with an error term δ_i that is computed at the output of the network and transported backwards through every downstream layer (Figure 2.7). A Hebbian update can therefore be applied at every forward pass, during the normal operation of the network.

2.4.2 The Generalized Hebbian Rule

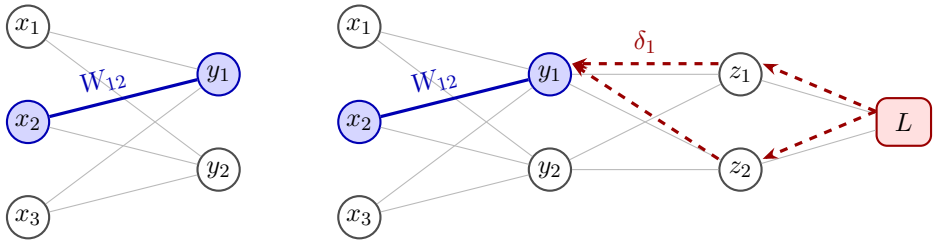
A local rule is any function $F(\cdot)$ of the quantities available at the synapse:

$$\Delta W_{ij} = \eta \, F(x_j, y_i, W_{ij})$$

Gerstner and Kistler [24] note that, for small activities, any such F can be expanded in a Taylor series around $x_j = y_i = 0$, and that truncating the expansion after the bilinear term gives the most general rule that is at most linear in each activity:

$$\Delta W_{ij} = \eta \left(A_{ij} \, x_j \, y_i + B_{ij} \, x_j + C_{ij} \, y_i + D_{ij} \right), \tag{2.26}$$

usually referred to as the *ABCD rules* [53, 69]. Each coefficient has a distinct meaning:



(a) (b)

Figure 2.7: Locality of the Hebbian update. (a) Under Hebb's rule, the change of the highlighted weight W_{12} is a function of the two activities at its ends, x_2 and y_1, and of nothing else; it can be computed in the forward pass, at the synapse. (b) Under backpropagation, the same weight's gradient keeps the presynaptic factor x_2 but replaces y_1 with an error δ_1, which only exists once a loss L has been evaluated at the output and its gradient carried back (dashed) through every layer downstream of the synapse.

- A is the *correlation* term, Hebb's rule itself. A positive A strengthens a synapse when its two ends are simultaneously active, a negative one weakens it, which decorrelates the neuron's output from that input;
- B is the *presynaptic* term: the weight changes whenever the input is active, regardless of what the neuron does. With $B < 0$, an input that keeps firing without contributing to the output is progressively weakened, the "pre active, post silent" form of suppression;
- C is the *postsynaptic* term: the weight changes whenever the neuron fires, regardless of that particular input. With $C < 0$, every synapse of a neuron is weakened each time it fires, a normalization that redistributes strength from the inputs that stay silent to those that actually drive it;
- D is a constant drift, independent of activity: a steady creep of the weight in one direction, which can act as a bias, or, with the opposite sign to the weight, as a crude decay.

It should be noted that in case of repeated activations, the rule may lead to a runaway effect, where the weights grow without bound. This is particularly true for the A term, which can cause a positive feedback loop if the inputs are consistently active. To overcome this problem, various normalization techniques can be applied, such as weight clipping or weight decay. While weight clipping simply limits the maximum value a weight can take, weight decay introduces a small penalty to the weight values, effectively reducing them over time, in a fashion similar to regularization in supervised learning:

$$\Delta W_{ij} = \eta\,(A_{ij}\, x_j\, y_i + B_{ij}\, x_j + C_{ij}\, y_i + D_{ij}) - \lambda W_{ij}$$

The coefficients can be shared by an entire layer, as one rule for all the synapses, or, as written in (2.26), be a separate matrix each, of the same shape as $\boldsymbol{W}$, so that

every synapse follows its own rule [53, 52]. In the latter case a layer of $m \times n$ weights is described by $4mn$ coefficients, and the learning rate η may itself be per synapse. If η is not fixed but modulated by a dedicated signal, this is called *neuromodulation* [69]. The point of this parametrization is that the same template, with different coefficients, produces qualitatively different behaviours: a pure correlation detector, a decorrelating one, a synapse that only ever decays, one that potentiates and then forgets. Which of them is appropriate at a given synapse is not something the rule can decide, since it has no access to the task; the coefficients are the knobs through which a purpose is imposed on a rule that is otherwise blind to it.

2.4.3 Plasticity Rules as the Object of Optimization

In a *plastic* network, the weights are no longer the parameters that define the network: they are part of its state, initialized to some value at the beginning of a *lifetime*, either an episode or the deployment itself, and driven from then on by the rules, along a trajectory that depends on the inputs the network happens to receive. What stays fixed across lifetimes, and is therefore what actually defines the network, are the rules, the rate η and the initial weights. The consequence is that the same network, placed in two different environments, ends up with two different sets of weights, each shaped by the activity that environment induced. This is the mechanism by which a plastic network can adapt, within a lifetime and without any gradient, to conditions that were not present when it was designed, and at the same time the reason a rule is not a solution: it is a procedure, which only becomes useful once its coefficients are such that the weight trajectory it produces improves the network's behaviour rather than degrading it.

Finding such coefficients is an optimization problem, with an unusual objective: the quality of a rule is the performance of the network at the end of, or throughout, a lifetime of applying it, which depends on the coefficients only through the whole sequence of weight updates and of the outputs those weights produced. This is a form of *meta-learning* in which the outer level searches over learning rules and the inner level is the lifetime of the network under one set of them. The objective can in principle be differentiated through the chain of updates [48], but when the lifetime is a closed-loop interaction with an environment, as in control, the dependence of the outcome on the coefficients is long, indirect, and only defined by running it. In such cases it is possible to treat them as a black box and search them with the evolutionary algorithms of section 2.3. Networks obtained this way, whose genome encodes rules rather than weights, are known as *evolved plastic artificial neural networks*, or EPANNs [20, 68].

Related Work & Challenges

The setting of this thesis is a winged drone flying through a forest of obstacles. The drone is evaluated on two quantities: how far it travels and how much energy it spends along the way. Its body is described by a vector of morphological parameters. Its controller is a neural network that reads the drone's sensors and drives its actuators. The whole chapter is built on the nested problem of (3.1), which the first section introduces. There, a body $\boldsymbol{m}$ and a controller $\boldsymbol{\theta}$ are evaluated together by a performance $F(\boldsymbol{m}, \boldsymbol{\theta})$, which measures the quality of the flight of the body $\boldsymbol{m}$ under the controller $\boldsymbol{\theta}$. The only way to obtain the value of F is to simulate that flight.

The chapter has two parts. The first part is the section on related work. It reviews the previous works on the co-design of body and controller and on evolved plasticity, and it states the position of this thesis with respect to both. The second part is made of the three sections, which present three challenges. Each challenge limits what a search over bodies and controllers can find, and each one therefore shapes the method of this thesis. The first challenge is the sequential design challenge. When the body is designed by hand and only the controller is optimized, a better body is never even tried. The second challenge is the morphology optimization challenge. When one controller is kept fixed and only the body is optimized, the search prefers the bodies that suit that controller over the bodies with the greatest potential. The third challenge is the computational feasibility challenge. The direct remedy, which trains a new controller for every body the search visits, costs more than any search can afford. Together, the three challenges define the requirement that the rest of the thesis sets out to meet: a controller that adapts to the body it is given, at a cost small enough to fit inside the body search.

3.1 Related Work

This thesis sits at the meeting point of two lines of research that have so far been pursued separately: the joint optimization of a robot's body and of its controller, and the design of neural controllers whose weights keep changing during operation under rules that are themselves the object of optimization. This section reviews the works of each line that are closest to what is done here, singling out in each case the design choice that the present work makes differently, and closes by stating where it stands with respect to both.

3.1.1 Co-Design of Morphology and Control

Designing a robot means choosing a body and a controller, and the two choices depend on each other: the best controller depends on the body it drives, and the value of a body can only be judged through the best behaviour it affords. *Co-design* makes this coupling explicit by optimizing both. Writing $\boldsymbol{m}$ for a morphology in a design space $\mathcal{M}$, $\boldsymbol{\theta}$ for the parameters of a controller and $F(\boldsymbol{m}, \boldsymbol{\theta})$ for the

performance of the closed loop they form, obtained by simulating it, the problem is a nested one:

$$\boldsymbol{m}^\star = \arg\max_{\boldsymbol{m}\in\mathcal{M}} F\big(\boldsymbol{m}, \boldsymbol{\theta}^\star(\boldsymbol{m})\big), \qquad \boldsymbol{\theta}^\star(\boldsymbol{m}) = \arg\max_{\boldsymbol{\theta}} F(\boldsymbol{m}, \boldsymbol{\theta}). \tag{3.1}$$

The formulation states the problem without solving it: F is a black box, and evaluating the outer objective at a single candidate $\boldsymbol{m}$ requires solving, or approximating, the inner problem for that body. The works reviewed below differ mainly in how they pay for the inner level, and (3.1) is a convenient way to compare them.

The earliest approach collapsed the two levels into one. Sims [67] evolved virtual creatures whose genome, a directed graph, encoded at the same time the segments of the body and the neural circuits embedded in each segment, obtaining swimmers, walkers and jumpers from a single genetic algorithm; Lipson and Pollack [43] applied the same idea to machines made of bars, actuators and artificial neurons, and fabricated the evolved designs by rapid prototyping. In both cases the pair $(\boldsymbol{m}, \boldsymbol{\theta})$ is a single individual and the inner problem is never solved explicitly: the controller of a body is whatever the mutations happened to pair with it.

Cheney et al. [6] identified why this becomes a problem as the design space grows. A controller is co-adapted to the body it evolved with, so a mutation of the body is almost always harmful in the short term, however promising the new body may be once a controller is newly adapted to it. Selection therefore removes morphological novelty faster than it can be exploited, and the search converges prematurely on a body found early. Their remedy, *morphological innovation protection*, temporarily lifts selection pressure from individuals whose body has just changed, giving the controller time to catch up before the body is judged. The underlying principle carries over to every method in this section: a body can only be assessed fairly once a controller has been adapted to it.

Eiben and Hart [13] turned that principle into a design rule, arguing that every newborn body should go through a learning phase before its fitness is measured. Gupta et al. [27] showed what this buys at scale. In their Deep Evolutionary Reinforcement Learning framework, bodies are kinematic trees of cylinders connected by hinge joints, a space of the order of 10^{18} morphologies; an asynchronous tournament evolution mutates limbs and joints, and the controller of every new body is trained from scratch by PPO (subsection 2.1.3) on the body's own egocentric sensors for a fixed budget of interactions, with the fitness of the body being the reward reached at the end of training. Two findings are relevant here. Bodies evolved in harder environments, uneven terrain and manipulation rather than flat ground, perform better when transferred to new tasks, which the authors read as environmental complexity fostering morphological intelligence. And although nothing selects for it explicitly, the time a controller needs to reach a given performance halves within ten generations: evolution finds bodies that are easier to control, the morphological counterpart of the Baldwin effect. What makes these results possible is also what makes the method expensive: the inner problem of (3.1) is paid in full at every evaluation, with a complete reinforcement learning run per body and no inheritance of the controller, which required more than a thousand CPUs running in parallel.

The same division of labour has recently been applied to drones. Muff et al. [51] evolve hexacopters in which each of the six arms is described by its length,

two placement angles, two motor tilt angles and the spin direction of its rotor. A $(\mu + \lambda)$ evolution strategy with tournament selection acts on these morphological parameters, with repair operators that resolve collisions between arms and a static hovering test that discards unviable designs, while the controller of each morphology, a small feed-forward network, is learned from scratch by PPO on four waypoint-following tasks (circle, slalom, shuttle run and figure of eight). The evolved drones, with tilted rotors and asymmetric arms, outperform the conventional hexacopter on every task. The authors also introduce metrics for the learning process itself and find that evolved bodies reach their learning breakthrough earlier and converge to higher rewards, confirming the Baldwin-like effect of Gupta et al. As in that work, evolution owns the morphology and reinforcement learning owns the controller, and the cost of the inner level, several hundred million simulation steps per body, is what limits the search to under a thousand morphologies.

Bergonti et al. [4], in the laboratory where this thesis was carried out, address a morphing winged drone of the same family as the one used in the present work, a fixed-wing glider whose wings carry actuated joints. The outer level is NSGA-II (subsection 2.3.2) over the wing geometry, the number and orientation of the morphing joints of each wing, the choice of servomotors and propulsion unit from commercial catalogues, and a scalar that weighs the two objectives, energy consumption and mission time over a suite of scenarios. The inner level is not a learned controller but a trajectory optimization: for every candidate design, a nonlinear program solved by direct multiple shooting computes the time series of joint torques and thrust that completes each scenario at minimum cost, subject to the multibody dynamics with aerodynamic forces, the actuator limits, the obstacles and the checkpoints. The co-designed drones cut energy by more than a third and mission time by a fifth or more with respect to the commercial glider they start from. The choice of an open-loop optimal trajectory as inner solution is what makes tens of thousands of inner evaluations affordable on CPUs, but it assumes a known model and no disturbances, and it leaves the closed-loop controller that would fly the real drone outside the optimization.

A different way to escape the cost of the inner level is to amortize it. Gupta et al. [28] train a single Transformer policy, MetaMorph, on a large set of bodies at once, encoding each limb as a token that combines its proprioceptive readings with a description of its geometry and joints, so that the policy reads the morphology it is controlling as one more input modality. The result generalizes zero-shot to unseen variations of kinematics and dynamics and transfers to new bodies with little additional training. In the terms of (3.1), $\boldsymbol{\theta}^{\star}(\boldsymbol{m})$ is replaced by a single $\boldsymbol{\theta}$ for a policy $\pi(a \mid o, \boldsymbol{m})$ conditioned on $\boldsymbol{m}$: the inner problem is paid once, for the whole design space, at the price of telling the controller which body it has.

3.1.2 Evolved and Learned Plasticity

The idea of letting evolution choose the learning rule rather than the weights, introduced in subsection 2.4.3, also has a history in the laboratory where this work was done. Floreano and Urzelai [20] evolved controllers for a Khepera robot in which the genome specified, for each synapse, which of four Hebbian rules it obeyed and

at what rate, while the weights themselves were drawn at random at the beginning of every life and shaped only by the rule. On a task that required a sequence of behaviours, these controllers not only learned it within a life but, once evolved in simulation, transferred to the physical robot, and to a larger robot with a different body, better than controllers with evolved fixed weights. The review by Soltoggio, Stanley and Risi [68] consolidates two decades of such EPANNs and draws from them one lesson that matters for the present work: plasticity is selected only when something varies, between or within lifetimes, that a fixed genome cannot anticipate. On a stationary task evolution finds it cheaper to write the solution into fixed weights, and a plastic mechanism that is never needed is at best neutral and often harmful. Neuromodulation, the gating of the learning rate by a dedicated signal, is the other recurring theme of that literature [69].

Miconi, Stanley and Clune [48] took the opposite route and made the plasticity differentiable. Each connection carries a fixed weight W_{ij} and a Hebbian trace H_{ij} whose contribution is scaled by a coefficient α_{ij}:

$$y_i = \sigma\Big(\sum_j \big(W_{ij} + \alpha_{ij}\, H_{ij}\big)\, x_j\Big), \qquad H_{ij} \leftarrow (1-\eta)\, H_{ij} + \eta\, x_j\, y_i, \tag{3.2}$$

and $\boldsymbol{W}$, $\boldsymbol{\alpha}$ and η are all trained by gradient descent, backpropagating through the entire lifetime of the network, trace updates included. The decomposition is instructive: the weight is split into a part that is learned once and a part that is written by experience and forgotten geometrically, and $\boldsymbol{\alpha}$ decides, synapse by synapse, how much of the second part the network is allowed. Plastic networks trained this way outperformed non-plastic recurrent networks on a pattern memorization task and competed with dedicated architectures on few-shot image classification, and a later extension, Backpropamine [49], replaced the fixed η by a neuromodulatory signal computed by the network itself. The price of differentiability is that the lifetime must be differentiable: the base weights and the rule are co-adapted by the same gradient, and the method does not apply when the lifetime is a closed-loop interaction with a physics simulator, which is why the works below, and this thesis, return to evolution.

Najarro and Risi [52] did exactly that, with the parametrization of (2.26) extended by a learning rate per synapse, so that every connection is described by five coefficients, and a natural evolution strategy [63] searching them. The weights of the network are redrawn uniformly at random at the start of every episode and organize themselves under the rules within a few dozen time steps, without any reward signal during the lifetime. On a simulated quadruped this produced the result that has since become the reference for the approach: a Hebbian network evolved on the intact body and on one damaged leg keeps walking when a leg it has never seen damaged is removed, where a network with evolved fixed weights falls. The same experiment also shows the cost: on the intact body the fixed-weight network walks farther than the plastic one. Plasticity buys robustness to a change of body at a price on the nominal body, and the random initialization at each episode is what forces the rules to do the whole job of building the controller.

Two later works ask where and how many rules are needed. Pedersen and Risi [56] evolve the rules of the same quadruped and periodically merge them by clustering,

so that many synapses come to share one rule; the number of evolved coefficients drops by more than an order of magnitude and the smaller set generalizes better to unseen damage, a computational version of the genomic bottleneck. Ferigo et al. [17] evolve one rule per synapse in the controllers of voxel-based soft robots, whose body deforms as it moves, and report that the Hebbian controllers match or exceed static ones in locomotion and adapt to damage that was never seen during evolution.

Van Diggelen et al. [11] bring evolved Hebbian rules to swarm robotics. Each robot carries a small memoryless network with a rule of the form (2.26) on every synapse and a fixed learning rate, and a single set of rule coefficients, evolved by CMA-ES (subsection 2.3.1) on the performance of the whole swarm, is shared by all its members. At the beginning of each trial every robot draws its own weights at random; since each robot then experiences a different sensory history, identical rules drive the weights of different robots to different places, and heterogeneous roles, including collective behavioural switching, emerge from a homogeneous genome. The rules outperform multi-agent reinforcement learning baselines on standard benchmarks and transfer from simulation to ten physical robots with a loss of under one percent. Two features of this design recur throughout the plasticity literature reviewed here: the controller converges from a random initialization at every trial, and the same rule set is used by every agent regardless of its individual situation.

3.1.3 Positioning of This Thesis

Read through (3.1), the co-design works above pay for the inner level in one of three ways:

1. *in full*, with a reinforcement learning run per body [27, 51];
2. *exactly solved*, but for an open-loop trajectory, which leaves the closed-loop controller out of the design [4];
3. *once*, with a policy that must be told which body it controls [28].

The plasticity works, for their part, all build the controller from random weights at every episode [20, 52, 17, 11] or co-train the base weights and the rule by the same gradient [48]; none places evolved rules on top of a policy that was pretrained by reinforcement learning and then frozen. Ferigo et al. [18] co-evolve the body of voxel-based soft robots together with Hebbian rules, but each body carries its own set of rules in its genome. None uses one set of rules across a population of bodies inside a co-design loop.

This thesis works in the intersection of the two gaps. Its controller is a policy pretrained by reinforcement learning and then frozen, made plastic only at its output layer by evolved rules of the form (2.26), so that the plastic part is, as in (3.2), a departure from a fixed point, except that the departure starts from trained weights rather than from random ones; and it is evaluated across a population of bodies inside a morphology search rather than on a single body. Why each of these choices is needed is the subject of the three challenges that follow, and how they are realized of chapter 5.

3.2 The Sequential Design Challenge

Usually, the design of a flying robot starts from the body. An engineer designs the body, or a commercial product provides it. A controller is then optimized for that body. This thesis trains the controller by reinforcement learning; other works tune it by hand or compute it by trajectory optimization. With respect to (3.1), this procedure corresponds to fixing $\boldsymbol{m}$ at a chosen body $\boldsymbol{m}_0$ and solving only the inner problem, which returns $\boldsymbol{\theta}^\star(\boldsymbol{m}_0)$. The result is the best controller for the body $\boldsymbol{m}_0$. The best controller for one body is not the same thing as the best pair of body and controller. Any body $\boldsymbol{m}$ with $F(\boldsymbol{m}, \boldsymbol{\theta}^\star(\boldsymbol{m})) > F(\boldsymbol{m}_0, \boldsymbol{\theta}^\star(\boldsymbol{m}_0))$ would be a better choice than $\boldsymbol{m}_0$, and the sequential procedure never tries it.

Such better bodies exist, because the body design constrains what the controller can do. For example, the energy spent per metre of flight has a lower bound. This bound is set by the airframe, by its lift-to-drag ratio and by the mass it carries, and no controller can change it during flight. The task of this thesis rewards two things: the distance travelled through a cluttered forest and the energy efficiency of the flight. A task that rewards both distance and efficiency asks for a trade-off between the two objectives. A single hand-designed body sits at one point of that trade-off, and that point was chosen before the task was specified. Other bodies may be better on distance, better on efficiency, or better on both objectives at once. Co-design studies report exactly this outcome: when every body is given its own controller, the evolved bodies outperform the conventional design they started from [27, 51, 4], and the conventional design is never the winner.

The sequential design challenge is that the body is chosen before any search begins, so a better body can never be found, whatever controller is trained on the chosen one. The thesis must therefore search over bodies as well as over controllers.

3.3 The Morphology Optimization Challenge

The opposite of the sequential procedure is to fix the controller and search over the space of possible bodies. This is the cheap way to co-design. A single controller $\boldsymbol{\theta}_G$ is trained once, general enough to fly any body in the search space. The same controller then flies every candidate body, and the search maximizes $F(\boldsymbol{m}, \boldsymbol{\theta}_G)$ instead of $F(\boldsymbol{m}, \boldsymbol{\theta}^\star(\boldsymbol{m}))$. These two scores are different functions of $\boldsymbol{m}$. Such difference can reverse the order of two bodies. Under the shared controller, the body $\boldsymbol{m}_1$ scores higher than the body $\boldsymbol{m}_2$. Under their own controllers, $\boldsymbol{m}_2$ scores higher than $\boldsymbol{m}_1$:

$$F(\boldsymbol{m}_1, \boldsymbol{\theta}_G) > F(\boldsymbol{m}_2, \boldsymbol{\theta}_G) \qquad \text{and} \qquad F(\boldsymbol{m}_1, \boldsymbol{\theta}^\star(\boldsymbol{m}_1)) < F(\boldsymbol{m}_2, \boldsymbol{\theta}^\star(\boldsymbol{m}_2)). \tag{3.3}$$

In this case the search under $\boldsymbol{\theta}_G$ keeps $\boldsymbol{m}_1$ and discards $\boldsymbol{m}_2$, which is the body that would have been better.

There is a systematic reason to expect (3.3) to hold often rather than rarely. A generalist controller is trained on a whole distribution of bodies. Its weights must serve every body of that distribution, so on any single body they are a compromise, therefore achieves less than a controller trained on that body alone. This gap is smallest near the centre of the training distribution. The gap grows towards the

edges of the distribution, because there the dynamics of the body differ most from the average dynamics that the controller has learned to expect. The edges, however, are where the search wants to go. A body that is better than the average body of the distribution must differ from that average body, and the more it differs, the more it is penalized by a controller that was never adapted to it. The fixed controller thus does two things at once:

1. It undervalues the bodies that differ most from its training distribution, which are exactly the bodies the search should find.

2. It rewards the bodies that it already flies well, which are the bodies close to the average one.

A search under a fixed generalist is therefore biased towards the neighbourhood of the training distribution of that generalist. No amount of evolutionary pressure removes this bias, because the bias is built into the fitness function itself.

Training a controller after the search does not repair this bias. Suppose the search under $\boldsymbol{\theta}_G$ has returned its best body, and a controller is then trained on that body alone. The pair obtained is the best controller for the body that scored highest under the generalist. This pair is not the best pair either. Consider the body $\boldsymbol{m}_2$ of (3.3): it scored lower than $\boldsymbol{m}_1$ under $\boldsymbol{\theta}_G$, but it gains more from a controller of its own. The search discarded $\boldsymbol{m}_2$, so no training afterwards can recover it. Specialization applied after the search cannot correct a search that did not account for specialization.

The morphology optimization challenge is that a search under one fixed controller does not find the bodies with the greatest potential, but the bodies that suit that controller. What the search needs is a score that reflects what each body can do with a controller adapted to it. The next section shows why the direct way of obtaining such a score is too expensive.

3.4 The Computational Feasibility Challenge

The previous sections suggest that the co-design of body and controller is a promising direction for this thesis. The direct way to co-design is the one adopted by the co-design works that train their controllers by reinforcement learning [27, 51]. These works solve the inner problem exactly: they train a controller from scratch for every body the search evaluates. The cost of this approach in the setting of this thesis can be estimated from three measured quantities. The first is the time needed to train one controller on many bodies. The second is the time needed to train one controller on a single body. The third is the time taken by one search over bodies in which each body is scored by flights alone, with no training inside the loop. Table 3.1 gives the figures.

A single training of the generalist controller takes fourteen hours. A single body search evaluates 1 600 distinct bodies. Training a controller of that kind for each of them would take two and a half years on the same hardware. A specialist controller, trained on one body alone, is far cheaper: less than two hours on a single GPU. The specialist is the realistic form of the remedy. Even so, with two specialists trained at a time, one per GPU, one training per body brings the search to almost

Quantity	Value
Training of one generalist controller (1 000 PPO iterations, 65 536 parallel environments)	14 h
Training of one specialist controller on a single body (same settings, on one of the two GPUs)	1.7 h
Distinct bodies assessed by one body search (64 at first, then 32 new ones at each of 48 updates)	1 600
One body search in which bodies are scored by flights alone, with no training inside the loop (Proposed Co-Design Pipeline)	**94 h**
One body search with a controller training per body, at the generalist's cost	22 400 h
The same, with a specialist trained on the single body (two at a time, one per GPU)	1 360 h

Table 3.1: Cost of a controller training inside the body search, from the production runs of this thesis. All times are wall-clock hours of one computing node with two GPUs (NVIDIA V100 PCIe 32GB). The last two rows are hypothetical; the others are measured.

two months. The same search takes four days when every body is scored by flights alone. The factor between the two is almost fifteen at the present size of the search. And the size of the search is not fixed. Every additional body, generation or scenario multiplies the gap again.

The size of the search cannot be kept small. The performance of a body is a random quantity, for two reasons: the forests are generated at random, and the controller acts stochastically. A single flight therefore says little about a body. In this thesis each body is flown on sixty forests before its score is used, and even then the order of the bodies is far from noise-free. A co-design loop whose inner level is a training run therefore cannot grow along any of the three axes the search needs: scenarios per body, bodies per generation and number of generations. The works that take this route stay under a thousand bodies in total [51], or require over a thousand processors for a single run [27]. Both works search far smaller design spaces than the one of this thesis, and both use far cheaper evaluations than the flight of a winged drone through a forest.

The computational feasibility challenge is that training a controller for each body the search visits costs more than any search over bodies can afford. The feasibility challenge thus rules out the natural answer to the morphology optimization challenge. The controller cannot be trained per-body, so it must be trained once, for the whole space of bodies. A controller trained once is, by construction, not adapted to any one body. Its adaptation to the body being scored must therefore come from another mechanism, and whether an adaptation that cheap can change which bodies the search finds is the empirical question of this thesis.

Simulation Environment

Every quantity optimized in this thesis is measured in simulation. This chapter describes the three parts of that simulation. The first section presents the simulator, what its design gives and what it costs. The second section presents the task: the forest, how a flight ends and what a flight measures. The third section presents the drone, the space of bodies that the search explores and the reference body. The internal organization of the simulator, the full aerodynamic model, the two forest generators that were not used for the results and the derivations behind this chapter are collected in Appendix A.

4.1 Genesis: The Simulator

A body is scored by flying it, and a controller is trained by flying it. The searches of the following chapters fly about a quarter of a million drones in every generation. The simulator therefore decides which experiments are affordable, and its design leaves visible marks on the method. This work uses *Genesis* [23], an open-source physics engine for robotics released in December 2024, in its version 0.3.7. Genesis is written in Python, but its numerical core is compiled at run time into code for the Graphics Processing Unit (GPU). Tens of thousands of independent copies of the same scene therefore advance in parallel on a single card. Genesis has no model of the air. The forces that keep a winged drone aloft are computed by a custom aerodynamic solver, which is not part of the public release and was developed outside this thesis (it is credited in subsection 4.1.3). This section describes what the engine gives and what it costs, then the two halves of a simulation step: the rigid body dynamics that Genesis provides and the aerodynamic forces that were added to it. It closes with the random perturbations that are injected into both.

4.1.1 Parallel Simulation on the GPU

A Genesis program builds a *scene*, which holds the drone and the solvers that act on it. The scene is compiled for the GPU the first time it runs, and the compiled code is specialized to that scene. After the build, the state of the simulation and the physical parameters can change, but nothing structural can: not the number of bodies, not their geometry, not the size of any array. The most important structural parameter is the number of parallel *environments* B. The B environments are independent copies of the scene that share one model of the body and differ only in their state, and every kernel of the engine advances all of them in one launch. Environments never interact, which is exactly the structure of the workloads of this thesis: many flights of the same body, each in its own forest. The state of every environment can be read and written as a PyTorch tensor that resides on the same GPU. The neural controllers, the forests and the reward computations are PyTorch code; the physics is written in Taichi [38, 39], the parallel language on which Genesis is built. During a run only logging scalars ever leave the GPU. How the engine is organized

internally, and how its code reaches the GPU, is described in section A.1.

The first reason to choose Genesis is throughput. The generalist controllers of this thesis are trained on 65 536 simultaneous flights, and a single generation of the body search scores 245 760 of them. With physics, perception and control all resident on the GPU, these sizes are reachable on one or two cards, and the costs reported in Table 3.1 are the measured result. The second reason is that the engine is open to extension. A new physical model can be written in the same language as the engine and compiled into the same GPU pipeline, with direct access to the state of the rigid bodies. The aerodynamic solver used in this thesis is such an extension. So is the depth sensor of the drone, a custom kernel that measures the distance to the trees of each environment. The third reason is the interface. Bodies are read from standard robot description files, which a script generates from a genome. The state is exposed as batched PyTorch tensors, the format that reinforcement learning libraries expect. And physical parameters such as the masses and the centres of mass of the links can be perturbed independently in every environment. Lastly, Genesis is open source, so the whole stack can be packaged in a container and run unchanged on a workstation and on a cluster.

The costs follow from the same design. Compilation is slow: the first build of a scene compiles hundreds of kernels and can take several minutes, against the milliseconds of a simulation step. A cache on disk removes most of this cost from later runs, but every scene still has to be built, allocated and initialized. This matters for co-design more than for ordinary robot learning, because all the environments of a scene share one body. A population of different bodies needs one scene per body, and replacing the bodies means building new scenes. Two further restrictions make this worse. Genesis can be initialized only once in a process, and all the scenes of a process share one runtime, so they are stepped one after the other. Real parallelism across scenes requires separate processes, each with its own share of the GPU memory. And since a compiled kernel is bound to the arrays it was compiled with, every array whose size might change has to be dimensioned for the worst case at build time.

Numerical precision is a second limitation. Genesis runs in 32-bit floating point by default, with the relaxed arithmetic rules that compilers use for speed, and the order in which parallel GPU threads accumulate sums is not guaranteed. Two runs with the same random seed therefore do not produce the same trajectories. The differences start at the last bit and, because a flight among obstacles is a chaotic system, grow until the two flights end differently. The outcome of a flight must be treated as a random variable even when every explicit source of randomness is held fixed. A fitness value is only meaningful as an average over many flights.

Finally, Genesis is a young project. Its programming interface still changes between releases, and parts of its documentation lag behind the code. The only aerial model of Genesis is a quadrotor. Everything aerodynamic in this thesis comes from the solver added to the engine. An added solver exchanges forces with the rigid body solver explicitly, once per substep, so its numerical stability has to be secured by a small time step and by limits on the forces, as described below.

4.1.2 Physics Simulation

The drone is an articulated rigid body. A fuselage moves freely in space and carries six revolute joints, driven by servomotors, which sweep and twist the two wing panels and deflect the elevator and the rudder (the body is described in section 4.3). The rigid body solver of Genesis follows the formulation of MuJoCo [74]. It advances the equation of motion in generalized coordinates, with the aerodynamic forces as external forces and the mechanical limits of the joints as constraints. The equations, the constraint solver and the integrator are given in section A.2.

Each servomotor is modelled as a position controller with a bounded torque. It drives its joint towards the angle commanded by the controller, against the damping and the friction of the joint. The gains are the same on every body, while the torque limit and the range of the wing joints depend on the gear ratio of the body (subsection 4.3.1). The response of a control surface to a command is therefore that of a damped second order system with a saturating motor, and its delay is part of what the controller has to cope with.

Two time scales are used (Figure 4.1). The controller acts at 25 Hz. Every control step of 0.04 s is divided into four physics substeps of 0.01 s, so that the dynamics and the aerodynamic forces are updated at 100 Hz while the commands are held. A lifting surface reacts to a change of incidence with a force that opposes it, which makes the equations stiff, and an explicit scheme remains stable only if the step is small compared with the fastest of these reactions.

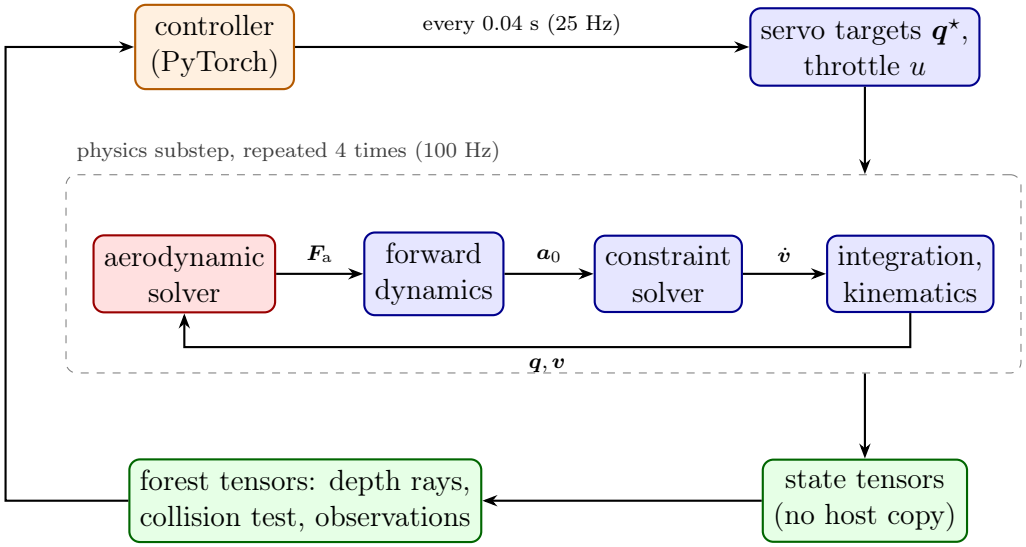


Figure 4.1: One control step. The commands of the controller are held while four physics substeps are executed; in each of them the aerodynamic solver turns the current state into forces, and the rigid body solver of Genesis computes the unconstrained acceleration, enforces the joint limits and integrates. Everything in the figure runs on the GPU, for all environments at once.

Genesis has a complete collision pipeline, but none of it is used here, and the trees of the forest are not objects of the physics engine. There are two reasons. A flight ends the moment the drone touches anything, so a contact only has to be

detected. And the geometry of a scene is shared by all of its environments, whereas this work needs a different random forest in every environment. The forests are therefore stored outside the engine, as tensors that hold the position of every trunk in every environment. The two operations that involve them, the depth sensing of the drone and the detection of a collision, are computed on those tensors for all environments at once (section 4.2). Likewise there is no ground plane: a flight that descends below a minimum altitude is ended.

4.1.3 Aerodynamics Simulation

The aerodynamic solver described in this subsection, and in full in section A.3, is not a contribution of this thesis. It was designed and implemented by Andrea Vicari as part of his doctoral research, which is still unpublished at the time of writing, and it is used here as he provided it, as the aerodynamic model of every simulation in this work. The credit for this part of the simulator is entirely his, and the author is grateful to him for making it available. What follows documents the model as it was used, so that the results of this thesis can be read and reproduced; it should not be taken as the reference description of his work, which will be his own publication.

The solver computes, at every physics substep and for every environment, the force that the air exerts on each part of the drone, and hands these forces to the rigid body solver. It is a *quasi-steady component build-up*. The aircraft is divided into seven elements: the fuselage, the two panels of the wing, the two halves of the elevator, the rudder and the propeller. Each element receives one lumped force that depends only on the instantaneous flow it sees, through lift and drag coefficients of closed form. A model of this kind is orders of magnitude cheaper than any solution of the flow field, which is what allows it to run a hundred times per simulated second in a quarter of a million environments. It is also algebraic in the geometry of the body, so the aerodynamic data of a new body come from its geometry alone: areas, chords, spans and aspect ratios are read from the same file that defines the rigid body.

Each lifting surface carries a reference frame that is attached to its link and moves with it. This is how the morphing of the wing enters the aerodynamics without any dedicated model: a twist of a panel changes its angle of attack, and a sweep changes its sideslip and displaces its point of application. For small angles of attack, lift and drag follow the classical theory of the finite wing [1]. Beyond stall the solver switches to the coefficients of a flat plate in fully separated flow, and the two regimes are blended around the stall angle, as is common in the flight dynamics models of small unmanned aircraft [2]. The parameters of the wing section are read from a table computed in advance with the panel code XFOIL [12], for every airfoil that the genome can select. Two interactions between elements are modelled, both directed at the tail: the downwash of the wing, which reduces the angle of attack of the elevator, and the slipstream of the propeller. The thrust of the propeller decreases as the aircraft flies faster [46]. This dependence matters for the two objectives of this thesis. A thrust proportional to the throttle would make fast flight as cheap as slow flight. With this model, the energy spent per metre depends on how well the drag of the body matches the propeller over the range of speeds at which the controller is

asked to fly. The equations of the model, its parameters and its numerical safeguards are given in section A.3.

The model is a compromise between fidelity and the cost of evaluating it a hundred times per second in every environment. It is quasi-steady, each surface carries a single lumped force, the flow induced by the rotation of the body is neglected, and the air is at rest apart from the noise described in the next subsection.

4.1.4 Simulation Noise

A deterministic simulator invites overfitting. A controller trained by reinforcement learning will exploit any regularity of the simulator that helps, and so will an evolutionary search that compares bodies by small differences in fitness. The standard countermeasure is to randomize the simulator, so that a solution scores well only if it works across a family of slightly different worlds [72, 57]. In this thesis noise is injected at three levels of the simulation.

The first level is the aerodynamic force itself. At every physics substep, and independently for every lifting surface of every environment, the force is perturbed in magnitude and in direction by random amounts of standard deviation 5%, and its point of application is displaced along the chord by a small random offset. The perturbation grows with the incidence, up to four times its base value in fully reversed flow, so that the force is least reliable exactly where the model is: around and beyond stall. The thrust of the propeller is not perturbed. Drawn anew a hundred times per second, this noise acts on the drone as a broadband buffeting. Its effect on the trajectory partly averages out, but it denies the controller a perfectly repeatable response to its commands. The formula is given in section A.4.

The second level is the body. At the beginning of every flight the mass of each link is multiplied by a factor drawn from a normal distribution with mean one and standard deviation 2%, and its centre of mass is displaced along each axis by a normal offset of standard deviation 4 mm. Both remain fixed for the duration of the flight. Genesis supports these perturbations natively: the mass and the centre of mass of every link carry an offset that is specific to the environment. For a winged drone the position of the centre of mass relative to the wing decides the stability of the aircraft. A few millimetres are therefore a meaningful disturbance, of the size that a real airframe shows from one assembly to the next.

The third level is the interface between the simulator and the controller. The observations are corrupted by sensor noise. The commands reach the servomotors with a random latency of up to one control step and with a small random offset. The forest and the initial conditions are drawn at random for every flight. These belong to the definition of the task rather than to the simulator. Their values are given with the training of the controllers (subsection 5.1.2).

The consequence is that the outcome of a flight is a random variable, and so is every quantity derived from it. This is so by design and, as noted above, also because of the arithmetic of the GPU. A single flight says little about a body or about a controller. Throughout this thesis performance is an average over many flights in different forests, and the comparisons between bodies and between methods are made with that variability in view.

4.2 Flight Environment

The task is the same in every experiment of this thesis: fly forward through a forest, as far as possible. The forest stands in a straight corridor. Let x be the direction of flight, y the lateral direction and z the vertical. The corridor is $W = 100$ m wide ($|y| \leq 50$ m), and the trees occupy the stretch $0 \leq x \leq L$. The drone is released 30 m before the first tree, on the axis of the corridor, 15 m above the ground and heading along x. A tree is a vertical cylinder of radius $r = 0.75$ m and is treated as infinitely tall, so that a forest cannot be flown over. Avoiding the trees is thus a planar problem, and the altitude matters only to the flight itself.

A forest is the list of the positions of its trunks, kept outside the physics engine (subsection 4.1.2). A batch of F forests of T trees each is a single tensor of $F \times T$ positions that resides on the GPU. At the beginning of every flight an environment is assigned one of the F forests. From then on the forest enters the simulation through two operations only, computed for all environments at once: what the drone sees, which is described with its sensors in subsection 4.3.2, and whether it has hit something.

Collision. The drone has hit a trunk when the trunk overlaps the segment that the wing projects on the ground. The segment is the combined span b of the two wing panels, 1.4 m for the reference drone, and it shortens as the drone banks (the test is drawn as a rectangle around the wing in Figure 4.4). A drone can therefore pass through a gap narrower than its wingspan, but only by rolling steeply while it does so. The test is an approximation: it ignores the width of the fuselage, the length of the body and the sweep of the panels. For the same reason the segment is shorter than the wingspan, which measures 1.5 m from tip to tip once the fuselage between the panels is counted. A safety margin is added around the segment: 1 cm when a body is evaluated and 20 cm during the training of the controllers, which teaches them to keep some distance from the trunks. The test is written out in section A.5.

End of a flight. A flight ends when one of the following happens: the drone hits a trunk, leaves the corridor sideways, descends to the ground, exceeds a quarter of a turn in roll, pitch or yaw (it has tumbled, or turned back), flies past the end of the forest, or reaches a time limit. The distance Δx covered along the corridor, from the point of release to the point where the flight ends, is the *progress* of the flight. Progress is the first of the two quantities by which bodies are evaluated in this thesis; a flight that ends at the first tree has a progress of 30 m. The second quantity is the energy spent per metre, which is defined with the drone in subsection 4.3.1.

What remains to be defined is how the positions of the trunks are drawn. The choice is less innocent than it seems, because the forest decides what the progress of a flight measures. Three generators were written in the course of this work. They share the corridor and the trees described above and differ only in the law of the positions. The generator described below produced every result of this thesis. The other two are described in section A.6: a forest of uniform density, which was used

for internal tests, and a stratified forest, which was explored and then abandoned. The lengths and densities used in the experiments are given with the method.

4.2.1 Growing Density Forests

In the forests used for all the results of this thesis the density grows linearly along the course, from λ_0 trees per metre at the entrance to λ_1 at the end:

$$\lambda(x) = \lambda_0 + (\lambda_1 - \lambda_0)\,\frac{x}{L}. \tag{4.1}$$

The number of trees is the integral of the profile, $N = \lceil (\lambda_0 + \lambda_1)L/2 \rceil$. The lateral coordinate of each tree is uniform across the corridor, and its coordinate along the course is drawn from the density (4.1) in closed form (section A.5). A whole batch of forests is thus generated with a handful of tensor operations. The entrance density λ_0 can also be drawn at random for each forest within a given range. This option is used during training, so that the controller meets dense forest at the beginning of a flight as well as at the end. Figure 4.2 shows one forest and its profile.

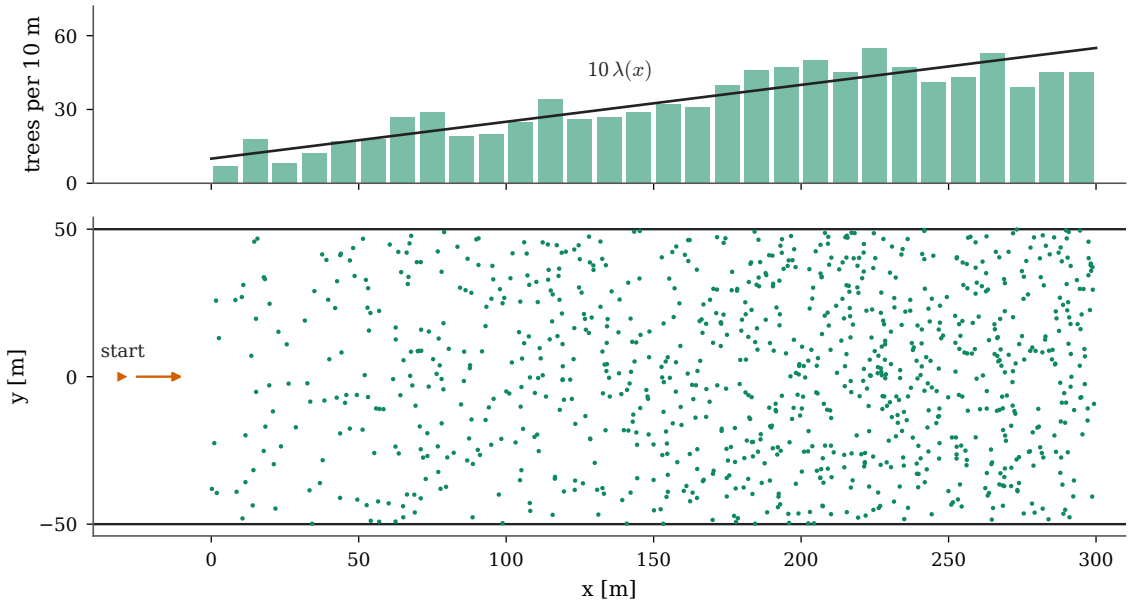


Figure 4.2: A growing density forest seen from above, with $\lambda_0 = 1$ and $\lambda_1 = 5.5$ trees per metre over 300 m, for 975 trees in total. The upper panel counts the trees in each 10 m of course and compares them with the profile (4.1).

The reason for this design is what it does to the measure. A growing forest is a course of increasing difficulty. Every flight, of whatever body, ends where the forest has become too dense for it, so the progress of a flight reads directly as the density that the pair of body and controller can cope with. A single course therefore orders the weak and the strong alike, the former in its first part and the latter in its last. As long as the course is long enough that nobody reaches its end, the measure does not saturate. The role of chance shrinks as well. Consider a drone that flies straight and blind. In a forest of constant density its risk of hitting a tree in the next metre

is the same whatever the distance already flown, so long flights are merely rare. In a growing forest the risk grows with the distance already flown. Long flights become rapidly improbable, and flights end in a band rather than anywhere (section A.5). In the forest of Figure 4.2 the blind flight covers 26 m on average from the first tree, and gets past 100 m less than once in a hundred attempts. Whatever a drone achieves beyond that is owed to what it sees and to how well its body lets it act on what it sees. That is exactly what the progress is meant to measure.

The design also helps learning. A controller at the beginning of its training crashes early and therefore only ever experiences the sparse part of the forest. As it improves, it reaches denser stretches, so the difficulty of its experience grows with its competence without any schedule having to be imposed.

The positions of the trees are independent, and this gives the forest the irregular texture of a real one, with its thickets and its clearings (Figure 4.2). Independent positions have a cost. Two trunks can be drawn on top of each other, and a few of them can line up so closely that they form a wall. A flight that ends against such a wall says little about the body or about the controller. The stratified generator was written to remove this effect, and it was abandoned because it also removes the thickets and the clearings (subsection A.6.2). Deciding when to go around a thicket and when to cut through a clearing is part of what flying through a forest means. The choice made in this thesis is therefore to keep the forest natural and to pay for its luck with numbers: the chance of an unlucky forest is averaged out over many of them.

4.3 Drone Model

The body that flies through the forest is a small electric aircraft with a morphing wing. Every body is generated by a script that turns a genome of fifteen numbers into a URDF file (Unified Robot Description Format), with its geometry, its masses and inertias, its joints and the reference frames of the aerodynamic solver of subsection 4.1.3. All bodies share their architecture: the parts they are made of, how these parts are actuated and what the drone can sense. What distinguishes them is the size, the proportions and the placement of those parts. This section describes the common architecture first, then the genome, and finally the particular body that is used as the reference throughout the thesis.

4.3.1 Drone Shape and Actuators

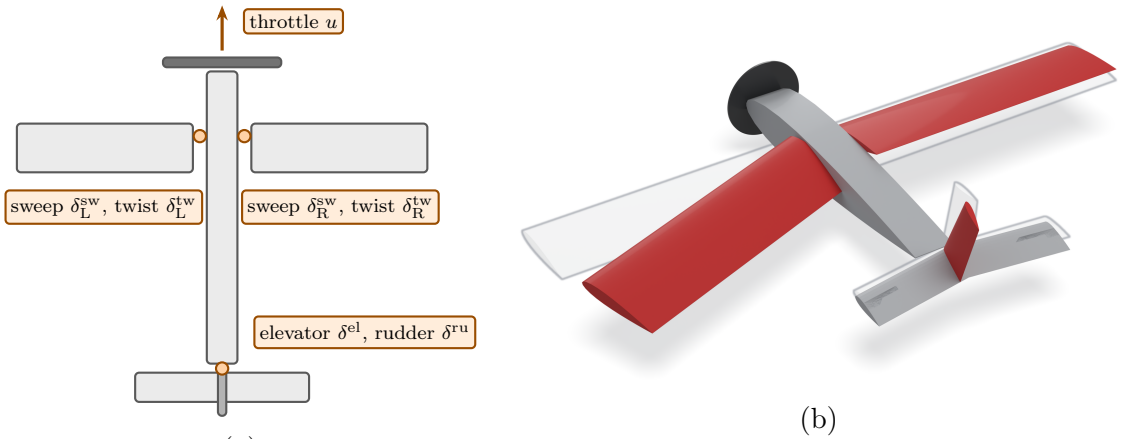
The drone has a conventional layout (Figure 4.3). A straight fuselage of constant cross-section, 10 cm wide, carries a propeller of 10 cm radius at its nose, a wing made of two independent rectangular panels attached to its sides, and a tail at its rear end. The tail is formed by a horizontal surface, the elevator, and a vertical one, the rudder. The fuselage also houses the battery, the electronics and a small camera at the nose, which are fixed masses.

What is not conventional is the way the drone is controlled. There are no ailerons, no flaps and no hinged control surfaces of any kind: the surfaces move as a whole. Each wing panel is connected to the fuselage by two revolute joints in series, which

share the same point at its root, on the quarter-chord line. The first joint rotates the panel about the vertical axis and changes its *sweep*. The second rotates it about its own spanwise axis and changes its *twist*, that is, its incidence. The elevator is a single all-moving surface that pitches about a hinge at the end of the fuselage. The rudder is an all-moving fin that turns about a vertical axis mounted on the elevator, so that it pitches with it. Together with the throttle of the propeller, this gives the seven commands that a controller issues at every control step:

$$\boldsymbol{a} = \left(u,\ \delta^{\mathrm{sw}}_{\mathrm{L}},\ \delta^{\mathrm{sw}}_{\mathrm{R}},\ \delta^{\mathrm{tw}}_{\mathrm{L}},\ \delta^{\mathrm{tw}}_{\mathrm{R}},\ \delta^{\mathrm{el}},\ \delta^{\mathrm{ru}}\right), \tag{4.2}$$

the throttle $u \in [0, 1]$ and six target angles, each bounded by the mechanical range of its joint. The two panels are commanded independently. The wing can therefore be moved symmetrically, to trim the aircraft and to change its lift, or asymmetrically, to roll and to turn: a differential twist does the work of the ailerons of a conventional aircraft. This is the kind of morphing wing studied for winged drones by Bergonti et al. [4], here combined with a moving tail.



(a) (b)

Figure 4.3: Parts and actuators of the drone. (a) The seven commands, on a schematic top view with the nose up: the throttle of the propeller, the sweep and the twist of each wing panel, the elevator and the rudder. (b) The reference drone with the left panel swept back to the limit of its range, the two panels twisted in opposite directions, and the elevator and the rudder deflected; the neutral position of every surface is shown in light grey.

Servomotors and gear ratios. Every joint is driven by a servomotor, modelled as in subsection 4.1.2: a position controller with a bounded torque, acting against the damping and the friction of the joint. The sweep joints use a larger servomotor than the twist and the tail joints (25 g against 10 g). Between the servomotor and a wing joint there is a reduction gear whose ratio k is part of the genome: one ratio for the sweep of both panels and one for their twist. A gear trades range for strength. The panel can move within

$$|\delta^{\mathrm{sw}}| \le \frac{0.7}{k_{\mathrm{sw}}}\ \mathrm{rad}, \qquad |\delta^{\mathrm{tw}}| \le \frac{0.5}{k_{\mathrm{tw}}}\ \mathrm{rad}, \tag{4.3}$$

with a torque limit of $0.75\,k_{\mathrm{sw}}$ and $0.6\,k_{\mathrm{tw}}$ N m respectively, while the damping and the friction felt at the joint grow with k^2 and with k. With the ratios allowed by the genome, between 1.5 and 3.5, the sweep range goes from $\pm 27^\circ$ to $\pm 11^\circ$ and the twist range from $\pm 19^\circ$ to $\pm 8^\circ$. A body can thus have a wing that moves widely but weakly, or narrowly but firmly. Which of the two is better depends on the aerodynamic loads that its panels generate, that is, on the rest of the body. The tail joints have no gear: their range is ± 0.25 rad ($\pm 14^\circ$) and their torque limit 1 N m on every body. The controller never commands an angle directly. Its six outputs lie in $[-1, 1]$ and are scaled by the range of the corresponding joint, so that the same output means "full deflection" on every body.

Masses. The structure is foam, and each part receives the mass of its volume and the inertia tensor of a box of its dimensions. Fixed masses are added for the battery and the electronics, the camera, the motor and the propeller, the six servomotors, and the hinges and fittings. The fixed masses amount to about 0.40 kg and are the same for every body; the structural mass is what the genome controls. It grows quickly with the size of the wing, since the volume of a panel is proportional to the cube of its span for a given aspect ratio and section. The lightest body that the genome can express weighs 0.44 kg and the heaviest 2.5 kg, while nine bodies out of ten drawn uniformly at random weigh between 0.50 and 1.10 kg. The position of the centre of mass is controlled by a gene as well, which places the mass of the fuselage, battery included, along its length. The densities and the fixed masses are listed in section A.7.

Energy. The second quantity by which a body is evaluated, next to the progress of section 4.2, is the energy it spends. The power absorbed by the propeller follows from its rotation speed and its advance ratio, with the same kind of law as the thrust (section A.7); at full throttle with the drone at rest it is about 83 W. Each servomotor absorbs the mechanical power it delivers plus a resistive loss proportional to the square of its torque, and never less than zero, since a servomotor that is driven backwards by the air does not recharge the battery. Holding a panel against a steady aerodynamic load therefore costs energy even when nothing moves. No other consumption is modelled, and the losses of the motor and of its speed controller are neglected. The energy E of a flight is the integral of the total power up to the end of the flight. It is compared across bodies of different weight through the dimensionless *cost of transport* (CoT),

$$CoT = \frac{E}{m\,g\,\Delta x}, \tag{4.4}$$

where m is the nominal mass of the body, g the acceleration of gravity and Δx the progress of the flight: the energy spent to carry one unit of weight over one unit of distance along the corridor. Distance flown sideways or in circles adds to the energy and not to the progress, so a body that needs wide manoeuvres to get through the trees pays for them.

4.3.2 Sensors

Every body carries the same sensors, and their readings are the only information about the world that a controller receives. Table 4.1 lists them. The drone knows its altitude, its attitude and its velocity, and it has a depth sensor that looks ahead. It does not know where it is along or across the corridor. It has no measure of its angular rates, of its airspeed or of the angles of its own joints, and no sensor tells it which body it is flying. The measurements are delivered at the control rate of 25 Hz, and each is corrupted by Gaussian noise, as anticipated in subsection 4.1.4. Besides the measurements, the controller receives two quantities that are not sensed: the seven commands it issued at the previous step, and the forward speed it is asked to keep.

Quantity	Size	Noise (standard deviation)
Altitude above the ground	1	0.1 m
Attitude, as a unit quaternion	4	0.01 on each component
Linear velocity in the world frame	3	0.4 m/s along the corridor, 0.2 m/s on the other axes
Depth ahead, one value per sector	20	1.5 m
Commands issued at the previous step	7	none
Commanded forward speed	1	none

Table 4.1: What the controller receives at every control step: 28 measurements, with the noise added to each, and 8 quantities that are not measured, for 36 numbers in total.

Depth sensor. The depth sensor is what lets the drone see the forest. It looks forward in the horizontal plane. A fan of 80° centred on the heading is divided into 20 equal sectors. Each sector reports the distance to the surface of the nearest trunk that falls inside it, even partially, or to the lateral wall of the corridor if that is closer, up to a range of 30 m (Figure 4.4). When the drone banks, the fan narrows by the cosine of the roll angle, as the horizontal footprint of a sensor fixed to the body would. The result is a one-dimensional depth image of 20 pixels, a deliberately poor sensor. It has no vertical resolution, its range is about two seconds of flight, and a trunk hidden behind another is invisible until the first has been passed. The sensor is one of the custom kernels mentioned in subsection 4.1.1, and it runs for all environments at once.

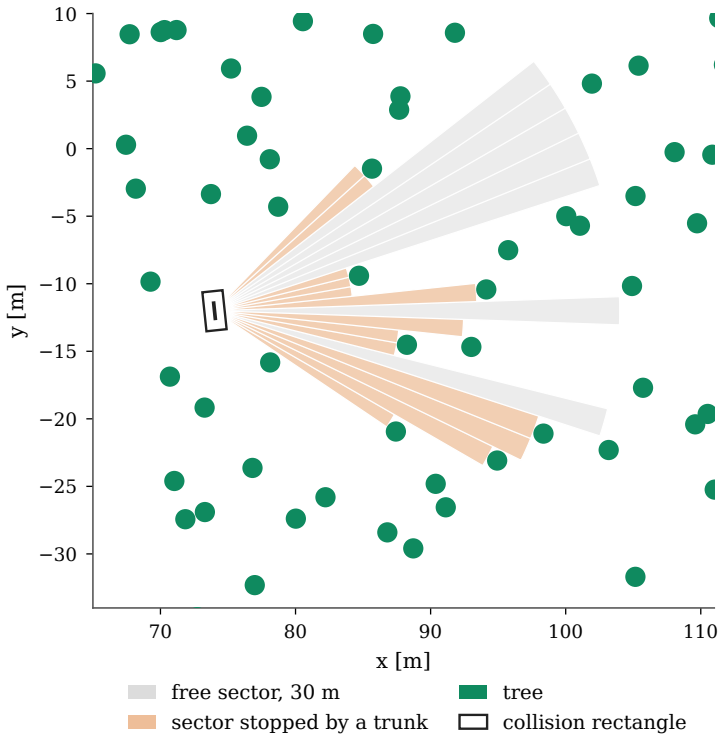


Figure 4.4: What the forest is to the drone, seen from above. The depth sensor divides a fan of 80° ahead of the drone into 20 sectors and reports for each the distance to the nearest trunk, up to 30 m. The collision test of section 4.2 asks whether the centre of a trunk lies inside the rectangle drawn around the wing (trunks and wingspan are to scale).

4.3.3 Drone Genome

A body is a point in a space of fifteen parameters, its *genome* (Table 4.2). Each gene is stored as a number in $[0, 1]$, the form in which the evolutionary algorithms handle it, and is mapped linearly onto the physical range of its parameter when the body is built. Twelve genes are continuous. The remaining three select the wing section among the four-digit NACA airfoils and are discrete: the unit interval is divided into as many equal bins as there are admissible values, and the gene takes the value of the bin it falls in.

The first two genes define the wing. Its panels are rectangular, and the genes give the span b_p of one panel and its aspect ratio, from which the chord is $\bar{c} = b_p/AR_p$. The two panels are mounted on the sides of a fuselage of width $w_f = 0.1$ m. The wing as a whole therefore has a span of $2b_p + w_f$ from tip to tip, between 0.9 and 1.9 m, and an aspect ratio of $(2b_p + w_f)/\bar{c}$. This is the aspect ratio that enters the aerodynamic model (subsection 4.1.3), and for the reference drone it is 7.5. The same construction holds for the elevator, whose gene is its total span, while the rudder is a single panel. Genes 3 to 5 lay the aircraft out along its axis. The length of the fuselage sets the lever arm of the tail, which sits at its end. The positions of the wing and of the mass of the fuselage along it decide, together, how far the centre of mass lies from the point where the lift of the wing acts. This distance governs the longitudinal stability of an aircraft, and a small change of either gene can turn a docile body into one that is hard or impossible to control. The dihedral gene tilts the two panels upwards or downwards at their root. The two gear ratios set the authority of the morphing wing, as described above.

The airfoil genes act in two ways. The thickness of the section is the thickness of the panel, and therefore changes its mass and its inertia. And the section as a whole selects a row in the table of airfoil parameters of subsection 4.1.3, from which the aerodynamic solver reads the zero-lift angle, the stall angle, the parasitic drag and the nominal Reynolds number of the wing. The tail surfaces are not part of the genome in this respect: they always use the same symmetric section, 16% thick.

No constraint is imposed on the genome beyond its ranges. The generator does not check that a body is well proportioned, that its wing and its tail do not interfere, or that it is stable. Any point of the unit hypercube becomes a body, and whether that body can fly is left for the flights to decide. The ranges were chosen so that a large part of the space is flyable, but nothing prevents the search from visiting the rest.

4.3.4 The Standard Drone

One body is singled out as the reference of the whole thesis (Figure 4.5). It is the body against which the evolved ones are compared, and the point from which the quantities of this chapter were illustrated. Its genome is the last column of Table 4.2. Its wing measures 1.5 m from tip to tip, with a chord of 0.2 m and a NACA 3416 section. Its fuselage is 0.73 m long, for an overall length of 0.89 m, and it weighs 0.747 kg.

The dimensions of the standard drone and the section of its wing are those of the H-King Bixler 3 [34], a commercial foam glider with a wingspan of 1.55 m and a

	Gene	Range	Reference drone
1	Span of one wing panel	0.4 to 0.9 m	0.7 m
2	Aspect ratio of one wing panel (span over chord)	1.5 to 5	3.5
3	Length of the fuselage	0.4 to 0.9 m	0.73 m
4	Position of the mass of the fuselage, as a fraction of its length from the nose	0.30 to 0.60	0.38
5	Position of the wing root, as a fraction of the fuselage length from the nose	0.30 to 0.60	0.38
6	Span of the elevator	0.2 to 0.6 m	0.5 m
7	Aspect ratio of the elevator	1.5 to 4	4
8	Span of the rudder	0.1 to 0.4 m	0.2 m
9	Aspect ratio of the rudder	1.5 to 4	2
10	Dihedral angle of the wing panels	$-4°$ to $4°$	$0°$
11	Gear ratio of the sweep joints	1.5 to 3.5	2
12	Gear ratio of the twist joints	1.5 to 3.5	2.5
13	Airfoil: maximum camber (first NACA digit)	0 to 4	3
14	Airfoil: position of the maximum camber (second digit)	2 to 5	4
15	Airfoil: thickness (last two digits)	10 to 20	16

Table 4.2: The genome of the drone: fifteen genes, their physical ranges, and the values of the reference drone of subsection 4.3.4. The last three genes are discrete.



(a) (b)

Figure 4.5: The standard drone with all joints in their neutral position: (a) three-quarter view, (b) top view with the nose up. The wing panels and the rudder are drawn in red, the fuselage and the elevator in grey, the disc of the propeller in black.

length of 0.95 m. The Bixler 3 is widely used as a research platform, and Bergonti et al. [4] adopt it as the baseline of their co-design study. For this reason the figures of this thesis label the standard drone with the name of that aircraft. The correspondence, however, stops at the geometry. The Bixler 3 is flown with ailerons, an elevator and a rudder hinged on fixed surfaces, and is pushed by a propeller mounted behind its wing. The standard drone steers by sweeping and twisting its wing panels and by moving its tail surfaces as a whole, and it is pulled by a propeller at its nose. It is also lighter, since the commercial aircraft weighs 0.89 kg ready to fly. The two aircraft share a shape, not a set of actuators, and their flight dynamics are therefore not directly comparable. The standard drone should be read as a body of familiar proportions inside the design space of Table 4.2, not as a model of the commercial aircraft.

Methodology

chapter 3 ended with the shape that any workable answer must have: a controller that is paid for once, for the whole space of bodies, and an adaptation to the body being scored that costs no more than the flights that score it. This chapter describes the method that gives that shape a content, in the order in which the method is built. section 5.1 trains by reinforcement learning the controllers on which everything else rests. section 5.2 takes one of them, freezes it, and makes its output layer plastic through Hebbian rules. section 5.3 describes the co-design pipeline, in which an inner loop evolves those rules and an outer loop evolves the bodies they are meant to serve, and whose results are reported in chapter 6.

5.1 Base Controllers

Everything in the method rests on a controller that can already fly. Two kinds of such controllers are used in this thesis. They share the architecture, the task, the reward and the learning algorithm, and differ only in the bodies they are trained on. A *generalist* is trained on a large pool of different bodies at once: it is the $\boldsymbol{\theta}_G$ of section 3.3, a single set of weights that is expected to fly any body of the design space, and it is the controller on which the co-design loop is built. A *specialist* is trained on one body alone: it is the closest that this thesis comes to the $\boldsymbol{\theta}^\star(\boldsymbol{m})$ of (3.1), and it serves as a term of comparison and as a second base on which the rules of section 5.2 are evolved. Both are trained with PPO (subsection 2.1.3), in the implementation of the RSL-RL library [62], which was written for the regime used here: thousands of environments simulated in parallel on the GPU, each contributing a short segment of experience to every update. This section describes the network first, then the environment in which it is trained and the reward that defines what flying well means, and finally the two training regimes with their numbers.

5.1.1 Architecture of the Controller

The controller is the actor of an actor-critic pair (subsection 2.1.2). At every control step, 25 times per second, it receives the 36 numbers of Table 4.1, collected in the observation $\boldsymbol{o}_t$, and returns the seven commands $\boldsymbol{a}_t$ of (4.2). Before entering the network each component of the observation is standardized with the running mean and standard deviation of the observations collected during training; these statistics are frozen after the first 10^8 samples, about a hundred iterations, and are stored with the weights. The network itself is small (Figure 5.1): an LSTM layer of 128 units (section 2.2), two fully connected layers of 128 and 32 units with the exponential linear unit (ELU) as activation [7], and a linear output layer of seven units.

The recurrent layer comes first, and its role is to make the controller stateful. The task is only partially observable: the drone does not measure its angular rates, its airspeed or the angles of its joints (subsection 4.3.2); its measurements are noisy; and its commands take effect after a latency that changes from flight to flight,

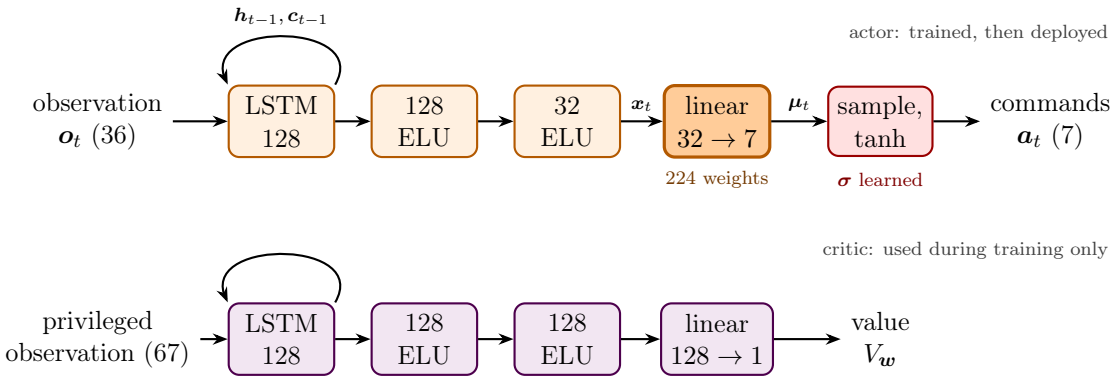


Figure 5.1: The two networks of a base controller, with the sizes of the generalist. The actor maps the observation of Table 4.1 to the seven commands; its output layer, highlighted, is the part of the network on which the rest of the method acts. The critic estimates the value of the current state from a richer input, which for the generalist includes the genome of the body (the critic of a specialist does without it and has 52 inputs), and is discarded when training ends.

through servomotors that respond with a delay of their own. A controller without memory would have to decide from a single noisy snapshot of all this. A controller with a state can use the history of what it observed and commanded: a rate from successive attitudes, the state of an actuator from the commands that were sent to it, a measurement steadied by those that preceded it. This is why the previous commands are part of the observation, and why the memory is placed where it acts on the raw history.

The two fully connected layers compress the state of the LSTM into a vector $\boldsymbol{x}_t \in \mathbb{R}^{32}$, from which the output layer computes

$$\boldsymbol{\mu}_t = \boldsymbol{W}\,\boldsymbol{x}_t + \boldsymbol{b}, \qquad \boldsymbol{W} \in \mathbb{R}^{7\times 32}. \tag{5.1}$$

The width of the last hidden layer fixes the size of the output layer at $7 \times 32 = 224$ weights, a number that matters later: the output layer is the part of the controller that section 5.2 makes plastic, and every one of its weights will carry evolved coefficients. In total the actor has 105 863 weights and biases, four fifths of which belong to the LSTM.

The policy is stochastic. The output $\boldsymbol{\mu}_t$ is the mean of a Gaussian from which a vector is sampled and then squashed,

$$\boldsymbol{z}_t \sim \mathcal{N}\big(\boldsymbol{\mu}_t,\, \mathrm{diag}(\boldsymbol{\sigma}^2)\big), \qquad \tilde{\boldsymbol{a}}_t = \tanh(\boldsymbol{z}_t) \in (-1, 1)^7, \tag{5.2}$$

where $\boldsymbol{\sigma}$ is a vector of seven learned parameters that does not depend on the observation, initialized at 0.3. The first component of $\tilde{\boldsymbol{a}}_t$ becomes the throttle, $u = (\tilde{a}_1 + 1)/2$, and each of the other six is multiplied by the range of its joint (4.3) to give a target angle. The squashing keeps every command inside its range by construction. The usual alternative, a Gaussian whose samples are clipped at the limits, makes the executed command differ from the sampled one whenever a limit is hit, and the probability ratio on which PPO relies is then computed for an action that was not the one taken. With (5.2) the density of the executed command

is known exactly: it is the Gaussian density of $\boldsymbol{z}_t$ corrected by the Jacobian of the hyperbolic tangent [29].

One input is absent on purpose: the actor never receives the genome of the body, nor any other description of it, whether it is trained on one body or on many. The controller does not know which body it is flying. The purpose is to let it learn to fly any morphology without hints on what its flight dynamics could be. This is the opposite of the choice made by MetaMorph [28], whose policy reads a description of its morphology. A generalist trained in this way is one set of weights for all bodies and is adapted to none of them in particular; adapting it to the body it is given is the task of the next section.

The critic has no such restriction, because it is never deployed. It is a separate network with its own LSTM layer of 128 units, two fully connected layers of 128 units and a scalar output, and it is an asymmetric critic [58]. Its input has 67 components: the 36 of the actor, but without sensor noise; the angular velocity of the fuselage, the angles and the rates of the six joints, and the thrust actually delivered by the propeller, as a fraction of its maximum; and the 15 genes of the body. The value of a state depends on the body that is in it, since the same attitude and the same trees ahead promise more to a body that turns readily than to one that does not, and the critic can simply be told what the actor is never told. The genome shown to the critic is perturbed by Gaussian noise, with a standard deviation of 10% of the range of each gene drawn once per flight and of 2% drawn at every step (the discrete airfoil genes excepted), so that it reads as a description of a body rather than as the label of one of the training bodies.

5.1.2 Training Environment

The controllers are trained on the task of section 4.2, with the simulation noise and the sensors described in the same chapter. What that chapter left open are the numbers, which Table 5.1 collects.

The course is short, 150 m, and dense from the start. With an entrance density drawn between 0 and 3 trees per metre a forest holds between 300 and 525 trees, and the controller meets, in the first metres of some flights, the density that other flights reach only at the end: this is the option anticipated in subsection 4.2.1, and it prevents the first part of every flight from being an empty stretch in which nothing is learned about trees. A flight that ends is replaced at once, in the same environment, by a new one with a new forest, a new commanded speed and a new release.

The release is randomized for two reasons. The environments all begin together, and without it their first seconds would be thousands of copies of the same flight. And a controller that is always released in trimmed flight on the axis of the corridor never learns to recover: here it starts anywhere across the corridor, as low as 5 m above the ground, and at a speed that has nothing to do with the one it is asked to keep, so that the first thing it has to do in every flight is accelerate or slow down. The range of the commands is wider than what most bodies can hold: the reference drone stalls below about 6 m/s, and above 23 m/s the propeller of (A.16) delivers no thrust to any body. A command near either end cannot be met in level flight, and the tracking term of the reward then favours the closest speed that the body can

Quantity	Value during training
Course	growing density forest (subsection 4.2.1), 150 m long and 100 m wide
Density at the end, λ_1	4 trees per metre
Density at the entrance, λ_0	uniform between 0 and 3 trees per metre, drawn for each forest
Forests	ten for each environment, generated once; one of them drawn at random at every flight
Commanded speed	uniform between 5 and 25 m/s, drawn at every flight and held
Release point	$(-30, 0, 15)$ m, displaced uniformly by up to ± 15 m along the course, ± 40 m across it and between -10 and $+5$ m in altitude
Initial velocity	forward component uniform between 5 and 25 m/s; lateral and vertical components normal, with a standard deviation of 1 m/s; nose aligned with the velocity
End of a flight	as in section 4.2: the ground is an altitude of 0.1 m, the collision margin is 0.2 m, the attitude limits are 90° and the time limit is 100 s (2 500 control steps)
Latency of the commands	zero or one control step, drawn at every flight
Offset of the joint targets	normal, with a standard deviation of 0.01 rad drawn at every flight plus 0.005 rad drawn at every step

Table 5.1: The training environment in numbers. Sensor noise is that of Table 4.1; the aerodynamic noise and the randomization of masses and centres of mass are those of subsection 4.1.4.

reach; the evaluations of the following sections use a narrower range of commands. The commanded speed makes of the controller a family of controllers indexed by one number. Both quantities by which flights are judged depend on the speed, since a faster flight leaves less time to react to a trunk that appears at 30 m and changes the energy spent per metre, and leaving the speed as an input keeps that choice open instead of burying it in the weights. The evaluations of the following sections exploit it by flying every body over a range of commanded speeds.

5.1.3 Reward Function

In this thesis the bodies are piloted by the Hebbian controllers of section 5.2 and judged by two quantities, the progress and the cost of transport (4.4), which are properties of a whole flight. They say nothing about which of the hundreds of commands of a flight were good ones, and a learning signal is needed at every step. The reward is therefore a separate object, designed to be informative step by step and to have, as its maximizer, a pilot that does well on both quantities. It is a weighted sum of eight terms,

$$r_t = \sum_{i=1}^{8} w_i \, f_i(t), \tag{5.3}$$

listed in Table 5.2. The weights of the table are those that multiply each term at every control step. One term is positive and seven are penalties; the same reward is used for every body and for both kinds of controller, and it contains nothing that depends on the body.

Speed tracking. The only positive term rewards the drone for moving along the corridor at the speed it was asked to keep. Here v_x is the component of its velocity along the axis of the corridor and $v^\star$ the commanded speed. The term is a bell curve of the relative error, with a width of 25%: it is worth 1 when the two speeds coincide, 0.61 when they differ by a quarter, 0.14 when they differ by half, and practically nothing for a drone that hovers, flies across the corridor or turns back. Because v_x is measured along the corridor and not along the heading, what is rewarded is the rate at which progress accumulates, and a drone that weaves more than the trees require loses reward as it loses ground. A flight that holds the commanded speed for T seconds collects $25\,T$ and covers $v^\star T$ metres: for a given command, the reward accumulated by this term is proportional to the progress of the flight. It is the progress of section 4.2 in the form of a rate, divided by the commanded speed so that a slow flight and a fast one earn the same per unit of time. Rewarding the distance itself would have been simpler, and would have pushed every body to the highest speed it can reach, whatever that costs in energy and in collisions.

Crash. A penalty of 5 is paid once, at the step in which the flight ends because the drone has hit a trunk (with the margin of 0.2 m), has left the corridor sideways or downwards, or has exceeded the attitude limits. The figure is small, five steps of perfect tracking, because the real cost of a crash is elsewhere: the flight is over,

Term	Expression f_i	Weight w_i
Speed tracking	$\exp\left[-\frac{1}{2}\left(\frac{v_x/v^\star - 1}{0.25}\right)^2\right]$	$+1$
Crash	1 at the step in which the flight ends in a crash, 0 otherwise	-5
Obstacle proximity	$\sum_{j=1}^{20} \exp\big(-1.5\,\max(\rho_j,\,0.5)\big)$, with ρ_j from (5.4)	-0.2
Energy	power P drawn from the battery, in watts	-0.002
Altitude	$\max(0,\,10 - z)^2$, with the altitude z in metres	-0.006
Smoothness	$\lVert \boldsymbol{a}_t - \boldsymbol{a}_{t-1} \rVert^2$	-0.2
Angular rate	$\lVert \boldsymbol{\omega} \rVert$, in rad/s	-0.01
Symmetry	$\big(q_{\mathrm{L}}^{\mathrm{sw}} + q_{\mathrm{R}}^{\mathrm{sw}}\big)^2 + \big(q_{\mathrm{L}}^{\mathrm{tw}} - q_{\mathrm{R}}^{\mathrm{tw}}\big)^2 + \big(q^{\mathrm{ru}}\big)^2$, angles in radians	-2

Table 5.2: The terms of the reward (5.3), with the weight that multiplies each of them at every control step. All quantities are the true ones of the simulator, without sensor noise.

and with it the reward that the rest of the flight would have collected. With the discount factor used here, $\gamma = 0.99$, a drone that is tracking its speed well expects a discounted return close to $1/(1-\gamma) = 100$, and a crash forfeits all of it. The explicit penalty is what distinguishes a crash from the other ways in which a flight can end, the end of the course and the time limit, which carry none.

Obstacle proximity. A crash is a late signal: by the time it arrives, the commands that caused it lie one or two seconds in the past. The proximity term anticipates it. Each of the 20 sectors of the depth sensor contributes a penalty that decays exponentially with an anisotropic distance to what the sector sees. If d_j is the depth of sector j and ψ_j its direction with respect to the heading, the obstacle seen by the sector lies at $(d_j \cos\psi_j,\ d_j \sin\psi_j)$ in the horizontal frame of the drone, and

$$\rho_j = \frac{1}{5\ \mathrm{m}} \sqrt{(d_j \cos\psi_j)^2 + 24\,(d_j \sin\psi_j)^2}. \tag{5.4}$$

The lateral offset of an obstacle weighs $\sqrt{24} \approx 4.9$ times its distance ahead, so the curves of equal penalty are ellipses centred on the drone and stretched along its heading, five times longer than they are wide. A trunk 5 m straight ahead gives $\rho = 1$ and a contribution of $e^{-1.5} = 0.22$ in every sector that it fills; the same trunk at the same distance but at the edge of the fan, 40° off the heading, gives $\rho = 3.2$ and a contribution below 0.01. The term therefore does not ask the drone to stay away from trees, which in a forest would be asking it not to fly: it asks the drone not to point at them, and lets it pass close beside a trunk. As a trunk ahead comes nearer the penalty grows twice over, because ρ shrinks and because the trunk fills more sectors, up to the floor $\rho \geq 0.5$ that bounds the contribution of a sector at 0.47. With a wall of trunks filling the view at close range the term exceeds the tracking reward, and slowing down or turning away becomes the better choice well before the collision. The lateral walls of the corridor are seen by the sensor like trunks and are penalized in the same way.

Energy. The power P is the total power drawn from the battery of the drone, by the propeller and by the servomotors, as defined with (A.23). It is the only term through which the cost of transport enters the training, and its weight is small: at the 83 W of full throttle at rest it amounts to 0.17 per step, a sixth of the tracking reward. At this level energy is never worth a crash or a missed command, but between two ways of holding the same speed through the same trees the cheaper one earns more.

Altitude. Below 10 m the drone pays a penalty that grows with the square of the missing altitude, 0.15 per step at 5 m and 0.6 at the ground; above 10 m it pays nothing. The trees cannot be flown over, so height brings no advantage in the forest, but it is the reserve from which a drone recovers after a steep turn, and the one-sided form tells the controller that the reserve matters long before the ground ends the flight.

Smoothness and angular rate. Two terms regularize the way the drone is flown. The first penalizes the squared change of the commands applied to the actuators between two consecutive steps, the throttle in its unit range and the joint targets in radians. A stochastic policy trained in simulation drifts easily towards commands that jump from one extreme to the other at every step, which a simulated servomotor tolerates and a real one does not; the term also keeps down the energy spent by the servomotors. The second penalizes the norm of the angular velocity $\boldsymbol{\omega}$ of the fuselage, and it is light: a turn at 1 rad/s costs one hundredth of the tracking reward. It discourages oscillations without discouraging manoeuvres.

Symmetry. With seven commands the aircraft is redundant: the same straight flight can be held by many combinations of joint angles, including crossed ones in which an asymmetry of the wing is balanced by the rudder. The last term selects among them. It penalizes the difference in sweep between the two panels (with the sign convention of the joints a symmetric sweep corresponds to opposite angles, hence the sum), the difference in twist, and the deflection of the rudder, all measured on the actual joint angles q and not on their targets. The symmetric sweep, the symmetric twist and the elevator, which are the controls that trim the aircraft, are left free. Asymmetric deflections are what rolls and turns the drone, and the term does not prevent them, since a differential twist of 0.2 rad costs 0.08 per step: it makes them transient, used for a manoeuvre and released after it.

Two further terms exist in the environment and carry a weight of zero in the final configuration: a bonus for reaching the end of the course and a penalty on the roll and pitch angles.

The reward has a second use after the training. Summed over a whole flight, it is the fitness by which the Hebbian rules are evolved (subsection 5.3.2), so that the rules are judged by the same measure as the controller on which they act. The search over bodies (subsection 5.3.3) never sees it: bodies are judged by progress and by cost of transport, measured on whole flights. With respect to those two quantities the reward is a dense proxy, with the tracking term standing for the rate of progress, the crash and proximity terms for survival, and the energy term for the cost of transport, and the correspondence is only approximate. A base controller is therefore a competent pilot and not an optimum of either objective; optimizing against the objectives themselves is the task of the search over bodies.

5.1.4 Training the Generalist

The generalist is trained on a pool of $K = 256$ bodies. Their genomes are drawn independently and uniformly from the unit hypercube, once, before training, and each is built into a body by the generator that builds every body of this thesis. No body is rejected: since the genome has no constraint beyond its ranges, the pool contains bodies that fly well, bodies that fly badly and bodies that hardly fly at all, in the proportions in which the design space contains them. The reference drone is deliberately left out, so that for the generalist it is a body like any other that the search may propose: one that it has never flown. What the generalist maximizes is

the average, over the pool, of the expected discounted return of the reward (5.3),

$$J_G(\boldsymbol{\theta}) = \frac{1}{K}\sum_{k=1}^{K} \mathbb{E}_{\pi_{\boldsymbol{\theta}},\, \boldsymbol{m}_k}\left[\sum_{t\geq 0} \gamma^t\, r_{t+1}\right], \qquad \boldsymbol{m}_k \sim \mathcal{U}\big([0,1]^{15}\big), \tag{5.5}$$

where the inner expectation is over the forests, the commanded speeds, the releases, the noise and the actions of the policy when the body is $\boldsymbol{m}_k$. Two hundred and fifty-six points are a sparse sample of a space of fifteen dimensions. Their role is to make the strategy that works across bodies the only one that pays, because the network is never told which of them it is flying. The objective also states the compromise of section 3.3 in one line: an average can be raised by flying many ordinary bodies well, and nothing in (5.5) asks for the best that each single body could give.

Each body is simulated in 256 parallel environments, for 65 536 environments in total. The environments of a Genesis scene share one model, so every body needs a scene of its own: the 256 scenes are distributed over eight processes of 32 bodies each, four processes on each of the two GPUs of the node of Table 3.1. The scenes advance in lock-step. At every control step the learner gathers the 65 536 observations, evaluates the single policy once for all of them, and sends the commands back. After 15 control steps, 0.6 s of flight in every environment, the 983 040 transitions collected from all bodies are pooled in one buffer and used for one PPO update, and the cycle repeats for 1 000 iterations. Table 5.3 lists the settings.

A few of these settings deserve a comment. At 25 Hz a discount factor of 0.99 gives a horizon of about a hundred steps, or 4 s: twice the time that separates the drone, at 15 m/s, from a trunk at the limit of its depth sensor, beyond which there is nothing to plan for. The learning rate is adapted during training. After every mini-batch the KL divergence between the policy being optimized and the one that collected the data is compared with the target: the rate is divided by 1.5 when the divergence exceeds twice the target and multiplied by 1.5 when it falls below half of it, which keeps the size of the policy updates roughly constant while the landscape changes. Since the policy is recurrent, a mini-batch is a set of environments, each with its whole segment of 15 steps, and the advantages are standardized within the mini-batch, that is, within a group of eight bodies. Gradients are propagated through time inside a segment only, while the recurrent state itself is carried from one segment to the next and is reset when a flight ends: what the controller remembers beyond 0.6 s is shaped by the gradient only indirectly.

The experience consumed is large in absolute terms and modest per body. A thousand iterations amount to 9.8×10^8 control steps, fifteen months of flight, of which every body receives one part in 256: 3.8×10^6 steps, or 43 hours. The training takes fourteen hours on the two GPUs (Table 3.1), most of them spent in the simulation, because the 256 scenes cannot be advanced all at once: inside each process they are stepped one after the other. The weights obtained at the last iteration are the generalist used in the rest of this thesis. The critic is discarded and the actor is frozen, and the co-design loop of the following sections is built on top of it.

Quantity	Value
Parallel environments	65 536 (256 for each of 256 bodies)
Control steps per environment per iteration	15 (0.6 s of flight)
Transitions per iteration	983 040
Iterations	1 000
Discount factor γ	0.99
Decay parameter of the advantage estimate (GAE)	0.9
Clipping parameter ϵ	0.15
Epochs per iteration	2
Mini-batches per epoch	32, of 2 048 environments each
Optimizer and learning rate	Adam [41], 10^{-4} at the start, then adapted
Target KL divergence between successive policies	0.006
Value loss coefficient c_1	0.3, with clipped value loss
Entropy coefficient c_2	0.002
Limit on the norm of the gradient	0.5

Table 5.3: Settings of the training of the base controllers. The symbols are those of subsection 2.1.2 and subsection 2.1.3.

5.1.5 Specialist Controllers

A specialist is the same procedure with $K = 1$ in (5.5). The 65 536 environments all fly the same body, in a single scene on a single GPU; the network, the environment, the reward, the settings of Table 5.3 and the number of iterations are unchanged. The only difference is in the critic, which no longer receives the genome, a constant, and has 52 inputs. Because the experience is not shared with other bodies, a specialist sees of its body 256 times what the generalist sees of any of its own, and its training, with one scene instead of 256, takes less than two hours (Table 3.1).

In this thesis a specialist is trained for the reference drone of subsection 4.3.4. It is the sequential design of section 3.2 in concrete form, a given body with a controller optimized for it alone, and it has two uses. It is the term against which the generalist is compared on a body that it was not trained on. And, frozen like the generalist, it is a second base for the Hebbian rules: evolving them on the reference drone on top of its own specialist asks whether plasticity still has something to add when the controller was already trained for the body.

5.2 The Hebbian Controller

The generalist of the previous section is one set of weights for all bodies, adapted to none of them in particular. This section describes the mechanism that adapts it. The trained actor is frozen, and the weights of its output layer alone are allowed to change during the flight, at every control step, under local Hebbian rules of the form (2.26). The rules are shown neither the genome of the body nor the reward: they see the activity that flows through the layer, and the body shapes that activity through the way in which it answers the commands. Their coefficients are evolved, and they are the only part of the controller that is searched for once the training of section 5.1 is over.

The way in which the rules are exploited is the main novelty of this thesis. In the evolved plastic controllers reviewed in subsection 3.1.2 the weights are drawn at random at the start of every episode, and the rules have to build a controller out of nothing every time. Here the controller already exists and already flies. The rules start every flight from the trained weights and only have to move them, by a bounded amount, towards what the body in flight requires: plasticity is not used to learn the task, but to adapt to a body a solution of the task that was learned for all bodies. The following subsections describe the mechanism, the reasons for placing it at the output layer, the decay that keeps the weights bounded, and the values of the parameters involved.

5.2.1 Hebbian Rules on the Output Layer

When the training of a base controller ends, the actor is frozen. The statistics that normalize the observation, the LSTM, the two fully connected layers and the bias $\boldsymbol{b}$ of the output layer keep the values of the last iteration, and no gradient is computed from this point on. The only part of the network that is released is the matrix $\boldsymbol{W}$ of (5.1). It stops being a parameter and becomes part of the state of the controller,

like the state of the LSTM: it has a value $\boldsymbol{W}_t$ at every control step, it starts every flight from the trained weights, written $\boldsymbol{W}_0$ from here on, and what happens to it in between is decided by the rules.

At step t the frozen layers deliver the vector $\boldsymbol{x}_t \in \mathbb{R}^{32}$, and the output layer computes the mean of the command distribution with the weights it has at that moment, $\boldsymbol{\mu}_t = \boldsymbol{W}_t\,\boldsymbol{x}_t + \boldsymbol{b}$; the command is then sampled and squashed as in (5.2). The same two vectors are the activities at the two ends of every synapse of the layer, the presynaptic $x_{j,t}$ and the postsynaptic $\mu_{i,t}$, and from them each weight computes its own Hebbian term, which is the bracket of (2.26) with $\boldsymbol{\mu}_t$ in the place of $\boldsymbol{y}$:

$$H_{ij,t} = A_{ij}\,x_{j,t}\,\mu_{i,t} + B_{ij}\,x_{j,t} + C_{ij}\,\mu_{i,t} + D_{ij}. \tag{5.6}$$

The weights used at the next step are

$$\boldsymbol{W}_{t+1} = \boldsymbol{W}_t + \eta\,\boldsymbol{H}_t - \lambda\,(\boldsymbol{W}_t - \boldsymbol{W}_0)\,, \tag{5.7}$$

where η is the learning rate and the last term is a decay towards the trained weights, to which subsection 5.2.3 is dedicated. The postsynaptic activity is the mean $\boldsymbol{\mu}_t$, taken before the sampling and before the hyperbolic tangent: the exploration noise of the policy does not enter the rule directly, and what the rule sees is the intention of the controller and not the squashed command. Figure 5.2 shows the two loops that result, the control loop that passes through the body and the plasticity loop that closes around the output layer.

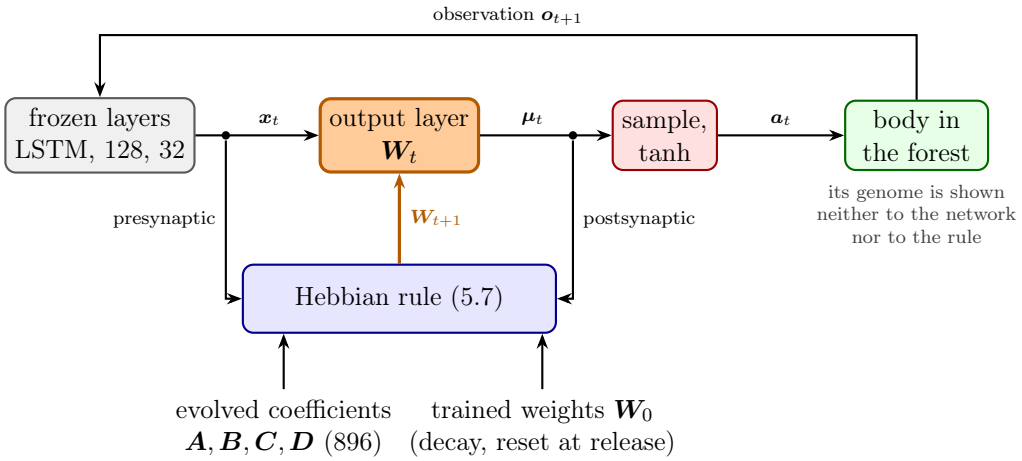


Figure 5.2: The Hebbian controller. Everything that precedes the output layer is frozen. At every control step the output layer computes the mean of the commands with its current weights, and the rule then rewrites the weights from the activities at the two ends of the layer, with coefficients that were evolved and do not change during the flight. The body is inside the control loop, so the activities from which the weights are rewritten carry its mark, although nothing describes the body to the network or to the rule.

The update inherits the properties discussed in subsection 2.4.1. It is local, since the change of a weight depends on the two activities at its ends and on nothing else, and it needs no reward, no error and no gradient, so that it runs during the

flight, inside the forward pass of the network, at the cost of a few multiplications for each of the 224 weights. This is the adaptation that section 3.4 asked for, one that costs no more than the flights that score the body. Every simulated flight has its own copy of $\boldsymbol{W}_t$, as it has its own LSTM state; what the flights share are the frozen layers and the coefficients.

Every weight follows its own rule: $\boldsymbol{A}$, $\boldsymbol{B}$, $\boldsymbol{C}$ and $\boldsymbol{D}$ are four matrices with the shape of $\boldsymbol{W}$, $4 \times 224 = 896$ coefficients in all, and they are the genome of the Hebbian controller, the object of the search of subsection 5.3.2. The rate η and the decay λ are shared by all the weights and are fixed. Within a flight the coefficients are constant and the weights change. The coefficients are also the same on every body, so that what differs from one body to another is the activity and, through it, the trajectory of the weights.

One property of this parametrization is used throughout the rest of the chapter. With all the coefficients at zero the Hebbian term vanishes and $\boldsymbol{W}_t = \boldsymbol{W}_0$ at every step: the frozen base controller is a point of the search space of the rules, its origin. Where the search of the rules starts (subsection 5.3.2) and what its results are compared with (subsection 5.3.5) both follow from this.

5.2.2 Why the Output Layer

The two parts of the network do different jobs. The frozen layers compute $\boldsymbol{x}_t$ from the history of observations and commands: a compact, high-level representation of the situation of the flight, in which what the sensors deliver, and what the LSTM has reconstructed of the quantities that they do not deliver, is condensed into 32 features. The output layer is the read-out that translates the features into the seven commands, and each W_{ij} is a gain: how much feature j moves actuator i. These 7×32 gains are the place of the network where the body matters most directly. What a situation calls for, raising the nose, turning away from a trunk, slowing down, is largely the same on every body. How much elevator, twist or throttle that takes is not: it depends on the size of the tail, on the span and the chord of the wing, on the mass, on the gear ratios of the joints. A generalist has to settle for one set of gains for all bodies. Changing the gains in flight changes the control law, and with it the dynamics of the closed loop that the body forms with its controller: how promptly it answers, how well it is damped, how tightly it turns. This is the role given to plasticity here: to keep the representation that the training built, and to retune at run time, for the body that is flying, the map from that representation to the actuators.

The choice has a counterpart in gradient-based meta-learning. MAML [19] learns an initialization from which a few gradient steps adapt a network to a new task, and Raghu et al. [60] showed that almost all of that adaptation takes place in the last layer: the features of the layers below are reused unchanged from one task to another, and an algorithm that adapts the last layer alone performs as well as the complete one. If the bodies are read as tasks, the generalist is a network whose features were trained across 256 of them, and the finding says that the last layer is where adapting to a new one pays. The difference is in the mechanism. In those works the last layer is adapted by gradient steps on a loss measured on the new

task; in flight there is no loss to descend, and the layer is adapted by a local rule whose coefficients were evolved so that the adaptation is a useful one.

There are practical reasons as well. The output layer is small: 224 weights give 896 coefficients, a number that an evolution strategy can search, whereas the LSTM alone has about 85 000 weights. It is linear, so the effect of a change of the weights on the commands is direct, $\Delta\boldsymbol{\mu}_t = \Delta\boldsymbol{W}\,\boldsymbol{x}_t$. And it is followed by the hyperbolic tangent of (5.2), which keeps every command inside the range of its actuator whatever the rules do to the weights.

The rules are never told which body is flying, and neither is the network below them. The body is nonetheless present in what the rules see, because it is inside the loop of Figure 5.2: the commands act on the body, the body answers according to its wings, its tail and its mass, the sensors report the answer, and the frozen layers turn it into the next $\boldsymbol{x}_t$. Two bodies that are given the same commands produce different sequences of activities and therefore, under the same rules, different weights. It is the mechanism by which identical rules differentiate the robots of a swarm [11], and here it differentiates the controllers of different bodies. As an illustration, on a body whose tail is small for its weight a pitch error lasts longer, and the features that carry it stay active together with the elevator command; a positive correlation coefficient on those synapses then raises the gain of the elevator more than it does on a body that answers at once. Whether evolution finds rules of this kind is a question for chapter 6. What matters here is that information about the body reaches the rule through the activity that the body induces, without being given.

There is finally the condition under which evolution selects plasticity at all: something must vary between lifetimes that fixed weights cannot anticipate [68]. When nothing does, fixed weights win, as they do on the intact quadruped of Najarro and Risi [52]. In this thesis what varies is the body. In the co-design loop a set of rules is never scored on one body: it is scored on all the bodies that the morphology search is considering at that moment, a population that changes as the search proceeds (section 5.3), and no single setting of the 224 gains can be right for all of them, which is the compromise of the generalist written in (5.5). The body is fixed within a flight, hidden from the controller and different from one flight to the next: the situation in which a rule that adapts within the lifetime has something to gain over any fixed weights. The rules evolved on the specialist of subsection 5.1.5 are the opposite case, in which the body does not vary and the weights were trained for it.

5.2.3 Weight Decay and the Lifetime of the Weights

Without its last term, (5.7) is an integrator: the weights are the trained ones plus η times the sum of all the Hebbian terms since the release, and a sum forgets nothing. A term with a mean different from zero makes the weight drift for as long as the flight lasts. A constant $D_{ij} = 1$ moves its weight by 0.005 at every step and by 7.5 over a flight of one minute, when the weights of the trained output layer have a mean absolute value of 0.05 and the largest of them is 0.42. The correlation term is worse, because it feeds on itself: a weight that grows raises the output, a larger output raises the product $x_j\,\mu_i$, and the product makes the weight grow faster. This is the runaway anticipated in subsection 2.4.2, and in a controller it ends with the

commands pinned at their limits.

The remedy recalled there is a decay of the weights towards zero, and it cannot be used as it is. A controller whose output weights are zero answers every situation with its biases, and a decay towards zero would keep erasing what the training built. The resting state of the plastic layer has to be the trained controller, so the decay of (5.7) acts on the deviation $\boldsymbol{W}_t - \boldsymbol{W}_0$ and not on the weights. Written for the deviation, the update is a linear recursion with the closed form

$$\boldsymbol{W}_t - \boldsymbol{W}_0 = \eta \sum_{k=0}^{t-1} (1-\lambda)^k \, \boldsymbol{H}_{t-1-k}. \tag{5.8}$$

The plastic weights are the trained weights plus a trace of the recent activity, in which the Hebbian term of k steps earlier is weighted by $(1-\lambda)^k$. It is the decomposition of (3.2), a part that is fixed and a part that is written by experience and forgotten geometrically, with a trained controller as the fixed part.

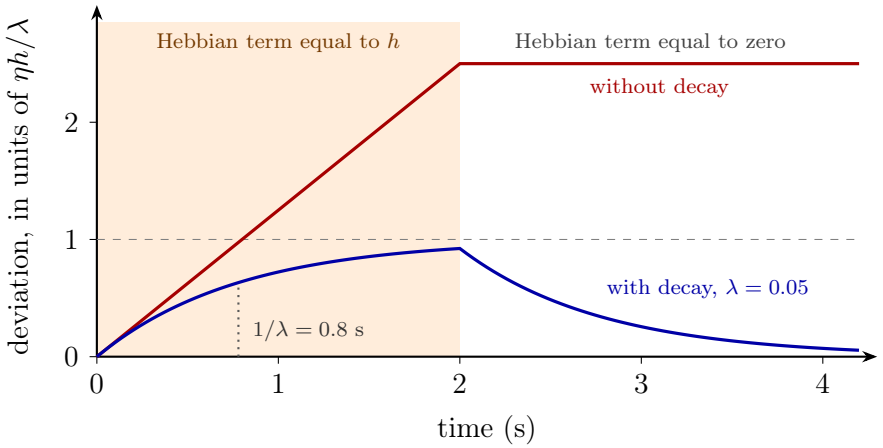


Figure 5.3: Deviation of one weight from its trained value when its Hebbian term is held at a constant value h for two seconds and then vanishes. Without decay the weight integrates: it grows for as long as the term lasts and keeps what it has accumulated. With the decay of (5.7) it settles at $\eta h/\lambda$ with a time constant of 0.8 s, and returns to the trained value at the same rate when the term vanishes.

Three properties follow from (5.8). The first is that the deviation is bounded whenever the activity is. The weights $(1-\lambda)^k$ of the trace sum to $1/\lambda$, so the deviation is η/λ times a weighted average of the recent Hebbian terms and cannot exceed η/λ times the largest of them; a Hebbian term held at a constant value h takes the weight to $\eta h/\lambda$ from its trained value and no further (Figure 5.3). The ratio η/λ is the authority of the rules over the weights. The second property is a time scale. The trace has a memory of $1/\lambda$ control steps: a deviation covers about two thirds of the way to its final value in that time, and is forgotten at the same rate when the activity that sustained it ceases. With the value used here the memory is 20 steps, 0.8 s of flight. The weights therefore describe the last second of the flight and not the whole of it. They settle during the approach, since the 30 m that separate the release from the first trees take about two seconds to fly, and they keep following the flight afterwards, so that the gains of a turn need not be those of

straight flight. Plasticity, with these values, is closer to a continuous retuning of the gains, according to a law that was evolved, than to a slow learning that accumulates over the flight. The third property is that the length of the flight drops out: nothing in the weights depends on how long the drone has been flying, and rules evolved on a short course remain meaningful on a longer one.

The decay does not make the runaway impossible: it gives it a threshold. Since $\boldsymbol{\mu}_t$ depends on the weights, the Hebbian terms of row i contain the deviation of that row, multiplied by the loop gain $g_i = \sum_j (A_{ij}\, x_j + C_{ij})\, x_j$. Without decay any positive gain, however small, makes the deviation grow geometrically. With it, growth requires $\eta\, g_i > \lambda$, which with the values used here means $g_i > 10$. In the flights of the frozen generalist the squared norm of $\boldsymbol{x}_t$ is about 5 on average and seldom above 12, so the threshold can be reached only by rules whose coefficients are large, and of concordant sign, along a whole row. Such rules saturate the commands, the drone crashes, and selection removes them.

The closed form also tells the four coefficients apart. D_{ij} is not plastic at all: after the first second its contribution is the constant $\eta D_{ij}/\lambda$, a correction of the trained weight that is the same on every body and in every flight. Through $\boldsymbol{D}$ the search can retune the output layer itself. The other three depend on the activity, and they are the ones that can tell one body from another: B_{ij} adds to the weight a filtered copy of its own input, C_{ij} a filtered copy of the command that it serves, and A_{ij} their filtered product, the correlation between what the network perceives and what it commands.

A lifetime, in the sense of subsection 2.4.3, is one flight. At every release the weights are set back to $\boldsymbol{W}_0$ and the state of the LSTM to zero, and nothing of what the weights became during a flight is carried to the next. This makes the flights independent samples. The score of a set of rules is an average over flights whose order does not matter, and the flights can be simulated side by side. It also forces the rules to adapt within the flight, every time from the same starting point; with a memory shorter than a second there would in any case be little to carry over.

5.2.4 Parameters of the Plastic Layer

Table 5.4 collects the quantities that define the plastic layer.

Two of these values are less independent than they look. The rate η multiplies all four coefficients, so that only the products matter: its value, together with the range of the coefficients, fixes the largest step that a weight can take, 0.0075 per unit of activity, and a different η with a rescaled range would describe the same family of rules. The decay is a true parameter, because it sets the memory. Both are constants: the rate is not modulated by any signal during the flight, so the neuromodulation mentioned in subsection 2.4.2 is not used. Both are shared by all the weights and were set by hand in preliminary runs. They could have been evolved for each weight, at the price of 448 more coefficients, and this was not done: the memory of the weights is thus a choice of the designer, and what the rules do within it is left to evolution.

The third value to read with care is the range. With $\eta/\lambda = 0.1$, a coefficient D_{ij} at the limit of its range moves a weight by 0.15, three times the mean absolute value

Quantity	Value	Meaning
Plastic weights	224 (7×32)	the matrix $\boldsymbol{W}$ of (5.1); the bias and all the other layers are frozen
Evolved coefficients	896	A_{ij}, B_{ij}, C_{ij} and D_{ij} for every weight: the genome of the controller
Range of a coefficient	$[-1.5, 1.5]$	bounds of the search; all coefficients at zero give the frozen controller
Learning rate η	0.005	size of a single update; shared by all weights, not evolved
Decay λ	0.05 per step	memory of the weights, $1/\lambda = 20$ steps or 0.8 s; shared by all weights, not evolved
Ratio η/λ	0.1	deviation of a weight per unit of sustained Hebbian term
Bound on a weight	$\pm 5\,000$	numerical guard, not reached by viable rules
Update	every control step	25 times per second, after the command of the step has been computed
Initial weights	$\boldsymbol{W}_0$	the trained weights, restored at every release together with a zero LSTM state

Table 5.4: Parameters of the plastic output layer.

of the trained weights. The activities are mostly below one in magnitude, with a root mean square of 0.4 for the features and of 0.5 for the outputs, so the other three coefficients move a weight by a few hundredths per unit, which is the order of the trained weights themselves. The range is therefore wide enough for the rules to overwrite the output layer, and not only to perturb it, which decides where the search of the rules has to start (subsection 5.3.2).

5.3 The Co-Design Pipeline

The previous section defined the Hebbian controller but left its 896 coefficients unset, and the design space of the drone leaves the body just as open. The co-design pipeline searches both at once, with two evolutionary algorithms nested one inside the other (Figure 5.4). The *inner loop* evolves the rules with CMA-ES (subsection 2.3.1): at every generation it flies a population of candidate sets of rules, on top of the frozen generalist, on all the bodies that are under consideration at that moment. The *outer loop* evolves the bodies with NSGA-II (subsection 2.3.2): once every few generations of the inner loop, a *phase*, it puts the current bodies through an *exam* under the best rules found so far, keeps the better half and replaces the rest. Every flight of the pipeline is a rollout of a network that is never trained again, which is what section 3.4 required of the adaptation.

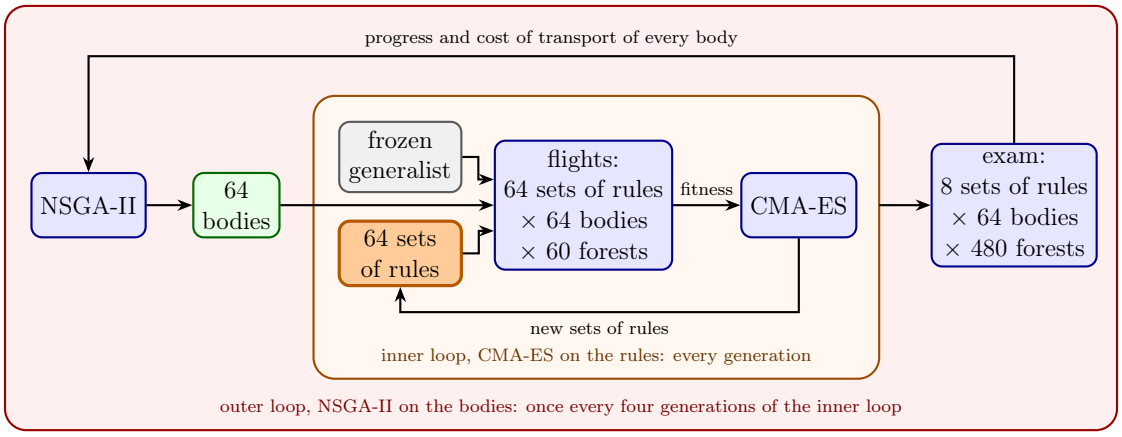


Figure 5.4: The co-design pipeline. In the inner loop CMA-ES proposes 64 sets of rules, each of them is flown on top of the frozen generalist on every body of the current population, and the mean reward of its flights is its fitness. Every four generations the eight best sets of rules fly the same bodies on a harder course, the exam, and the progress and the cost of transport measured there are the two objectives by which NSGA-II keeps half of the bodies and replaces the other half. The search of the rules is not restarted when the bodies change.

The names of the two loops need a word, because they do not map onto the two levels of (3.1) one to one. The outer loop is the search over $\boldsymbol{m}$ of that equation. The inner loop is not its inner problem, which would have to be solved once for every body: it evolves one set of rules for all the bodies of the population, and what is specific to a body, the adaptation of the output weights, takes place inside

each flight, at no cost beyond the flight itself. Inner and outer refer to how the two searches are nested in time, several generations of the rules for every update of the bodies. This section describes the flights on which both loops rest, then the two loops, the course of a complete run, and the control condition against which a run is compared.

5.3.1 Evaluation Flights

Both loops judge by flying, and the flights differ from those of training (Table 5.1) in what is randomized. In training every flight starts from a different state, because the controller has to learn to recover; here flights are measurements, and everything that is not meant to be compared is held fixed. The drone is released at $(-30, 0, 15)$ m, level, at 15 m/s along the corridor. The commanded speeds are evenly spaced between 10 and 20 m/s, one for each forest, so that every body and every set of rules is judged over the same range of speeds, from a slow flight that leaves time to react to a fast one that does not. A flight ends as described in section 4.2, with the collision margin of 1 cm, attitude limits of 90° in pitch and yaw and 100° in roll, a little wider than in training so that a steep turn is not counted as a loss of control, and a time limit of 60 s.

What is not held fixed is the noise. The sensor noise, the aerodynamic noise, the perturbation of masses and centres of mass, the latency of the commands and the offsets of the joint targets are those of training, drawn independently in every flight, and the commands are sampled from the policy as in (5.2): a flight of the pipeline is as stochastic as a flight of training. The plastic weights start from $\boldsymbol{W}_0$ at every release (subsection 5.2.3). Three quantities are recorded for each flight: the sum of the rewards of (5.3) over its steps, its progress, and the energy it consumed.

Two courses are used (Table 5.5). The *search course*, on which the rules are evolved, is short: 100 m, from an empty entrance to five trees per metre. A generation of the inner loop flies it a quarter of a million times, and most of what distinguishes a good set of rules from a bad one happens in the first hundred metres. The *exam course*, on which the bodies are judged, is three times longer, and is the kind of forest in Figure 4.2: it starts with one tree per metre, grows more slowly than the search course, and goes on to 5.5 trees per metre at 300 m. The forests of an exam are drawn anew, but the course they are drawn from, its length and its density profile, is the same at every exam of a run: what the exams have to show is that the bodies of a late phase are better than those of an early one, that is, that the evolution is making progress.

5.3.2 Inner Loop: Evolving the Hebbian Rules

The genome of the inner loop is the vector of the 896 coefficients, each encoded by a gene in $[0, 1]$ that is mapped linearly onto the range $[-1.5, 1.5]$ of Table 5.4. CMA-ES searches this unit cube with a population of 64, a full covariance matrix and the default settings of its reference implementation for everything else; samples that leave the cube are brought back by the boundary handling of the library.

	Search course	Exam course
Used by	the inner loop, at every generation	the outer loop, at the end of every phase
Length	100 m	300 m
Density at the entrance, λ_0	0 trees per metre	1 tree per metre
Density at the end, λ_1	5 trees per metre	5.5 trees per metre
Trees per forest	250	975
Forests	60, new at every generation	480 for each set of rules, new at every exam
Commanded speeds	60 values from 10 to 20 m/s	480 values from 10 to 20 m/s

Table 5.5: The two courses of the pipeline, both growing density forests (subsection 4.2.1) in the same corridor. Release, end of a flight and noise are the same on both.

Starting from zero. The initial mean of the search is the centre of the cube, where every coefficient is zero and the Hebbian controller is the frozen generalist (subsection 5.2.1), and its initial step size is 0.075 in units of the cube, 0.225 in units of the coefficients. The first generation is thus a cloud of small perturbations of a controller that already flies, which move a weight by about 0.02 where the trained weights have a mean absolute value of 0.05, and the search grows the rules outwards from there, as far as selection allows. A start from random rules was tried. With coefficients drawn from their whole range the rules overwrite the output layer within a second (subsection 5.2.4), and in preliminary runs started in this way almost every flight ended in a crash within a few seconds: the search was handed a population in which nothing flew, and had to find its way back to a working controller before it could improve on it. Starting from zero reverses the burden. A set of rules has to earn its departure from the trained controller, and a search that finds nothing useful stays where it started, at the frozen generalist.

One generation. Each of the 64 candidate sets of rules is flown on every body of the current population, 64 bodies, over 60 forests of the search course: $64 \times 64 \times 60 = 245\,760$ flights, all simulated at the same time, with one scene for each body as in the training of the generalist. The 60 forests and the 60 commanded speeds are the same for every set of rules and for every body, so that no candidate is favoured by an easier draw, while the noise of subsection 5.3.1 differs from flight to flight. The forests are generated anew at every generation: a set of rules is never selected twice on the same trees, and the search cannot fit the rules to a particular sample of forests.

Fitness. The fitness of a set of rules is the mean, over its $64 \times 60 = 3\,840$ flights, of the reward summed over the flight. It is the reward of subsection 5.1.3, unchanged, and not the pair of quantities by which the bodies are judged. The choice is a practical one. CMA-ES optimizes a single number, and the cumulative reward is a number that already exists: it weighs speed, survival and energy against each other, and it is known to produce a good pilot, since it trained one. Evolving the rules on it avoids engineering a second fitness function, with weights of its own to tune, for a task that already has one. The price is the one stated in subsection 5.1.3, that the correspondence between the reward and the two objectives is only approximate. Since CMA-ES uses the ranking of the candidates and not their fitness values (subsection 2.3.1), what matters is that the ranking is reliable, and both remedies recalled there are used: each fitness is an average over 3 840 stochastic flights, and the population is 64 where the default for 896 dimensions would be 24.

A last remark concerns what the rules are evolved for. One set of rules serves all the bodies, and its fitness is an average over them, which is the structure of the compromise of (5.5). There are two differences. First, what is averaged is a procedure that gives different weights to different bodies (subsection 5.2.2), where the generalist is one fixed set of weights. Second, the average is taken over the 64 bodies that the outer loop currently retains. This set is narrower than the whole design space, and it moves towards the bodies worth flying as the run proceeds. The average has a bias of its own: the reward is summed over the flight, so a body that

flies for long weighs more in it than one that crashes early, and the rules are selected above all on the bodies that already fly.

The inner loop can also run alone, on a set of bodies that is never updated. This is how the rules are evolved on the reference drone on top of its specialist (subsection 5.1.5): the population of bodies is that single body, and all the flights of a generation are flights of it.

5.3.3 Outer Loop: Evolving the Drone Morphology

The outer loop holds a population of 64 bodies. The first population is drawn uniformly at random from the unit hypercube of the genome, like the pool on which the generalist was trained but independently of it, and the reference drone is not among its members: whatever the search finds, it finds without being shown a design that works. The bodies stay fixed for a phase of four generations of the inner loop, during which the rules adapt to them, and at the end of the phase they are examined and updated.

The exam. The objectives of a body are measured in a separate set of flights, the exam. At the end of a phase the eight sets of rules with the highest fitness in the last generation, one eighth of the population, fly every body on the exam course, each over 480 newly generated forests, so that every body is judged on $8 \times 480 = 3\,840$ flights. The exam exists for three reasons:

1. Its course is longer than the search course and forest generator parameters do not change along the run, so that the scores of successive exams tell whether the bodies are improving.

2. Its forests are new, whereas the forests of the last generation are the ones on which those eight sets of rules were selected, and a score measured on them would be biased in their favour.

3. Each set of rules flies eight times as many forests as in a generation of the inner loop, which the ranking of 64 bodies on two noisy objectives needs.

Using eight sets of rules and not the best one alone makes the score of a body depend less on the accident of which candidate came first in a noisy ranking: what is measured is what the body achieves under the good rules of the moment.

Objectives. For every body and every one of the eight sets of rules, the *progress* is the mean of the progress of the 480 flights, and the *cost of transport* is (4.4) computed on the totals: the energy consumed in all the flights, divided by the weight of the body times the distance covered in all of them. The ratio of the totals is not the mean of the ratios, and the difference matters. A flight that ends a few metres after the release has a cost of transport of its own that is meaningless, and that a mean would either have to discard, rewarding the bodies that crash at once, or let dominate. In the ratio of the totals such a flight counts for the little energy it spent and the little distance it covered. The two objectives of the body are the

means, over the eight sets of rules, of these two quantities, the first to be maximized and the second to be minimized.

Admission. A body whose progress in the exam is below 80 m is not admitted to the selection. Without this gate the front fills with designs of no interest: a body that glides to the ground after twenty metres with its throttle closed has spent almost nothing per metre, nothing dominates it on the second objective, and NSGA-II dutifully keeps it and breeds from it. The gate removes such bodies from the survivors and from the parents; if fewer bodies pass it than there are places for survivors, the places left empty go to offspring.

Update. Among the admitted bodies, the 32 best under the crowded comparison of NSGA-II survive. The other 32 places are filled with offspring: two parents are drawn by binary tournament among the admitted bodies, recombined by simulated binary crossover (with probability 0.9 and distribution index 15) and mutated by polynomial mutation (each gene with probability 1/15, distribution index 20), with the genes kept in $[0, 1]$. The operators treat the three airfoil genes as real numbers, and the discrete value is taken when the body is built. The 64 bodies of the new population are then generated, a scene is compiled for each of them, and the next phase begins.

In the terms of subsection 2.3.2 the survivors are the parents P, the offspring are Q, and the population that flies is the merged pool $P \cup Q$. The difference from the textbook algorithm is that the pool is scored again, as a whole, at every exam. The scores of the survivors are not carried over from the phase in which they were selected, because the rules have moved in the meantime, and the score of a body under an old controller is worthless. All 64 bodies of an exam are flown by the same sets of rules, on the same forests, at the same speeds: for selection, bodies are only ever compared within an exam. Across exams only the fronts are compared, as a measure of the progress of a run: the length and the density of the course are fixed, its forests are drawn anew, and the sets of rules are those of the moment.

At every exam one more measurement is taken, which does not enter the search: the reference drone, flown by the frozen generalist without rules, on 4 096 forests of the exam course. It is the point against which the fronts of chapter 6 are read.

5.3.4 The Course of a Run

A run is 200 generations of the inner loop, that is 50 phases (Figure 5.5). It starts with the rules at zero and with 64 random bodies, most of which fly badly. For four generations CMA-ES perturbs the rules and is told which perturbations helped, on those bodies. The first exam then ranks the bodies under rules that are still close to the frozen generalist, the gate discards those that do not get through the first 80 m, and the first update breeds from the rest. From here on the two searches shape each other. The rules are selected on the bodies that survive, which resemble each other more at every update, so that the question put to them narrows from flying anything to flying well this family of bodies; and the bodies are selected under the rules of the moment, so that a body whose value shows only with a controller adapted to

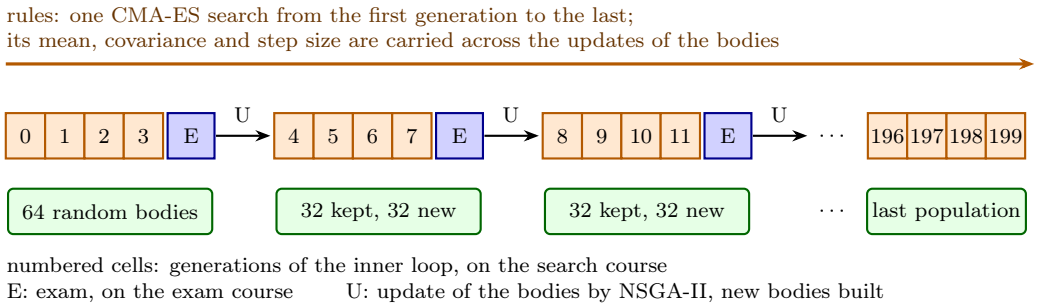


Figure 5.5: The course of a run of 200 generations. The bodies are fixed for phases of four generations of the inner loop; every phase but the last ends with an exam and with an update of the bodies, in which half of the population survives and is examined again at the end of the next phase.

it is no longer discarded, which is what the challenge of morphology optimization asked for.

The search of the rules is never restarted. When the bodies are updated, CMA-ES keeps its mean, its covariance and its step size, and goes on from where it was. Half of the bodies are the same as before and the new ones are their offspring, so the rules that served the old population are a good starting point for the new one, and a restart would throw away at every update what the four generations had gained. The cost of the choice is that the fitness of the rules is not comparable across phases, since the bodies on which it is measured change; within a generation, which is where CMA-ES uses it, it is.

A run holds 49 exams, since the last phase ends with the run and is not examined, and therefore scores $49 \times 64 = 3\,136$ bodies, of which at most $64 + 48 \times 32 = 1\,600$ are distinct. Every examined body is recorded with its genome and its two objectives, and this archive is the material from which the Pareto fronts of the results are computed, among the bodies that passed the gate. Two further measurements are taken along the run and do not enter the search: every four generations the frozen generalist without rules is flown on the same bodies and the same course, and at every generation the best set of rules and the frozen generalist are flown with the reference drone on 4 096 new forests of the search course, which the search never sees; this is not the measurement of subsection 5.3.3, which is taken on the exam course. The first shows what the rules add on the bodies they are evolved on. The same measurement is reported in subsection 6.1.1 for a search of the rules without the outer loop. The second shows what becomes of the rules on a body they are not evolved on, and subsection 6.2.5 reports it. Table 5.6 collects the settings; with them a run takes about four days on the node of Table 3.1, most of them spent in the flights, and the rest in building the bodies and compiling their scenes at every update.

5.3.5 The Control Condition

What the pipeline is compared with is the search of section 3.3: the same bodies, judged under one fixed controller. Because the frozen generalist is the Hebbian controller with its coefficients at zero (subsection 5.2.1), that search does not need

Quantity	Value
Generations of the inner loop	200
Sets of rules per generation	64
Initial mean and step size of CMA-ES	all coefficients at zero; 0.075 in the unit cube
Flights per generation	245 760 (64 sets of rules × 64 bodies × 60 forests)
Fitness of a set of rules	mean summed reward over its 3 840 flights
Bodies in the population	64, the first drawn at random
Length of a phase	4 generations
Exam	8 best sets of rules × 64 bodies × 480 forests
Objectives of a body	progress (maximized), cost of transport (minimized)
Admission gate	progress of at least 80 m in the exam
Survivors and offspring per update	32 and 32
Crossover	simulated binary, probability 0.9, index 15
Mutation	polynomial, probability 1/15 per gene, index 20

Table 5.6: Settings of the co-design pipeline. The courses are those of Table 5.5, the parameters of the plastic layer those of Table 5.4.

a second implementation. The *morphology-only* runs are runs of the same pipeline with the learning rate η set to zero, so that no set of rules changes any weight and every flight is flown by the frozen generalist. The base controller, the courses, the exam, the gate and the operators of NSGA-II are the same, and so are the random seeds, so that a morphology-only run starts from the same 64 bodies as the co-design run it is paired with. The two are flown under the same conditions and differ in the coefficients; their schedules differ too, as described next.

With rules that do nothing there is no inner search to wait for, and the schedule is adapted accordingly, so that the bodies are updated at every generation. In the exam the single controller flies every body on 3 840 new forests, which gives each body the same number of exam flights as in a co-design run. A morphology-only run lasts 88 generations, that is 87 exams and 5 568 examined bodies against 49 and 3 136, and about 130 hours of the node against 94. The co-design pipeline flies more in total, because four fifths of its flights go to the search of the rules, which the control does not need; in computing time, and in everything that concerns the bodies, the control is given more than the pipeline it is compared with, so that a better front of the pipeline cannot be the effect of a larger budget.

Experiments And Results

This chapter has two parts, which follow the two levels of the pipeline. The first part, section 6.1, isolates the inner loop. The outer loop is switched off and the bodies are given rather than selected. The question is what the evolved rules add to a base controller on a body that does not change: first to the generalist, on bodies it has never flown, then to a specialist, on the one body it was trained for. The second part, section 6.2, runs the whole pipeline and compares it with the morphology-only control of subsection 5.3.5. The comparison is made on the terms set by chapter 3: whether an adaptation as cheap as this one changes which bodies the search finds.

Every experiment of the chapter uses the controllers, the plastic layer, the courses and the flights of chapter 5, with the values of Table 5.4, Table 5.5 and Table 5.6. Where an experiment departs from them, the departure is stated with the result it belongs to. The base controllers are the generalist of subsection 5.1.4, trained once, and the specialists of subsection 5.1.5. Three specialists were trained for the reference drone with the same settings and different random seeds; unless a specialist is numbered, the first of them is meant. The evolutionary algorithms are taken from their reference implementations: CMA-ES from pycma 4.4.4 [31] and NSGA-II from DEAP 1.4.3 [22]. The networks run in PyTorch 2.10 [55] and the simulation in Genesis 0.3.7 [23]. A run is identified by its random seed, which fixes the samples of CMA-ES, the forests and, where there is one, the first population of bodies. Table 6.1 lists the runs of the chapter.

6.1 Inner Loop Experiments

The inner loop of subsection 5.3.2 was designed to run inside the outer loop, on bodies that the outer loop keeps replacing. Run on its own, on bodies that are given once and for all, it answers a narrower question. This question is nevertheless the first one to ask: whether the rules add anything to a base controller, and what they add. On a fixed body nothing varies from one flight to the next but the forest, the commanded speed and the noise. The body, which subsection 5.2.2 presented as the quantity that varies between lifetimes and that fixed weights cannot anticipate, is the same in every flight. The fixed body is therefore the setting in which the argument for plasticity promises the least. It is also the setting in which whatever the rules do can be read most clearly: the frozen base controller flies the same body under the same conditions, so any difference between the two is the work of the rules.

Two experiments are reported. In the first the base is the generalist, which flies every body as a compromise (subsection 5.1.4). The rules are evolved on many bodies, as in the pipeline, and then on the reference drone alone. The specialist trained for that drone gives the scale of what the generalist leaves on the table there. In the second experiment the base is a specialist, a controller that was trained for the body it flies and has nothing to recover. There are three specialists, and the experiment is run on each of them.

Section	Base controller	Bodies	Flights per generation	Measured at every generation, on new forests	Runs, generations
6.1.1	generalist	64 drawn at random, moved every 4 generations	245 760	the best rules and the frozen generalist on the reference drone, 4 096 forests	1, of 50
6.1.1	generalist	the reference drone	12 800	the best rules, the frozen generalist and a frozen specialist, 2 048 forests	3, of 200
6.1.2	a specialist	the reference drone	12 800	the best rules, the frozen specialist and the frozen generalist, 2 048 forests	3, of 200, one per specialist
6.2	generalist	64, half replaced at every exam	245 760	the exam every 4 generations (every 6 in two runs): the 8 best rules × 64 bodies × 480 forests of the exam course	8, of 200
6.2	generalist, rules held at zero	64, half replaced at every exam	245 760, all in the exam	the exam every generation: the frozen generalist × 64 bodies × 3 840 forests of the exam course	8, of 88

Table 6.1: Runs of this chapter. Every run uses 64 sets of rules per generation, the search course and the noise of subsection 5.3.1; a flight of the search is one set of rules, one body and one forest.

6.1.1 Recovering Performance on Generalist Controllers

The first experiment runs the inner loop as the pipeline runs it, with one change: the bodies are not selected. Sixty-four bodies are drawn at random, as in the first generation of a run, with the reference drone excluded. Every four generations, each gene of each body is moved by a Gaussian step of standard deviation 0.02 in the unit cube. The population thus keeps changing under the rules, as it does in a run, but it drifts as a random walk instead of following the objectives. Fifty generations are run, with twelve such refreshes, and with the population, the flights and the fitness of Table 5.6: 64 sets of rules, 245 760 flights of the search course per generation, forests drawn anew at every generation. At every generation the frozen generalist flies the same bodies on the same forests. In addition, the best set of rules of the generation and the frozen generalist fly the reference drone on 4 096 further forests of the search course, which the search never sees; this is the measurement described in subsection 5.3.4. The experiment thus shows the progress of the inner loop on the bodies it is given and on one body it is not given.

Figure 6.1(a) shows the fitness on the bodies of the search. At the first generation every one of the 64 sets of rules flies worse than the frozen generalist: the population's

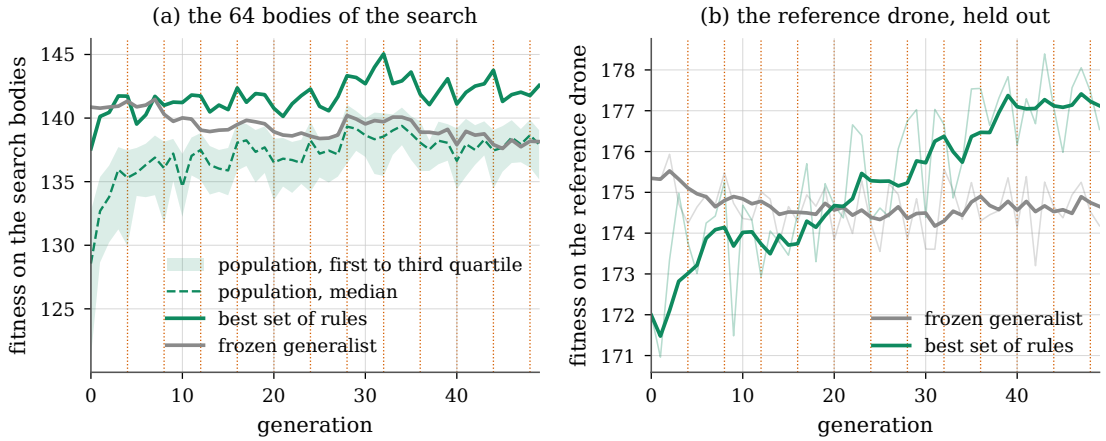


Figure 6.1: The inner loop on 64 random bodies. (a) Fitness on the bodies of the search: the best of the 64 sets of rules of each generation, the population's median with a band from the first to the third quartile, and the frozen generalist on the same bodies and forests. The orange lines mark the refreshes of the bodies. (b) The best set of rules of each generation and the frozen generalist on the reference drone, which the search never flies, over 4 096 new forests per generation; thin lines are the raw values, thick lines a moving average over five generations.

median is 129 against 141 for the frozen controller, and its best is 138. This is the starting point that subsection 5.3.2 chose. The rules begin as small perturbations of a controller that flies, and a perturbation, on average, costs. From there CMA-ES moves. From the seventh generation on, the best set of rules of each generation is above the frozen controller. Over the last ten generations the best set exceeds the frozen controller by 3.9, 142.2 against 138.2, a gain of about 3 %. The population's median, however, only catches up with the frozen controller, 137.9 against 138.2 over the same generations, and its upper quartile barely passes it. The search has found a direction in which the rules help, but a random perturbation around that direction still costs about as much as the direction gains. The population as a whole never becomes clearly better than the controller without rules. The curve of the frozen controller is not flat, because at every refresh the bodies change and the new population flies a little differently. This is the reason given in subsection 5.3.4 why the fitness of the rules cannot be compared across updates of the bodies.

Panel (b) is the same comparison on a body that the search never flies. On the reference drone the best rules of the first generation are worse than the frozen generalist by 3.4, and they remain worse for the first four phases, sixteen generations: rules selected on other bodies do not transfer at once. From the sixth phase on the sign is reversed. Over the last ten generations the best rules exceed the frozen controller by 2.5 ± 0.6 on the reference drone, 177.1 against 174.6, or 1.4 %.ssc The fitness of the frozen controller alone varies by 0.6 from one generation's forests to the next, so the gain is well above what a draw of forests can produce. The gain is also two thirds of what the rules gain on the bodies they are evolved on. This is the expected direction: part of what a set of rules gains is specific to the bodies on which it was selected.

Decomposed into the quantities of the flights, the held-out gain is of one kind. Progress and the fraction of flights that end in a crash, in the sense of subsection 5.1.3, barely move: 117.2 m against 116.8 m, and 52 % of the flights lost against 53 %, on a course that ends at five trees per metre. What moves is the speed and the energy. The rules lower the speed by 2 %, from 13.9 to 13.6 m/s over commands between 10 and 20 m/s, and they lower the energy per metre more, by 4 %. The rules do not make the generalist a better pilot of the reference drone in the sense in which a specialist is one. They make it fly the same course, with the same losses, a little slower and cheaper, and the reward pays for the change. Its energy term rewards the saving directly. Its tracking term is a bell curve that is flat near the commanded speed, so a flight a few per cent slower costs almost nothing per step, while the flight lasts longer over the same distance. The first generations show the opposite, a cost of transport above that of the frozen controller, and the search passes through it. Which term of the reward the rules end up serving is not decided at the start.

How much of what the generalist loses on the reference drone is this? The specialist trained for that drone gives the measure. In a second run the rules were evolved on the reference drone alone, with the same population and 200 forests for each set of rules, 12 800 flights per generation, for 200 generations. At every generation the best set of rules, the frozen generalist and the frozen specialist flew 2 048 fresh forests of the search course. Figure 6.2 shows the three curves. The frozen generalist scores 174.5 on the reference drone and the specialist 182.2. The gap of 7.7, or 4 %, is the price of the compromise of section 3.3 on a body of ordinary proportions. The rules, evolved on this body alone, recover 3.4 ± 1.2 of the gap over the last fifty generations, that is 44 % of it. The gain appears within the first twenty generations, reaches 3 by the sixtieth and hardly moves afterwards. The gap itself depends on which specialist is used as the measure. The three specialists that were trained score 182.2, 189.7 and 182.4 on the same kind of forests (subsection 6.1.2), so the rules recover between a quarter and a half of what a specialist has over the generalist.

But the two gains are of a different kind. The specialist crashes 28 % of its flights where the generalist crashes 53 %, and it covers 124 m where the generalist covers 117 m. It flies at the same speed as the generalist to within a tenth of a metre per second, and it spends more per metre, with a cost of transport of 0.32 against 0.28. The rules leave the losses and the progress where the generalist has them, and they lower the cost of transport by 9 %, to 0.26, at a speed 3 % lower. Measured in reward, the rules recover over two fifths of the gap. Measured in what the body does, they recover none of the specialist's advantage, which lies in avoiding the trees, and they find instead an economy that the specialist never looked for. In the terms of the objectives of subsection 5.3.3 the two changes are not on the same axis. The specialist moves the body along progress and pays for it in energy; the rules move it along the cost of transport at constant progress. This is the approximate correspondence between the reward and the objectives of which subsection 5.1.3 warned, seen from the side of the search: a change that the reward pays for need not be the change that a specialist would have made.

Two further observations from this run are used later. The first concerns the selection. The fitness with which the best set of rules is selected, on the forests



Figure 6.2: The inner loop on the reference drone alone, with the generalist as base: fitness of the best set of rules of each generation, of the frozen generalist and of the frozen specialist, over 2 048 new forests per generation flown by all three. Thin lines are the raw values, thick lines a moving average over five generations.

of the search, is 182.0 over the last fifty generations, against 177.9 for the same set of rules on the held-out forests. The maximum of 64 noisy measurements is thus optimistic by about 4, and a set of rules that is the best of its generation by that margin has often only been lucky on that generation's forests. This is one of the reasons why the pipeline evaluates the bodies on new forests and under eight sets of rules rather than one (subsection 5.3.3). The second concerns repeatability. The run was repeated once with the same seed and once with another seed, for 200 generations each. Runs with the same seed diverge in the simulator, as every run does (subsection 4.1.1). The two repeats nevertheless agree with the first run in the result, with gains of 4.5 and 3.6 ± 1.0 over their last quarters and the same decomposition into speed and energy.

6.1.2 Improving Performance on Specialist Controllers

The second experiment repeats the last one with a specialist in the place of the generalist, once for each of the three specialists trained for the reference drone. The rules are evolved on top of the frozen specialist, on the reference drone, with the same population, the same 12 800 flights per generation and 200 generations. At every generation the best set of rules, the frozen specialist and the frozen generalist fly 2 048 fresh forests. Whether the rules can help a specialist was left open in subsection 5.1.5. A specialist was trained for the body it flies. The argument of subsection 5.2.2, that plasticity is selected when something varies between lifetimes, gives the rules nothing to work on here, because the body is the one the weights were trained for. The question is whether a controller that already fits its body leaves the rules anything to improve, and, if it does, what.

Table 6.2 gives the answer, and it is not the same for the three specialists. On the

	frozen				with the best rules				
Base controller	fitness	speed	CoT	crashes	fitness	gain	speed	CoT	crashes
generalist	174.5	13.9	0.285	53 %	177.9	+3.4	13.4	0.259	53 %
first specialist	182.2	13.9	0.320	28 %	191.9	+9.6	13.3	0.257	28 %
second specialist	189.7	13.3	0.215	31 %	190.2	+0.5	13.3	0.212	34 %
third specialist	182.4	13.9	0.344	29 %	192.3	+9.9	13.3	0.243	28 %

Table 6.2: The four base controllers on the reference drone, frozen and with the best set of rules of each generation, over the last fifty generations of their runs: fitness, mean speed along the corridor in m/s, cost of transport and fraction of flights ended by a crash, on 2 048 new forests per generation. The gain is the difference in fitness between the two, measured on the same forests; its spread over generations is 1.2, 1.2, 1.0 and 0.8 from top to bottom.

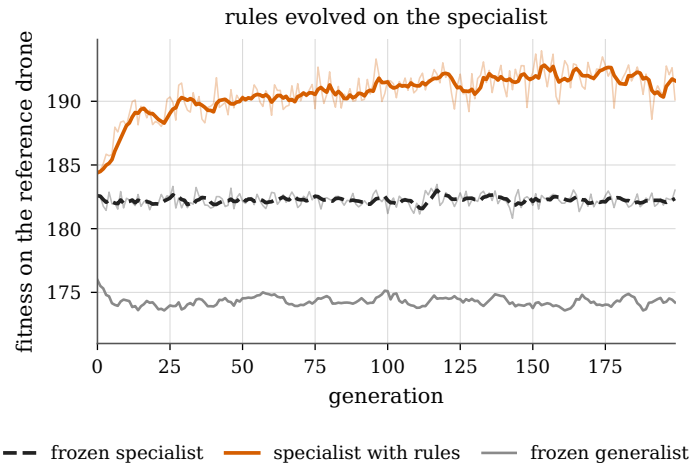


Figure 6.3: The inner loop on the reference drone alone, with the first specialist as base: fitness of the best set of rules of each generation, of that specialist frozen and of the frozen generalist, over 2 048 new forests per generation flown by all three. Thin lines are the raw values, thick lines a moving average over five generations.

first and the third specialist the rules add more than anywhere else in this section: 9.6 ± 1.2 and 9.9 ± 0.8, over 5 %, three times what they added to the generalist. Figure 6.3 shows the first specialist. The rules are above the frozen specialist from the first generation, by 1.8 at generation 0 and by 4.3 over the first ten generations. The gain reaches 7.8 by the fortieth generation and 9.0 by the hundredth, and the third specialist follows the same course. On the second specialist the rules add nothing: 0.5 ± 1.0, negative over the first ten generations and inside the noise ever after.

The frozen controllers explain the difference. The three specialists were trained with the same settings and differ in the random seed alone, and they did not arrive at the same way of flying the reference drone. The first and the third fly it at 13.9 m/s with a cost of transport of 0.32 and 0.34, dearer than the generalist. The second flies it at 13.3 m/s with a cost of transport of 0.215, and it scores 7.5 more than the other two for it, with the same progress and the same crashes. The rules bring the first and the third specialist down to 13.3 m/s and to a cost of transport of 0.26 and 0.24, with progress and crashes untouched, as they brought the generalist from 13.9 to 13.4 m/s and from 0.285 to 0.259. The second specialist is there already, and the rules leave it where it is. Two further differences from the generalist's run are worth noting on the first specialist:

1. The gain is there from the start, in the first generation, which is drawn with the same seed in the two runs; on the generalist, the best of the first generation was worse than the controller without rules.

2. The selection is less deceived by its own forests: the best fitness on the search forests exceeds the held-out fitness of the same rules by 2.5 instead of 4.

The flights of a specialist vary less from one forest to the next, as its fewer crashes suggest.

Where the four bases end up is the remarkable part, and Table 6.2 shows it. Under their respective rules, the generalist and the three specialists all fly the reference drone at the same speed, 13.3 to 13.4 m/s, three of them down from 13.9 m/s. The cost of transport ends between 0.21 and 0.26. It is lowest for the specialist that was already there and highest for the generalist. Progress and crashes stay where each frozen controller had them. The rules move each controller towards a slower and cheaper flight of the same course, the flight that the fitness of subsection 5.3.2 rewards on this body. What each controller gains is the distance it had to travel to get there: nothing for the specialist that was there already, 3.4 for the generalist, about ten for the two specialists that flew fastest and dearest. Why the training reached this point with one seed and not with the other two, the experiment does not say.

What a fixed body cannot show is the case that subsection 5.2.2 made for plasticity: a body that differs from one flight to the next, which the controller must fit without being told which one it is. That is what section 6.2 is for.

6.2 Full Co-Design Experiments

The second part of the chapter runs the pipeline of section 5.3 as a whole and compares it with the control of subsection 5.3.5. Eight co-design runs and eight morphology-only runs were made with the settings of Table 5.6, on eight random seeds. Six of the seeds carry a run of each condition, and the two runs of a seed start from the same 64 random bodies. The other two seeds carry a morphology-only run only. Two of the six shared seeds carry a second co-design run in which a phase lasts six generations instead of four, so that the run holds 33 phases instead of 50. All sixteen runs enter every comparison that treats the runs as independent. The paired analysis of subsection 6.2.3 uses the six shared seeds, with one co-design run for each. The morphology-only runs, whose bodies are updated at every generation, have 88 generations, 87 exams and 5 568 examined bodies, against 49 exams and 3 136 bodies for a co-design run. They took about 130 hours of the node against 94.

What a run leaves behind is the archive of subsection 5.3.4: every body it examined, with the two objectives measured on the exam course under the sets of rules of the moment. The result of a run is read from this archive as its *cumulative front*: the set of examined bodies that passed the 80 m gate and are dominated by no other examined body of the same run. The cumulative front is the best trade-off between progress and cost of transport that the run found, at whatever phase it found it. The point of comparison in every figure is the reference drone flown by the frozen generalist on the same course, measured at every exam as described in subsection 5.3.3: 190.8 m at a cost of transport of 0.288, the same in every run.

Fronts are sets, and comparing two groups of eight fronts needs indicators. Two are used, both read off a front at a fixed value of the other objective: the cost of transport of the cheapest body that reaches a given progress, and the progress of the farthest body within a given cost of transport. They are the attainment functions of Fonseca and Fleming [21, 44] read at one level. They answer the practical questions directly: what it costs to fly 170 m through this forest with the best body each search found, and how far a body within a given budget goes. The hypervolume of subsection 2.3.2 is used as well, with the reference point at 80 m of progress, the gate, and at a cost of transport of 0.5, above every body of every front. The unit of every test is the run. With eight runs in each group, the rank-sum test of Mann and Whitney [45] is used in its exact form, in which the p-value is computed over all 12 870 ways of assigning the sixteen runs to two groups of eight. The smallest p-value the test can return is 0.00016, reached when every run of one group is better than every run of the other; it is written as p-value < 0.001. The size of a difference is given as the Hodges–Lehmann shift [36]: the median of the 64 differences between a run of one group and a run of the other. The shift is reported with its exact 95 % confidence interval.

6.2.1 Co-Design vs. Morphology-Only Evolution

Figure 6.4 shows the sixteen cumulative fronts and, for each condition, the mean cost of transport at every progress over the runs whose front reaches it. The two families agree at their ends. At the tip, the farthest body of a run averages 213 to

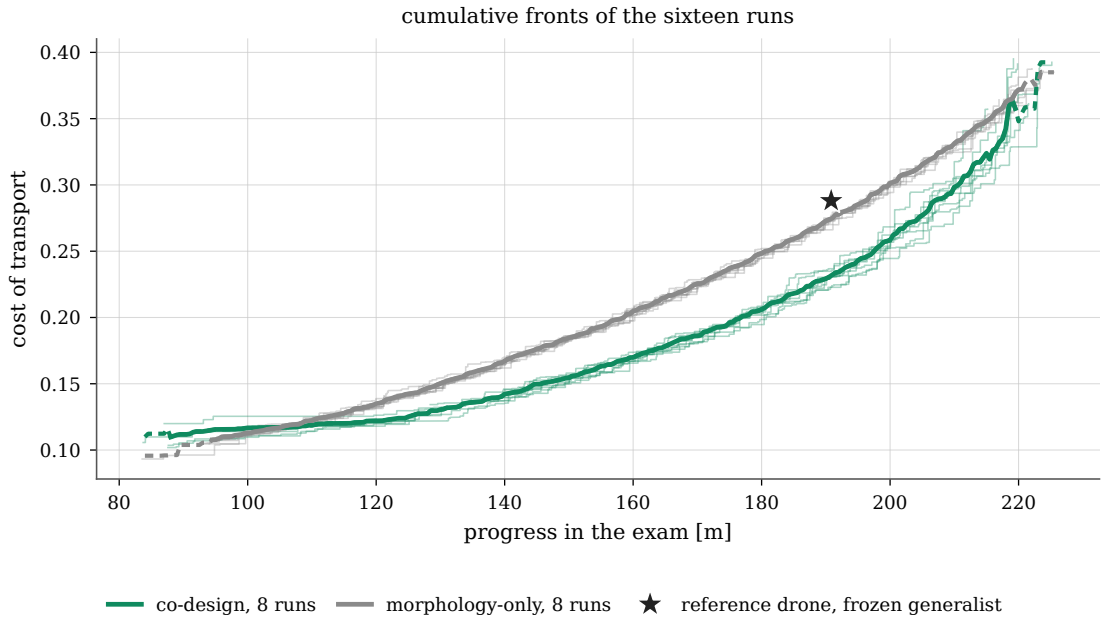


Figure 6.4: The cumulative fronts of the sixteen runs on the exam course, co-design in green and morphology-only in grey. Thin lines are the fronts of the single runs; the thick line of each condition is, at every progress, the mean over its runs of the cost of transport of the cheapest body that reaches that progress, drawn solid where at least four of the eight fronts reach it and dashed where fewer do. The star is the reference drone flown by the frozen generalist.

225 m over its exam flights in the co-design runs and 217 to 226 m in the control (p-value 0.44). No body of either condition gets, on average, past 226 m of the 300 m course, so the tip is common to both conditions and is not where they differ. At the cheap end, the arm of bodies that fly 85 to 110 m for a cost of transport of 0.10 to 0.12 is the same in both conditions, and three co-design runs have no body below 128 m. Between the two ends the fronts separate, and the separation is the result of this section. From 128 to 213 m of progress the co-design fronts are cheaper at every level. At 130, 150, 170, 190 and 210 m every one of the eight co-design runs is cheaper than every one of the eight control runs (Figure 6.5(a), Table 6.3). The gap is about a sixth of the cost. At 150, 170 and 190 m the shift is −0.031, −0.039 and −0.044, that is 17, 17 and 16 % of what the control's bodies spend at those distances. The gap narrows to 12 % at 130 m, where the fronts begin to merge into the common arm, and to 10 % at 210 m, near the common tip. Read the other way, in Figure 6.5(b), a body within a cost of transport of 0.15, 0.20 or 0.25 flies 16, 18 and 16 m farther on a co-design front than on a control front, 12, 11 and 9 % more. At 0.30 the gain is 10 m. At 0.35, where both fronts have reached their tips, it is 1.5 m and not significant.

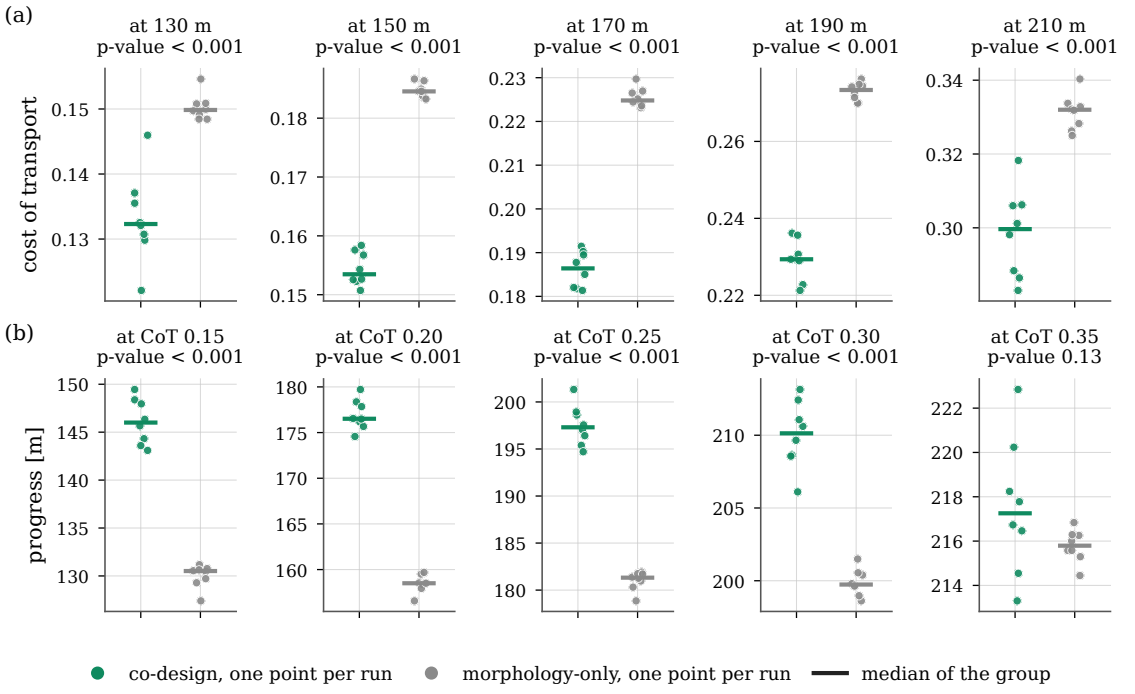


Figure 6.5: The fronts read at fixed progress and at fixed cost of transport, one point per run and the median of each group as a bar. (a) Cost of transport of the cheapest body of the front that reaches 130, 150, 170, 190 and 210 m. (b) Progress of the farthest body of the front within a cost of transport of 0.15, 0.20, 0.25, 0.30 and 0.35. The p-values are those of the exact rank-sum test over the sixteen runs.

Against the reference drone, both searches find bodies that fly farther for less, but not by the same margin. The reference drone, a body of ordinary proportions flown by the generalist, lies on the mean front of the search under that generalist. The control improves on it along the front, in range and in economy at short distances,

Indicator	Co-design	Morphology-only	Shift	95 % CI	p-value
CoT at 150 m	0.153	0.185	−0.031	−0.033 to −0.027	< 0.001
CoT at 170 m	0.186	0.225	−0.039	−0.043 to −0.035	< 0.001
CoT at 190 m	0.229	0.273	−0.044	−0.049 to −0.039	< 0.001
Progress at CoT 0.15	146.0	130.5	+15.9	+13.7 to +18.3	< 0.001
Progress at CoT 0.20	176.5	158.5	+18.1	+17.0 to +19.9	< 0.001
Progress at CoT 0.25	197.3	181.3	+16.2	+14.5 to +18.2	< 0.001
Progress of the tip	219.5	220.6	−1.4	−5.4 to +2.9	0.44
Hypervolume	44.8	41.9	+2.7	+2.0 to +3.3	< 0.001

Table 6.3: Indicators of the cumulative fronts, eight runs against eight: the median over the runs of each condition, the Hodges–Lehmann shift of co-design against morphology-only with its exact 95 % confidence interval, and the p-value of the exact rank-sum test; the single runs are shown in Figure 6.5 and Figure 6.6. CoT is the cost of transport of the cheapest body of the front that reaches the given progress; progress, in metres, is that of the farthest body within the given cost of transport; the hypervolume is measured from the reference point at 80 m and a cost of transport of 0.5.

but hardly at the reference drone's own operating point. There, the cheapest body of a control run costs 0.270 to 0.276 against the reference drone's 0.288, that is 4 to 6 % less. The cheapest co-design body at the same 191 m costs 0.221 to 0.236, 18 to 23 % less.

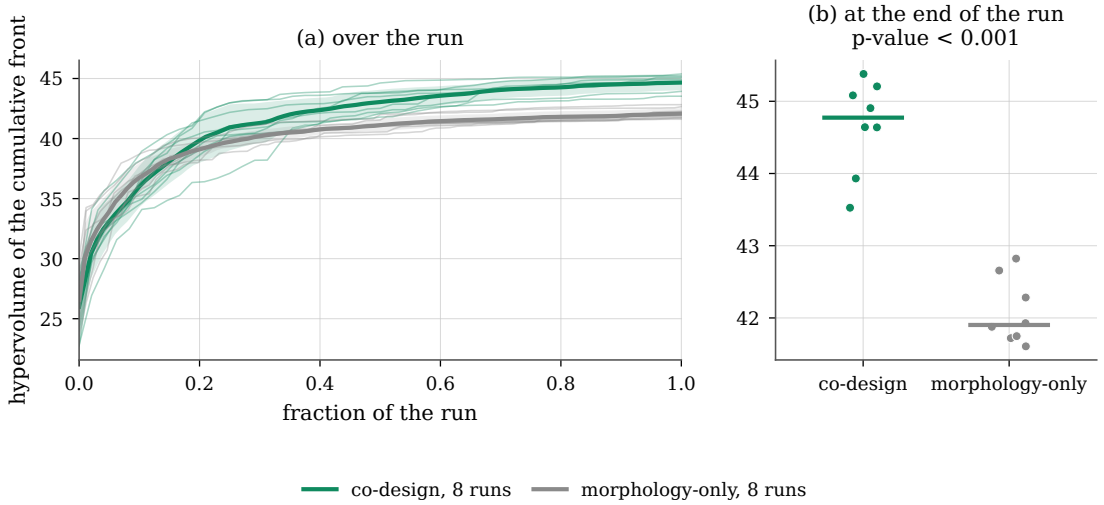


Figure 6.6: Hypervolume of the cumulative fronts, reference point at 80 m and a cost of transport of 0.5. (a) Over the course of the runs, on the axis of the fraction of the run that is completed, since a co-design run holds 49 or 33 exams and a control run 87: thin lines are the single runs, thick lines the mean of each condition with a band of one standard deviation. (b) At the end of the runs, one point per run.

The hypervolume tells the same story in one number and adds its time course (Figure 6.6). At the end of the runs it is 43.5 to 45.4 in the co-design runs and 41.6 to 42.8 in the control, with medians of 44.8 and 41.9. The shift is +2.7, with a confidence interval of $[2.0, 3.3]$ and no overlap between the groups. Over the run the two curves start from the same value, since the first population is the same. The control leads for the first sixth of the run, because its bodies are updated at every generation and its first exams come faster. The co-design curve overtakes it at about a fifth of the run. The control was given more of everything that concerns the bodies, 87 exams against 49 and 5 568 examined bodies against 3 136, and it does not catch up.

In the terms of chapter 3 the comparison is as clean as the pipeline allows. The two conditions start from the same bodies, use the same operators, the same exam course and the same gate, and the control is given the larger budget. The only difference is that in one condition the 896 coefficients are searched and in the other they are held at zero. The fronts differ where there was room for them to differ, between the arm that the gate cuts and the tip that the two conditions share. They differ by up to 17 % of the energy per metre at equal progress and by up to 12 % of the progress at equal energy. *This answers the question with which section 3.4 closed: an adaptation that costs no more than the flights that score a body is enough to change the outcome of the search.* What it does not say is whether the outcome is a different set of bodies, or the same bodies flown better by the rules they were selected with. The next three subsections take the question apart.

6.2.2 Swapping the Pareto Fronts

A front of the co-design pipeline is a front of pairs: bodies selected under the rules of the moment, and rules selected on those bodies. To separate the two, the bodies of a front are flown again under a controller they were not selected with. The last front of four co-design runs and of four morphology-only runs, 37 to 44 bodies each, was rebuilt from the genomes and flown on the exam course over 256 new forests. Two controllers flew every body on the same forests: the frozen generalist, and the Hebbian controller with the best set of rules of a co-design run. The best set is the one with the highest fitness over the whole run. On a co-design front these are rules of the run in which its bodies were selected. The control runs have no rules of their own, so the bodies of their fronts were flown with the best set of rules of one of the co-design runs, transplanted as they were. Figure 6.7 shows one front of each kind and Table 6.4 the eight.

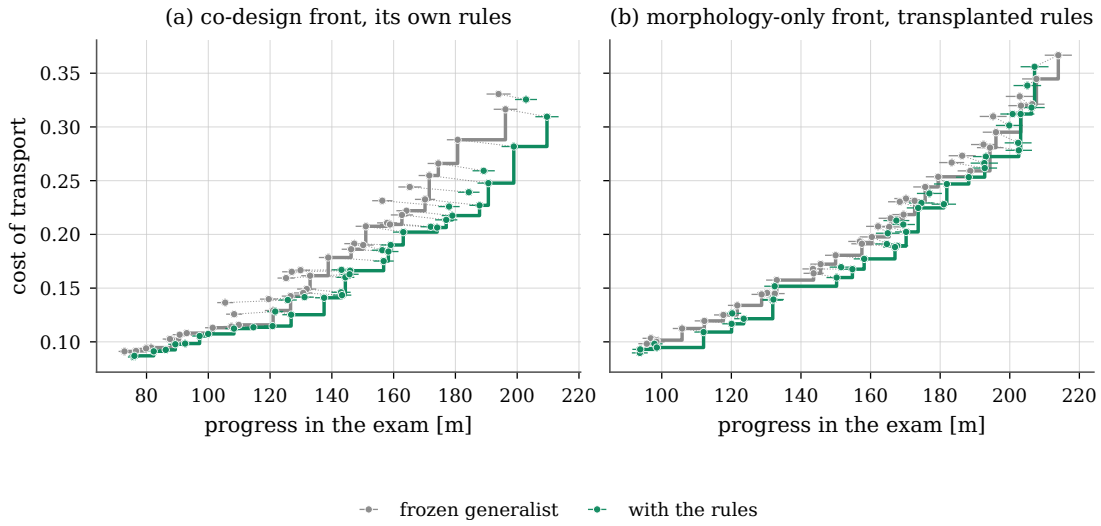


Figure 6.7: The bodies of a last front flown again on the exam course, over the same 256 forests, by the frozen generalist (grey) and by the Hebbian controller with the best set of rules of a co-design run (green); each body is one point in both colours, joined by a dotted line, with the standard error of its progress as a bar, and the steps are the fronts of the two sets of points. (a) A co-design front under its own rules. (b) A morphology-only front under the same rules, transplanted.

On the bodies they were selected with, the rules add 9 to 19 m of progress, and all but a handful of the bodies of every front fly farther. The front of a co-design run under its own rules lies to the right of the same bodies under the frozen generalist along its whole length (Figure 6.7(a)). On the bodies of a morphology-only front the same rules add 1 to 6 m, three to four times less, and on one front they add nothing that the 256 forests can resolve; the two fronts of panel (b) nearly coincide in progress. What transfers is the economy. The cost of transport falls by 0.005 to 0.006 on every front of the control, on almost every body, as it falls on three of the four co-design fronts. This is the saving that section 6.1 found the rules give to any base controller on a fixed body. The progress does not transfer. The rules let

Front	Bodies	Rules	Progress [m]	Bodies farther	CoT
co-design, first run	37	its own	$+19.3 \pm 1.7$	33	$+0.001 \pm 0.001$
co-design, second run	39	its own	$+11.7 \pm 0.9$	38	-0.003 ± 0.000
co-design, third run	42	its own	$+11.8 \pm 0.8$	41	-0.008 ± 0.001
co-design, fourth run	43	its own	$+8.8 \pm 0.7$	43	-0.004 ± 0.000
morphology-only, first run	40	transplanted	$+3.2 \pm 0.7$	28	-0.006 ± 0.000
morphology-only, second run	44	transplanted	$+6.3 \pm 0.7$	38	-0.005 ± 0.000
morphology-only, third run	44	transplanted	$+1.1 \pm 1.1$	25	-0.006 ± 0.000
morphology-only, fourth run	43	transplanted	$+3.1 \pm 0.8$	30	-0.006 ± 0.000

Table 6.4: What the rules add to the bodies of a front: change of the mean progress and of the cost of transport of the front's bodies when they are flown by the Hebbian controller instead of the frozen generalist, on the same 256 forests of the exam course, with the standard error of the mean over the bodies, and the number of bodies that fly farther under the rules. The co-design fronts are flown with the rules of their own run, the morphology-only fronts with the rules of the second co-design run.

a co-design front fly farther than the generalist would fly it, but only on the bodies they were evolved with, and not on bodies that were selected under the generalist. *A co-design front is therefore not the front of the generalist with a better controller on top.* The rules have specialized to the family of bodies that the outer loop retained, and the bodies were retained because the rules fly them, so what the front holds is the pair. This is the reversal of (3.3) seen from its far side: a body selected under an adapting controller is one whose value shows with a controller adapted to it.

6.2.3 PCA Analysis of the Evolved Morphologies

Given the same seed, the bodies of the two conditions can be compared directly, in the space of the genome. Do the two searches, started from the same 64 bodies, go to the same place? The population of a run at each phase is summarized by its centroid in the unit cube of the 15 genes of Table 4.2, and a run by the path of that centroid from the first phase to the last. Distances between centroids are taken in the 15 dimensions of the cube. There, a distance of 0.7 corresponds to a difference of about 0.18 on every gene, or 18 % of its range. The principal components serve the figure only. For each of the six seeds a PCA is fitted on the centroids of its two runs, and the first two components carry 70 to 83 % of their variance (Figure 6.8).

Pair of runs	Pairs	Distance	Range
two morphology-only runs	28	0.57 ± 0.17	0.28 to 0.94
two co-design runs	28	0.70 ± 0.15	0.38 to 0.99
a co-design and a morphology-only run, same seed	8	0.70 ± 0.20	0.50 to 1.08
a co-design and a morphology-only run, different seeds	56	0.74 ± 0.13	0.48 to 1.04

Table 6.5: Distances between the centroids of the last populations of the sixteen runs, in the unit cube of the genome, by kind of pair: mean, standard deviation and range over the pairs. The two co-design runs with phases of six generations are included; the paired distances of the six seeds of Figure 6.8 are 0.59, 0.50, 1.08, 0.51, 0.90 and 0.69.

In every panel of Figure 6.8 the two runs leave the same point together and part within the first five phases. Each run moves about 1.0 in the unit cube from the common starting point. Three of the six pairs end 0.69, 0.90 and 1.08 apart, farther than any two of the seven clustered control runs. The other three end 0.50 to 0.59 apart, which is the distance between two control runs of different seeds (Table 6.5). Two things follow from the table. First, the starting bodies do not decide where a run ends. A co-design run is no closer to the control run that started from its own 64 bodies than to a control run that started from others, 0.70 against 0.74. For only

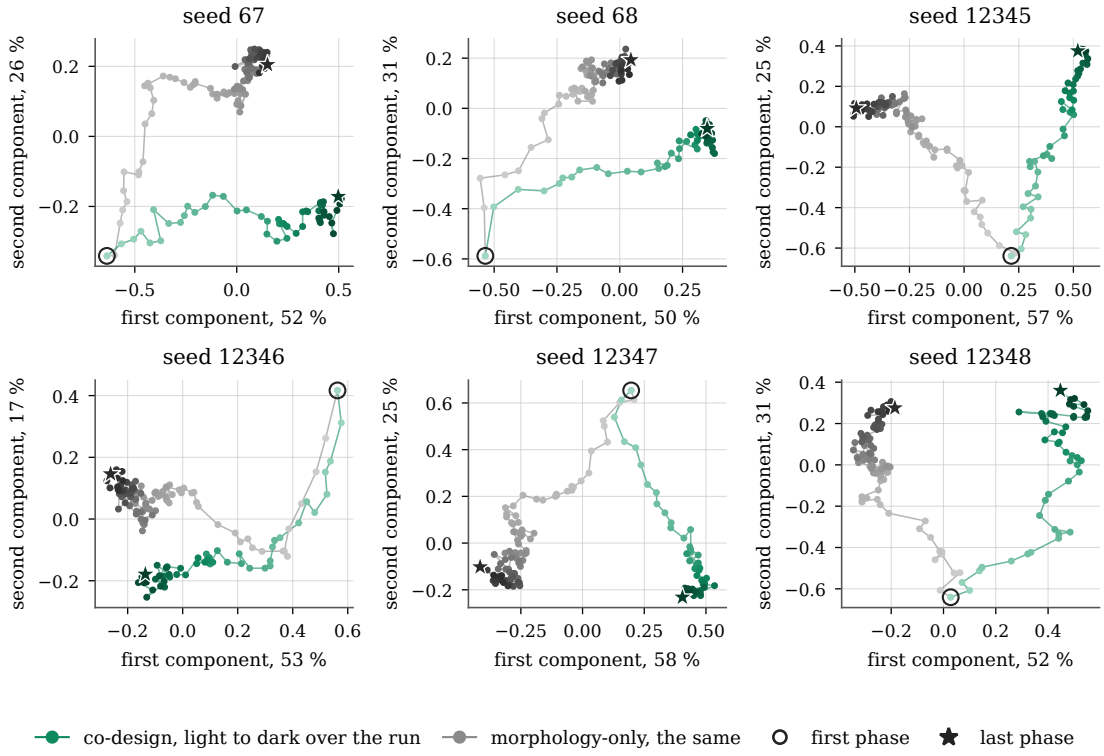


Figure 6.8: The path of the population of the two runs of each seed in the space of the genome, one panel per seed: the centroid of the 64 bodies at every phase, from the first (circle) to the last (star), co-design in green and morphology-only in grey, darker as the run proceeds. Each panel has its own principal components, fitted on the centroids of its two runs, with the share of the variance each carries on its axis; the axes are not comparable across panels.

one of the eight co-design runs its corresponing control run is the closest. Second, the two conditions do not scatter alike. Seven of the eight morphology-only runs end within 0.28 to 0.62 of each other, in one cluster. The eighth, one of the two runs without a co-design partner, ends 0.72 to 0.94 from the other seven. Under a fixed controller the search over bodies thus has one main optimum, which seven runs find. The co-design runs end 0.38 to 0.99 apart, 0.70 on average against 0.49 for the seven control runs. They also end farther from the cluster: the control run nearest to a co-design run is 0.48 to 0.75 away, where the control run nearest to one of the seven controls is 0.28 to 0.48 away. *The adapting controller does not shift the optimum of the body search by a fixed amount: it changes which bodies come first, as section 3.3 argued it would, and the bodies it makes reachable are different ones for different runs.* Which bodies these are, in terms a designer can read, is the subject of the next subsection.

6.2.4 Morphology Choices of The Evolution

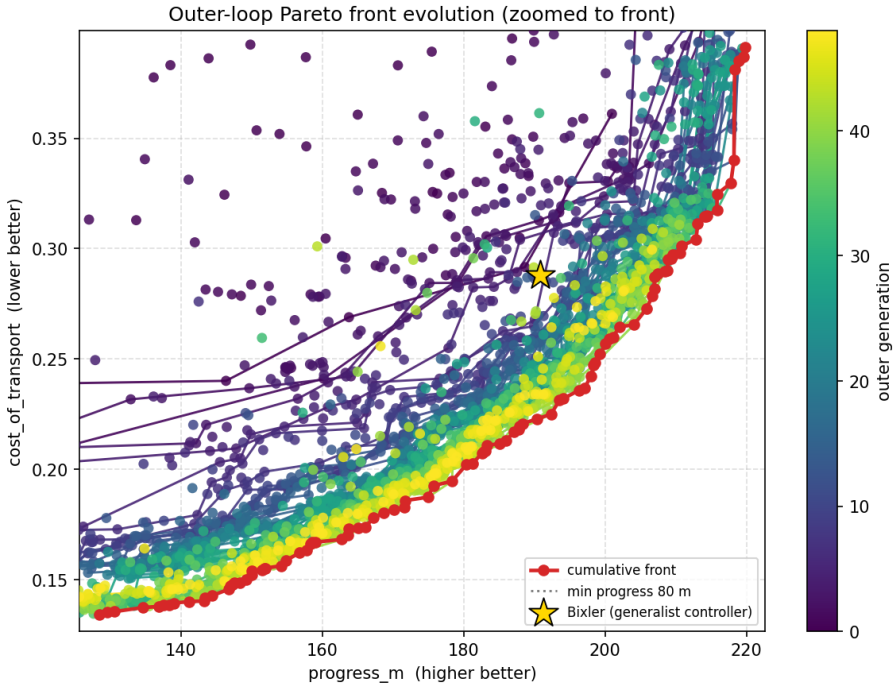


Figure 6.9: The examined bodies of the first co-design run in the plane of the objectives, coloured by the phase of their exam, with the cumulative front of the run in red; bodies below 128 m of progress are left out of the view. The lines join the successive exams of the bodies that survived more than one phase.

Figure 6.9 shows what a run does with its bodies: the 3 136 examined bodies of the first co-design run, coloured by phase. The early exams scatter above the front and the later ones lie along it. The front itself is built over the whole run: its cheap end comes from bodies of the middle phases and its tip from a body of the thirteenth phase. The two ends of a front are two designs, and Figure 6.10 shows the two of

this run: the body that flies farthest, 220 m at a cost of transport of 0.39, and the cheapest body that passes the gate, 128 m at 0.13. Table 6.6 gives their parameters.

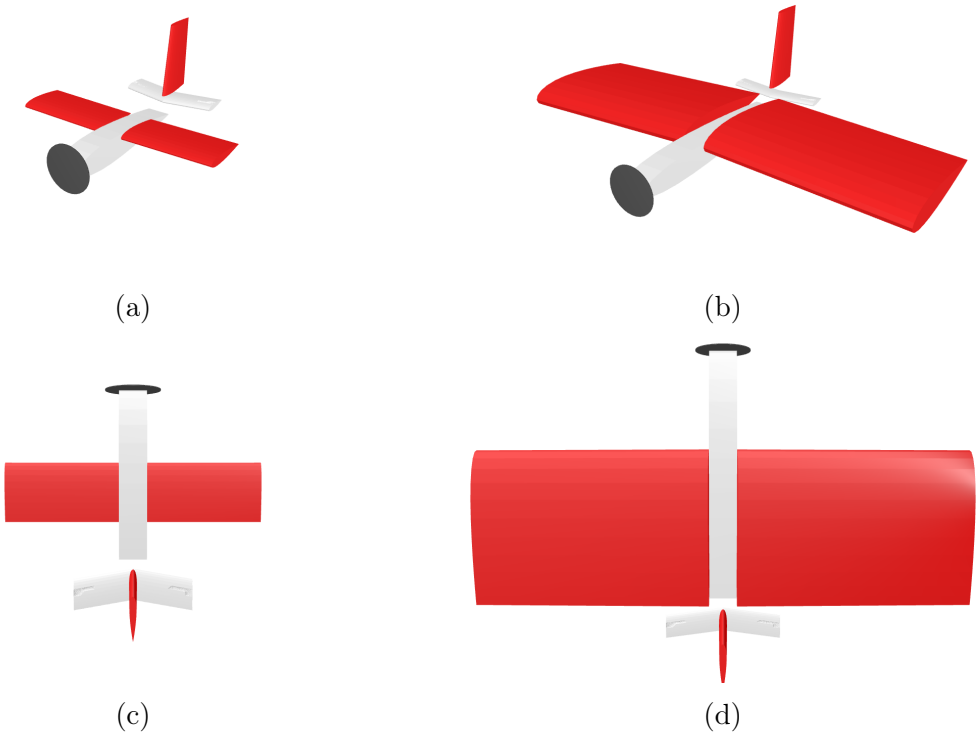


Figure 6.10: The two ends of the front of the first co-design run, drawn at one common scale. Left, (a) and (c): the body that flies farthest. Right, (b) and (d): the cheapest body that passes the gate. Three-quarter views above, top views with the nose up below; wing panels and rudder in red, fuselage and elevator in grey.

The two bodies are as different as the design space allows on its main axes, and alike on the others. The body that flies farthest is small and light. It has panels of 0.40 m, a wing of 0.17 m^2, a fuselage of 0.61 m and a mass of 0.58 kg in all. Its centre of mass is located quite on the back, at 56 % of the fuselage, close to the wing. It is a body with little static margin: its centre of mass lies close to the aerodynamic centre of the whole aircraft, so it turns readily. This is what the dense half of the course rewards, and it pays for the agility with a cost of transport three times that of the other body. The cheapest body is a glider. It has panels of 0.85 m with a chord of 0.56 m, 1.8 m from tip to tip, a wing of almost a square metre that weighs 1.45 of the body's 1.96 kg, a long fuselage with the centre of mass forward, at 31 %, and a positive dihedral. All of this makes for a stable and slow flight. The cheapest body stops at 128 m on average, four metres from the point where the front meets the common arm. In the second half of the course the forest thickens past what such a wing can be steered through. The two bodies agree on their gear ratios, 3.2 for the sweep and 2.2 for the twist, and on a cambered airfoil. A run that reaches both ends of the front from one population reaches them with the same actuation, inherited along the way, and moves the wing and the fuselage instead.

	Farthest body	Cheapest body
Progress and cost of transport in the exam	220 m, 0.39	128 m, 0.13
Span of one wing panel	0.40 m	0.85 m
Chord of the wing	0.21 m	0.56 m
Wing area, both panels	0.17 m^2	0.95 m^2
Length of the fuselage	0.61 m	0.90 m
Centre of mass along the fuselage	56 %	31 %
Attachment of the wing along the fuselage	52 %	56 %
Span of the elevator	0.42 m	0.41 m
Span of the rudder	0.33 m	0.37 m
Dihedral	$-0.9°$	$+2.8°$
Gear ratio of the sweep	3.19	3.18
Gear ratio of the twist	2.22	2.21
Airfoil	NACA 4417	NACA 4520
Mass of the body	0.58 kg	1.96 kg
Mass of the wing	0.15 kg	1.45 kg

Table 6.6: The two bodies of Figure 6.10: the genes of Table 4.2 in physical units, the wing area and the mass of the built body. The chord follows from the span and the aspect ratio of the panel.

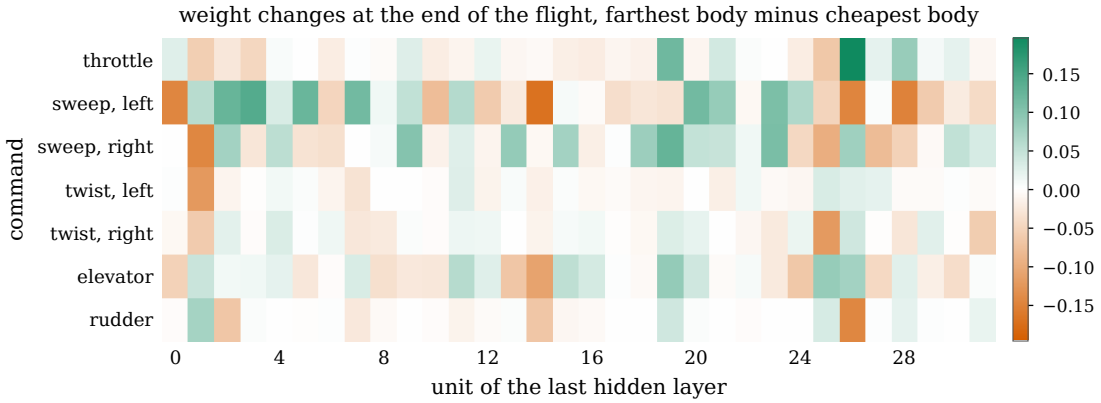


Figure 6.11: Difference between the changes of the output weights, $\boldsymbol{W}_t - \boldsymbol{W}_0$ at the end of the flight, that the last set of rules of the run produces on the two bodies of Figure 6.10, flown on the same forest: one row per command, one column per unit of the last hidden layer, green where the farthest body's weight moved further up, orange where the cheapest body's did.

The same rules do not do the same thing on the two bodies. Figure 6.11 compares the changes of the output weights at the end of a flight of each body under the best set of rules of the last generation of the run, on the same longer evaluation forest of 630 m, at a commanded speed of 12 m/s. The farthest body completes the forest in 54 s; the cheapest one flies 317 m and collides after 28 s. The two matrices of weight changes correlate at 0.76 over their 224 entries, and at 0.83 when the first is read at the same instant as the second ends. Where they differ, they differ most in the rows of the two sweep commands, by up to 0.20 where the trained weights have a mean magnitude of 0.05. This is one flight of each body and no more than an illustration. But it shows what subsection 5.2.2 asked of the plastic layer: the same rules, given a different body, arrive at different weights.

6.2.5 Generalization Capabilities of Evolved Controllers

As subsection 5.3.4 anticipated, rules selected on bodies that resemble each other more at every update are asked to fly a family rather than anything, and subsection 6.2.2 found that their gain in progress does not transfer to bodies of another front. Two measurements follow this along the run. The first is taken in every co-design run, at every generation, as described in subsection 5.3.4. The best set of rules of the generation and the frozen generalist fly the reference drone, a body that is never in the population, on 4 096 new forests of the search course. Over the first twenty generations the two differ by less than one point of fitness in all eight runs, in either direction. Over the last twenty generations the rules are below the frozen generalist by 10 to 16 points in all eight runs, flying 107 to 111 m of the course where the generalist flies 117 m. The loss grows monotonically along the run, and there is no early window in which the rules fly the reference drone better than the controller without rules.

The second measurement asks the same question of the bodies the rules were se-

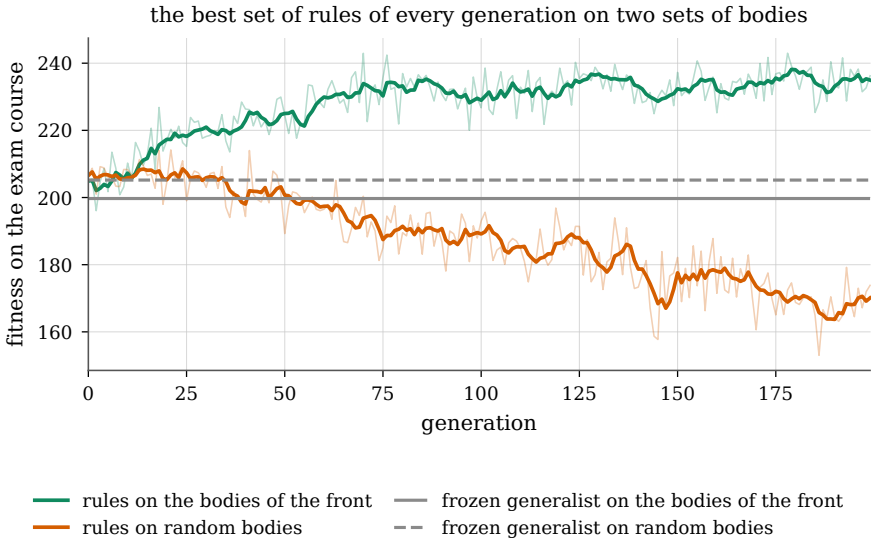


Figure 6.12: The best set of rules of every generation of the first co-design run, flown on the 37 bodies of the run's last front (green) and on 128 bodies drawn at random from the design space (orange), over the same 64 forests of the exam course for every controller: fitness, mean over the bodies; thin lines are the raw values, thick lines a moving average over five generations. The grey lines are the frozen generalist on the same bodies and forests.

lected for and of bodies they never met, on the same course as the exam (Figure 6.12). For the first co-design run, the best set of rules of each of the 200 generations was flown on the 37 bodies of the run's last front and on 128 bodies drawn at random from the design space, as the first population is. Every controller flew the same 64 forests of the exam course as the frozen generalist, on the same bodies. On the front bodies the rules of the first 25 generations already add 11 points of fitness and 8 m of progress to the frozen generalist. The gain grows to 29 points and 19 m by generations 50 to 74, and it levels off from generation 75 at 32 to 36 points and 22 to 25 m, 17 % of the fitness, with every one of the 37 bodies flying farther, while their cost of transport does not move. On the random bodies the same rules add 2 points in the first 25 generations, and they lose from then on without pause: 10 points by generations 50 to 74, 27 points by generations 125 to 149, and 37 points and 26 m of progress in the last 25 generations, an 18 % loss. Against this loss stands a saving of 0.01 to 0.02 in cost of transport, which is the portable economy of subsection 6.2.2 once more.

The two curves are what specialization looks like. The gain on the bodies of the front saturates within the first half of the run, when the family of bodies has largely formed. The loss on bodies outside the front goes on to the last generation. The rules are therefore not simply degrading: they are following the bodies that the outer loop keeps, whose population narrows in the meantime from a mean pairwise distance of 1.56 in the genome cube at the first phase to 0.70 at the last. *What the pipeline pays for its front is the generality of the controller it started from.* The rules that fly the evolved bodies 17 % better than the frozen generalist fly a random body 18 % worse and the reference drone 6 to 9 % worse. They could not be reused on

another family of bodies without being evolved again, as subsection 6.2.2 showed at the scale of a front. For the search this is not a defect. The controller was paid for once, as section 3.4 required, and the adaptation of its output layer to the bodies being scored is what the rules provide at the cost of the flights.

Conclusions & Future Work

7.1 Conclusions

This thesis asked whether a controller can adapt to the body it is given at a cost small enough to sit inside a search over bodies, and whether such an adaptation changes what the search finds. The shape of any answer was fixed in chapter 3: a controller trained once for the whole space of bodies, and an adaptation to the body being scored that costs no more than the flights that score it. That shape was given a content in chapter 5: a generalist trained by reinforcement learning and then frozen, Hebbian rules on its output layer evolved by CMA-ES, and a pipeline that nests the search of the rules inside a multi-objective search over the bodies. In chapter 6 the pipeline was run eight times against eight runs of the search under the frozen generalist, and the inner loop was isolated on fixed bodies. *The experiments answer the question in the affirmative: an adaptation that costs no more than the flights that score a body changes the outcome of the search over bodies, and it changes that outcome for the better on both objectives.* The paragraphs that follow state what was found, in the order in which the thesis built it.

The adaptation costs no more than the flights. The whole pipeline consists of flights of a network that is never trained again. A run of 200 generations flies about a quarter of a million flights per generation, examines 3 136 bodies, of which at most 1 600 are distinct, and takes about four days on one computing node with two graphics cards. The control condition, which updates its bodies at every generation and gives each body the same number of exam flights, took about 130 hours against 94. A controller training for each distinct body would have taken almost two months with a specialist trained for each body, and two and a half years with a controller of the generalist's kind. The cost of the adaptation is the cost of the flights, as the computational feasibility challenge required.

Co-design improves on morphology-only evolution on both objectives. On the exam course the cumulative fronts of the two conditions agree at their two ends and separate in between. The tip of a front is common to both conditions: no body of either condition flies past 226 m of the 300 m course. The cheap arm of the front, the bodies that fly 85 to 110 m for a cost of transport of 0.10 to 0.12, is the same in both conditions. Between 128 and 213 m of progress the co-design fronts are cheaper at every level. At 150, 170 and 190 m the cheapest co-design body spends 16 to 17 % less energy per metre than the cheapest body of the control. Within a cost of transport of 0.15 to 0.25 the farthest co-design body flies 9 to 12 % farther. On each of these indicators every one of the eight co-design runs is better than every one of the eight control runs, and the hypervolume of the fronts separates the two groups as completely. The control was given more exams, more examined bodies and more computing time, and it did not catch up. The reference drone, a body of ordinary proportions, lies on the mean front of the control. At the progress of the

reference drone the control finds bodies that are 4 to 6 % cheaper, and the co-design runs find bodies that are 18 to 23 % cheaper. *The adaptation makes the search find bodies that spend less at equal progress and travel farther at equal cost of transport, and the margin over the reference drone is several times the margin that the search under the fixed controller obtains from the same starting bodies.*

The body search takes a different direction when the controller adapts. Started from the same 64 bodies, the two conditions do not go to the same place in the space of the genome. Seven of the eight morphology-only runs end in one cluster, and the eighth ends elsewhere. Under a fixed controller the search over bodies has one main optimum, and most runs find it. The co-design runs end farther apart from each other, and farther from the cluster than the clustered control runs are from one another. A co-design run is no closer to the control run that started from its own bodies than to a control run that started from other bodies. This is the morphology optimization challenge observed in the data. *An adapting controller does not shift the optimum of the body search by a fixed amount: it changes which bodies come first.* The two ends of one co-design front show how different the bodies that come first can be. One end is a light body of 0.58 kg with wing panels of 0.40 m and its centre of mass well aft, which flies 220 m at a cost of transport of 0.39. The other end is a glider of 1.96 kg with a wing of almost a square metre and its centre of mass forward, which flies 128 m at a cost of transport of 0.13. The two bodies share the gear ratios of their joints and a cambered airfoil: the run moved the wing and the fuselage, and kept the actuation.

The controller adapts to the morphology. A co-design front is a front of pairs. Flown again under their own rules, the bodies of a co-design front travel 9 to 19 m farther than under the frozen generalist, and almost every body of every front travels farther. The same rules transplanted onto the bodies of a morphology-only front add 1 to 6 m. The progress that the rules add therefore belongs to the bodies they were evolved with, and those bodies were retained because the rules fly them. A co-design front is not the front of the generalist with a better controller on top. The weights that the rules produce depend on the body. The same set of rules, flown on the two bodies at the ends of a front, arrives at weight changes that differ most on the two sweep commands. The sweep is the command by which the two bodies differ most in what they can do. The rules are never told which body they fly. The body reaches them through the activity it induces in the frozen layers, and the rules turn that activity into gains that suit that body. *This is adaptation to the body in the sense in which section 5.2 defined it: the same rules, given a different body, arrive at a different controller.*

The rules recover part of what the generalist lacks on a given body. On the reference drone the frozen generalist scores 7.7 points of reward less than a specialist trained for that drone, a gap of 4 %. Rules evolved on that body recover 3.4 points, 44 % of the gap; measured against the three specialists that were trained, the recovery lies between a quarter and a half of the gap. The two gains are of a different kind. The advantage of the specialist lies in avoiding the trees. The

specialist loses 28 % of its flights where the generalist loses 53 %, and it travels farther at a higher cost of transport. The rules leave the crashes and the progress where the generalist has them, and they lower the cost of transport by 9 % at a speed 3 % lower. In reward, the rules recover part of what the generalist lacks. In objective space, they move the body along the cost of transport at constant progress, an axis along which the specialist does not move. The same decomposition holds for rules evolved on 64 random bodies and measured on the reference drone, which that search never flew: a smaller gain, of the same kind.

On a specialist, the rules make the flight dynamics plastic. Evolved on top of a specialist, the rules add more than anywhere else on a fixed body: 9.6 and 9.9 points on two of the three specialists, over 5 %, and nothing on the second. The frozen controllers explain the difference. The three specialists were trained with the same settings and different seeds, and they did not arrive at the same flight of the reference drone. The first and the third fly it at 13.9 m/s with a high cost of transport; the second flies it at 13.3 m/s and cheaply. Under their rules, all four base controllers, the generalist included, end at the same flight: 13.3 to 13.4 m/s, with crashes and progress untouched and a lower cost of transport. The second specialist was flying that flight already, and the rules leave it there. *Where a base controller ends up under the rules does not depend on where it started, and what it gains is the distance it had to travel.* A specialist with rules is, in effect, a controller whose flight dynamics are plastic. The gains of its output layer, and with them the dynamics of the closed loop that the body forms with its controller, are retuned during the flight towards the flight that the fitness rewards on this body. The gain is real in the currency of the search, because a lower cost of transport at equal progress is a better point in objective space.

The price is the generality of the controller. Along a run, the best rules of each generation fly the bodies of the final front better and better, up to 17 % more fitness than the frozen generalist, and they fly bodies drawn at random from the design space worse and worse, down to 18 % less. The reference drone, which is never in the population, ends 6 to 9 % worse under the rules than under the frozen generalist. The rules specialize to the family of bodies that the outer loop retains, and they could not be reused on another family without being evolved again. For the search this is not a defect. The search asked for an adaptation to the bodies being scored, and the adaptation to a family rather than to every body in the space is what keeps the adaptation cheap. For any other use of the evolved rules it is a limit.

Limits of the study. Every quantity of this thesis was measured in simulation, with an aerodynamic model that is trusted for the order of the bodies rather than for absolute values. The tip of the fronts is the same in both conditions, so the comparison was made where the bodies had room to differ. The fixed-body gains were measured on one reference drone and on three specialists whose training reached different flights with the same settings, so that what the rules add to a controller depends on where that controller starts. One reward function was used as the fitness

of the rules, and the economy that the rules find is the change that this reward pays for. The experiments say what an adaptation this cheap does inside this search. They do not say that evolved Hebbian rules are the best cheap adaptation, nor that the fronts found are the best fronts of the design space.

Within these limits, the thesis shows that a controller trained once and a few hundred evolved coefficients are enough to let a search over bodies find bodies that the adapted controller flies better than the fixed one. The results support the hypothesis that a search under the fixed controller would have discarded those bodies.

7.2 Future Work

Three directions follow from the results. The first concerns the representation on which the rules act, which this thesis left as the training produced it. The second and the third concern the transfer of the mechanism, to real drones and to systems other than drones.

7.2.1 Meta-Learning Techniques

The rules act on the latent representation of the last hidden layer. The frozen layers compute 32 features from the history of observations and commands, and the rules change only the gains that translate those features into the commands. The quality of the adaptation is therefore bounded by the quality of the representation. In this thesis the representation was trained by reinforcement learning for a controller with fixed gains, and nothing in that training anticipated that the gains would become plastic. The features are those that serve one compromise set of gains for all bodies, not those that would serve a set of gains adapted to each body. Gradient-based meta-learning trains representations for exactly this situation. MAML [19] learns an initialization from which a few gradient steps adapt a network to a new task. Raghu et al. [60] showed that adapting the last layer alone is enough when the features below are trained with that adaptation in the loop. If the bodies are read as tasks, the same procedure would train the features of the generalist so that an adaptation of the output layer to a body pays as much as possible. The Hebbian rules would then act on a representation built to be read by an adapted output layer. Two forms of this are possible. In the first, the features are meta-trained with gradient steps on the output layer over the training bodies, and the rules are evolved afterwards, as they are now. In the second, the rules of an earlier run are kept in place during the training of the generalist, so that the features are trained with the plastic output layer in the loop. The training of the features and the evolution of the rules then alternate. The second form resembles the co-training of differentiable plasticity [48], with the rules evolved instead of differentiated. *The expected gain is in the half of the mechanism that this thesis kept fixed: the representation that the rules read.*

7.2.2 Testing on Real Drones

The rules adapt the control to the body that is flying, read through the answer of that body to the commands. A real drone differs from its simulated model in its mass and inertia, in the aerodynamic forces on its surfaces and in the response of its servomotors. From the point of view of the rules, these differences are another body. The adaptation that the rules provide to a body they never met may therefore extend to the gap between simulation and reality, during flight and without a model of the discrepancy. The literature on evolved plastic networks gives reasons to expect this. Plastic controllers evolved in simulation transferred to a physical robot better than controllers with evolved fixed weights [20]. Shared Hebbian rules transferred to ten physical robots with a loss of under one percent [11]. The base controller of this thesis was trained with the randomization of forces, masses and latencies on which transfer from simulation usually relies [72, 57]. The rules would add an online adaptation to that offline robustness. None of this can be shown in simulation. The test requires that a body of a front, or the reference drone, be built with its morphing joints, that the network and the Hebbian update run on the onboard computer, and that the depth sectors be obtained from an onboard sensor. The computational part is within reach of any flight controller: the update of the 224 weights is a few thousand arithmetic operations per control step, at 25 steps per second. The laboratory where this work was carried out flies morphing winged drones of the same family [4], so the platform exists. *Whether the rules close part of the gap between simulation and reality, leave it unchanged or widen it is a question that only a flight can answer.*

7.2.3 Testing on Different Systems

The mechanism is general in what it requires: a network pretrained for a task over a family of conditions, a linear output layer, a stream of activity through that layer, and a quantity that varies between lifetimes and that the network is not told. Hebbian rules on top of a pretrained controller are, in this sense, a system-agnostic model. Within robotics, the body is not the only quantity of this kind. A payload, a worn actuator or a damaged limb change the dynamics in the same way, and the robustness to damage of evolved plastic networks is the result that made them known [52]. Outside robotics, the same structure appears wherever a pretrained model follows a stream whose generating process changes between episodes. Sequence analysis is one case: a model trained on many sources is applied to a new source, with no labels on which to fine-tune it. The analysis of stochastic processes is another: the parameters of the process are fixed within an episode, hidden from the model and different from one episode to the next. This is the situation that this thesis described for the body. Differentiable plasticity has been applied to memorization and few-shot classification [48], so plastic weights can act on tasks of that kind. The open question is whether evolved rules on top of a frozen pretrained model give there the gain they gave here. The experiments of this thesis also say what to measure. *The gain found here was a specialization to the family that the search retained, not a general improvement, so the measurement in any new system must be made both on the family and outside it.*

Bibliography

[1] John D. Anderson. *Fundamentals of Aerodynamics*. 6th ed. New York, NY: McGraw-Hill Education, 2017.

[2] Randal W. Beard and Timothy W. McLain. *Small Unmanned Aircraft: Theory and Practice*. Princeton, NJ: Princeton University Press, 2012.

[3] Richard Bellman. *Dynamic Programming*. Princeton, NJ: Princeton University Press, 1957.

[4] Fabio Bergonti et al. "Co-Design Optimisation of Morphing Topology and Control of Winged Drones". In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2024, pp. 8679–8685.

[5] Hans-Georg Beyer and Hans-Paul Schwefel. "Evolution Strategies: A Comprehensive Introduction". In: *Natural Computing* 1.1 (2002), pp. 3–52. DOI: 10.1023/A:1015059928466.

[6] Nick Cheney et al. "Scalable Co-Optimization of Morphology and Control in Embodied Machines". In: *Journal of The Royal Society Interface* 15.143 (2018), p. 20170937.

[7] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. "Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)". In: *International Conference on Learning Representations (ICLR)*. 2016.

[8] Kalyanmoy Deb. *Multi-Objective Optimization Using Evolutionary Algorithms*. Chichester: John Wiley & Sons, 2001.

[9] Kalyanmoy Deb and Ram Bhushan Agrawal. "Simulated Binary Crossover for Continuous Search Space". In: *Complex Systems* 9.2 (1995), pp. 115–148.

[10] Kalyanmoy Deb et al. "A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II". In: *IEEE Transactions on Evolutionary Computation* 6.2 (2002), pp. 182–197.

[11] Fuda van Diggelen et al. *Emergent Heterogeneous Swarm Control Through Hebbian Learning*. https://arxiv.org/abs/2507.11566. 2025.

[12] Mark Drela. "XFOIL: An Analysis and Design System for Low Reynolds Number Airfoils". In: *Low Reynolds Number Aerodynamics*. Ed. by Thomas J. Mueller. Vol. 54. Lecture Notes in Engineering. Berlin, Heidelberg: Springer, 1989, pp. 1–12. DOI: 10.1007/978-3-642-84010-4_1.

[13] Agoston E. Eiben and Emma Hart. "If It Evolves It Needs to Learn". In: *Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO '20)*. ACM, 2020, pp. 1383–1384.

[14] Agoston E. Eiben and James E. Smith. *Introduction to Evolutionary Computing*. 2nd ed. Springer, 2015.

[15] Jeffrey L. Elman. "Finding Structure in Time". In: *Cognitive Science* 14.2 (1990), pp. 179–211.

[16] Roy Featherstone. *Rigid Body Dynamics Algorithms*. New York, NY: Springer, 2008. DOI: 10.1007/978-1-4899-7560-7.

[17] Andrea Ferigo et al. "Evolving Hebbian Learning Rules in Voxel-Based Soft Robots". In: *IEEE Transactions on Cognitive and Developmental Systems* 15.3 (2023), pp. 1536–1546.

[18] Andrea Ferigo et al. "Totipotent Neural Controllers for Modular Soft Robots: Achieving Specialization in Body–Brain Co-Evolution through Hebbian Learning". In: *Neurocomputing* 614 (2025), p. 128811. DOI: 10.1016/j.neucom.2024.128811.

[19] Chelsea Finn, Pieter Abbeel, and Sergey Levine. "Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks". In: *International Conference on Machine Learning (ICML)*. 2017.

[20] Dario Floreano and Joseba Urzelai. "Evolutionary Robots with On-line Self-Organization and Behavioral Fitness". In: *Neural Networks* 13.4–5 (2000), pp. 431–443.

[21] Carlos M. Fonseca and Peter J. Fleming. "On the Performance Assessment and Comparison of Stochastic Multiobjective Optimizers". In: *Parallel Problem Solving from Nature – PPSN IV*. Vol. 1141. Lecture Notes in Computer Science. Springer, 1996, pp. 584–593. DOI: 10.1007/3-540-61723-X_1022.

[22] Félix-Antoine Fortin et al. "DEAP: Evolutionary Algorithms Made Easy". In: *Journal of Machine Learning Research* 13 (2012), pp. 2171–2175.

[23] Genesis Authors. *Genesis: A Generative and Universal Physics Engine for Robotics and Beyond*. https://github.com/Genesis-Embodied-AI/Genesis. Dec. 2024.

[24] Wulfram Gerstner and Werner M. Kistler. "Mathematical Formulations of Hebbian Learning". In: *Biological Cybernetics* 87.5–6 (2002), pp. 404–415.

[25] Wulfram Gerstner and Werner M. Kistler. *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge: Cambridge University Press, 2002. DOI: 10.1017/CBO9780511815706.

[26] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: https://www.deeplearningbook.org.

[27] Agrim Gupta et al. "Embodied Intelligence via Learning and Evolution". In: *Nature Communications* 12 (2021), p. 5721.

[28] Agrim Gupta et al. "MetaMorph: Learning Universal Controllers with Transformers". In: *International Conference on Learning Representations (ICLR)*. 2022.

[29] Tuomas Haarnoja et al. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor". In: *Proceedings of the 35th International Conference on Machine Learning (ICML)*. Vol. 80. Proceedings of Machine Learning Research. 2018, pp. 1861–1870.

[30] Nikolaus Hansen. *The CMA Evolution Strategy: A Tutorial*. https://arxiv.org/abs/1604.00772. 2016.

[31] Nikolaus Hansen, Youhei Akimoto, and Petr Baudis. *CMA-ES/pycma on GitHub*. Zenodo. Feb. 2019. DOI: 10.5281/zenodo.2559634.

[32] Nikolaus Hansen and Andreas Ostermeier. "Completely Derandomized Self-Adaptation in Evolution Strategies". In: *Evolutionary Computation* 9.2 (2001), pp. 159–195.

[33] Donald O. Hebb. *The Organization of Behavior: A Neuropsychological Theory*. New York: Wiley, 1949.

[34] HobbyKing. *H-King Bixler 3 (PNF) Glider 1550mm (61")*. Product page, https://hobbyking.com/en_us/h-king-bixler-3-glider-1550-pnf.html. Accessed 18 September 2026. 2026.

[35] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

[36] Joseph L. Hodges and Erich L. Lehmann. "Estimates of Location Based on Rank Tests". In: *The Annals of Mathematical Statistics* 34.2 (1963), pp. 598–611. DOI: 10.1214/aoms/1177704172.

[37] Ronald A. Howard. *Dynamic Programming and Markov Processes*. Cambridge, MA: MIT Press, 1960.

[38] Yuanming Hu et al. "Taichi: A Language for High-Performance Computation on Spatially Sparse Data Structures". In: *ACM Transactions on Graphics* 38.6 (2019), 201:1–201:16. DOI: 10.1145/3355089.3356506.

[39] Yuanming Hu et al. "DiffTaichi: Differentiable Programming for Physical Simulation". In: *International Conference on Learning Representations (ICLR)*. 2020.

[40] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. "Planning and Acting in Partially Observable Stochastic Domains". In: *Artificial Intelligence* 101.1–2 (1998), pp. 99–134.

[41] Diederik P. Kingma and Jimmy Ba. "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations (ICLR)*. 2015.

[42] Vijay R. Konda and John N. Tsitsiklis. "Actor-Critic Algorithms". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2000.

[43] Hod Lipson and Jordan B. Pollack. "Automatic Design and Manufacture of Robotic Lifeforms". In: *Nature* 406.6799 (2000), pp. 974–978.

[44] Manuel López-Ibáñez, Luís Paquete, and Thomas Stützle. "Exploratory Analysis of Stochastic Local Search Algorithms in Biobjective Optimization". In: *Experimental Methods for the Analysis of Optimization Algorithms*. Ed. by Thomas Bartz-Beielstein et al. Berlin, Heidelberg: Springer, 2010, pp. 209–222. DOI: 10.1007/978-3-642-02538-9_9.

[45] Henry B. Mann and Donald R. Whitney. "On a Test of Whether One of Two Random Variables is Stochastically Larger than the Other". In: *The Annals of Mathematical Statistics* 18.1 (1947), pp. 50–60. DOI: 10.1214/aoms/1177730491.

[46] Barnes W. McCormick. *Aerodynamics, Aeronautics, and Flight Mechanics*. 2nd ed. New York, NY: John Wiley & Sons, 1995.

[47] Michael D. McKay, Richard J. Beckman, and William J. Conover. "A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code". In: *Technometrics* 21.2 (1979), pp. 239–245. DOI: 10.1080/00401706.1979.10489755.

[48] Thomas Miconi, Kenneth O. Stanley, and Jeff Clune. "Differentiable Plasticity: Training Plastic Neural Networks with Backpropagation". In: *International Conference on Machine Learning (ICML)*. 2018, pp. 3559–3568.

[49] Thomas Miconi et al. "Backpropamine: Training Self-Modifying Neural Networks with Differentiable Neuromodulated Plasticity". In: *International Conference on Learning Representations (ICLR)*. 2019.

[50] Kaisa Miettinen. *Nonlinear Multiobjective Optimization*. Boston: Kluwer Academic Publishers, 1998. DOI: 10.1007/978-1-4615-5563-6.

[51] Jed R. Muff et al. "Unconventional Hexacopters via Evolution and Learning: Performance Gains and New Insights". In: *Applications of Evolutionary Computation (EvoApplications 2026)*. Vol. 16524. Lecture Notes in Computer Science. Preprint: arXiv:2505.14129. Springer, 2026.

[52] Elias Najarro and Sebastian Risi. "Meta-Learning through Hebbian Plasticity in Random Networks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020, pp. 20719–20731.

[53] Yael Niv et al. "Evolution of Reinforcement Learning in Uncertain Environments: A Simple Explanation for Complex Foraging Behaviors". In: *Adaptive Behavior* 10.1 (2002), pp. 5–24.

[54] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. "On the Difficulty of Training Recurrent Neural Networks". In: *International Conference on Machine Learning (ICML)*. 2013.

[55] Adam Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*. 2019, pp. 8024–8035.

[56] Joachim Winther Pedersen and Sebastian Risi. "Evolving and Merging Hebbian Learning Rules: Increasing Generalization by Decreasing the Number of Rules". In: *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '21)*. ACM, 2021.

[57] Xue Bin Peng et al. "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization". In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 3803–3810. DOI: 10.1109/ICRA.2018.8460528.

[58] Lerrel Pinto et al. "Asymmetric Actor Critic for Image-Based Robot Learning". In: *Robotics: Science and Systems XIV*. 2018. DOI: 10.15607/RSS.2018.XIV.008.

[59] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. New York: John Wiley & Sons, 1994. DOI: 10.1002/9780470316887.

[60] Aniruddh Raghu et al. "Rapid Learning or Feature Reuse? Towards Understanding the Effectiveness of MAML". In: *International Conference on Learning Representations (ICLR)*. 2020.

[61] Ingo Rechenberg. *Evolutionsstrategie: Optimierung technischer Systeme nach Prinzipien der biologischen Evolution*. Stuttgart: Frommann-Holzboog, 1973.

[62] Nikita Rudin et al. "Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning". In: *Proceedings of the 5th Conference on Robot Learning (CoRL)*. Vol. 164. Proceedings of Machine Learning Research. 2022, pp. 91–100.

[63] Tim Salimans et al. *Evolution Strategies as a Scalable Alternative to Reinforcement Learning*. https://arxiv.org/abs/1703.03864. 2017.

[64] John Schulman et al. "Trust Region Policy Optimization". In: *International Conference on Machine Learning (ICML)*. 2015.

[65] John Schulman et al. "High-Dimensional Continuous Control Using Generalized Advantage Estimation". In: *International Conference on Learning Representations (ICLR)*. 2016.

[66] John Schulman et al. *Proximal Policy Optimization Algorithms*. https://arxiv.org/abs/1707.06347. 2017.

[67] Karl Sims. "Evolving Virtual Creatures". In: *Proceedings of the 21st Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '94)*. ACM, 1994, pp. 15–22.

[68] Andrea Soltoggio, Kenneth O. Stanley, and Sebastian Risi. "Born to Learn: The Inspiration, Progress, and Future of Evolved Plastic Artificial Neural Networks". In: *Neural Networks* 108 (2018), pp. 48–67.

[69] Andrea Soltoggio et al. "Evolutionary Advantages of Neuromodulated Plasticity in Dynamic, Reward-Based Scenarios". In: *Proceedings of the 11th International Conference on Artificial Life (ALIFE XI)*. MIT Press, 2008, pp. 569–576.

[70] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. MIT Press, 2018.

[71] Richard S. Sutton et al. "Policy Gradient Methods for Reinforcement Learning with Function Approximation". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 1999.

[72] Josh Tobin et al. "Domain randomization for transferring deep neural networks from simulation to the real world". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, pp. 23–30. DOI: 10.1109/IROS.2017.8202133.

[73] Emanuel Todorov. "Convex and analytically-invertible dynamics with contacts and constraints: Theory and implementation in MuJoCo". In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2014, pp. 6054–6061. DOI: 10.1109/ICRA.2014.6907751.

[74] Emanuel Todorov, Tom Erez, and Yuval Tassa. "MuJoCo: A physics engine for model-based control". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2012, pp. 5026–5033. DOI: 10.1109/IROS.2012.6386109.

[75] Christopher J. C. H. Watkins and Peter Dayan. "Q-Learning". In: *Machine Learning* 8.3–4 (1992), pp. 279–292.

[76] Paul J. Werbos. "Backpropagation Through Time: What It Does and How to Do It". In: *Proceedings of the IEEE* 78.10 (1990), pp. 1550–1560.

[77] Ronald J. Williams. "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning". In: *Machine Learning* 8.3–4 (1992), pp. 229–256. DOI: 10.1007/BF00992696.

[78] Eckart Zitzler and Lothar Thiele. "Multiobjective Evolutionary Algorithms: A Comparative Case Study and the Strength Pareto Approach". In: *IEEE Transactions on Evolutionary Computation* 3.4 (1999), pp. 257–271.

Appendix A: Simulation Details

This appendix collects the parts of the simulation that chapter 4 summarizes: the internal organization of Genesis, the equations of its rigid body solver, the aerodynamic model in full, the formula of the aerodynamic noise, the collision test and the statistics of the forests, the two forest generators that were not used for the results, and the mass model of the drone with the power of its propeller. It documents the simulator as it was used in every experiment of this thesis.

A.1 Genesis Architecture and Taichi Kernels

A Genesis program builds a *scene*. The scene owns a *simulator*, and the simulator owns a set of *solvers*, one for each physical model the engine supports: articulated rigid bodies, and a family of continuum models for liquids, granular media, cloth and deformable solids (the material point method, smoothed particle hydrodynamics, finite elements, position based dynamics and a grid based fluid solver). A *coupler* exchanges forces between solvers when objects of different kinds touch. The objects themselves are *entities*, each defined by a *morph* that gives its geometry (a primitive shape, a mesh, or a robot description file) and by a material that decides which solver is responsible for it. Only the solvers that own at least one entity are stepped. In this thesis the scene contains a single entity, the drone, loaded from a Unified Robot Description Format (URDF) file that is generated from the genome of the body; it is handled by the rigid body solver, and the aerodynamic solver acts on it as one more solver of the same simulator. Rendering is a separate subsystem and stays switched off during training and search.

All the numerical work of Genesis is done by compiled code. Genesis is built on *Taichi* [38, 39], a domain specific language for parallel numerical computation that is embedded in Python, and of which the Genesis developers maintain their own fork. A Taichi *kernel* is a function written in a statically typed subset of Python syntax and marked with a decorator. The first time a kernel is called, Taichi does not execute its body: it reads the syntax tree of the function, translates it into its own intermediate representation, applies the usual optimizations of a compiler (constant folding, elimination of common subexpressions and of dead branches, simplification of memory accesses) and lowers the result through LLVM to machine code for the selected backend, which here is CUDA. This is *just-in-time* compilation: the program is compiled while it runs, the first time each piece is needed, and every later call launches the compiled binary directly (Figure A.1).

What makes a kernel parallel is a single rule: the outermost `for` loop of a kernel is parallelized automatically, and on a GPU every iteration of that loop becomes one thread. The body of the loop can be arbitrary scalar code, with branches, local variables, inner loops and calls to other Taichi functions, and all of it is fused into one GPU program. This is the opposite of the way a tensor library such as PyTorch reaches the GPU, where every elementary operation is a separate GPU program that reads and writes whole arrays. Physics code is full of logic that differs from element

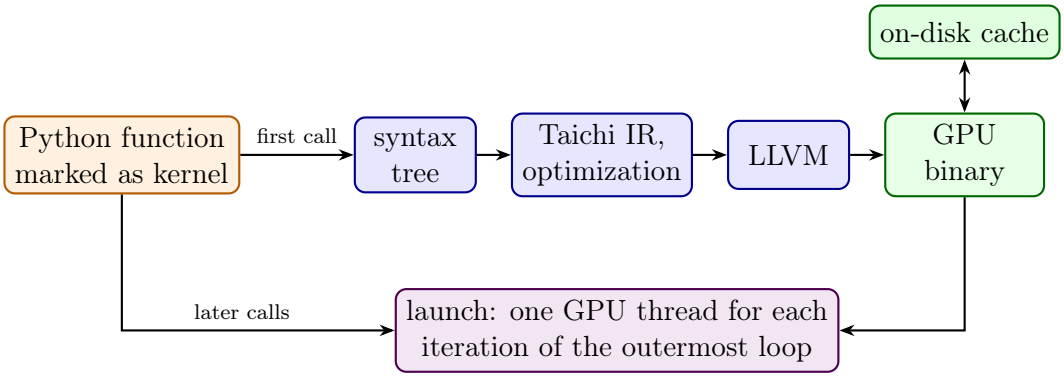


Figure A.1: Life of a Taichi kernel. The first call compiles the Python function down to a GPU binary, which is also written to a cache on disk; later calls, and later runs of the same program, skip the compilation and only launch the binary. IR stands for intermediate representation.

to element (a joint limit that is active in one copy of the scene and not in the next, a wing that is stalled here and not there), and it is awkward and wasteful to express as a chain of whole-array operations, while inside a kernel it is an ordinary `if`. Every kernel launch from Python has a small fixed cost, so the engine is organized as a few large kernels rather than many small ones.

The data that kernels work on live in *fields*: typed, multidimensional arrays that are allocated on the GPU and declared separately from the kernels that use them. Keeping the description of the data apart from the computation is the central idea of Taichi [38], and it has a consequence that matters in practice. A kernel refers to its fields as global variables, so at compilation time it is bound to those specific fields and to their shapes; and the parameters that are declared static (the options of a solver, the number of links of a robot, which branches of the code are enabled) are compiled in as constants, with the unused branches removed. A compiled kernel is therefore specialized to one scene. Genesis makes this explicit by separating the construction of a scene from its *build*: entities and options are declared first, and a single build call then allocates every field and fixes every static parameter. After the build only the values stored in the fields can change (subsection 4.1.1), and a structural change requires a new scene and a new build.

The number of parallel environments B is given to the build call. The model that the B environments share comprises the kinematic tree, the geometry and the nominal inertias. Every state field has an axis of length B: the generalized positions form an array of size $n_q \times B$, the joint space inertia matrix an array of size $n_v \times n_v \times B$, and the parallel loop of each kernel runs over pairs (element, environment), so that a single launch advances every link of every environment. The fields can be read and written as PyTorch tensors that reside on the same GPU: the exchange is a copy between two regions of the memory of the card and never passes through the host.

A.2 Rigid Body Dynamics

The configuration $\boldsymbol{q}$ of the drone collects the position of the fuselage, its orientation as a unit quaternion and the six joint angles, for $n_q = 13$ numbers, and its generalized velocity $\boldsymbol{v}$ collects the linear and angular velocity of the fuselage and the six joint rates, for $n_v = 12$ degrees of freedom. The rigid body solver advances the equation of motion in generalized coordinates,

$$\boldsymbol{M}(\boldsymbol{q})\,\dot{\boldsymbol{v}} + \boldsymbol{c}(\boldsymbol{q}, \boldsymbol{v}) = \boldsymbol{\tau}_{\text{act}} + \boldsymbol{\tau}_{\text{pas}} + \boldsymbol{J}_{\text{a}}(\boldsymbol{q})^{\top}\boldsymbol{F}_{\text{a}} + \boldsymbol{J}_{\text{c}}(\boldsymbol{q})^{\top}\boldsymbol{f}_{\text{c}}, \tag{A.1}$$

where $\boldsymbol{M}$ is the joint space inertia matrix, $\boldsymbol{c}$ gathers the Coriolis, centrifugal and gravitational terms, $\boldsymbol{\tau}_{\text{act}}$ and $\boldsymbol{\tau}_{\text{pas}}$ are the actuator and the passive joint torques, $\boldsymbol{F}_{\text{a}}$ stacks the aerodynamic forces with $\boldsymbol{J}_{\text{a}}$ the Jacobian of their points of application, and $\boldsymbol{f}_{\text{c}}$ are the constraint forces with Jacobian $\boldsymbol{J}_{\text{c}}$. At every substep the solver computes $\boldsymbol{M}$ with the composite rigid body algorithm and $\boldsymbol{c}$ with the recursive Newton-Euler algorithm [16], both of which traverse the kinematic tree in a time linear in the number of links, and factorizes $\boldsymbol{M}$ once for all the linear systems that follow.

The servomotors are modelled as proportional-derivative position controllers with a bounded torque,

$$\tau_{\text{act},j} = \operatorname{clip}\!\big(k_p\,(q_j^{\star} - q_j) - k_v\,v_j,\ -\tau_j^{\max},\ \tau_j^{\max}\big)\,, \tag{A.2}$$

with $k_p = 8\ \text{N m/rad}$, $k_v = 2\ \text{N m s/rad}$ and a torque limit of 1 N m for the tail joints. The servomotors of the wing have a fixed limit of their own, and what reaches the joint is that limit multiplied by the gear ratio of the body (subsection 4.3.1): 1.5 N m on the reference drone. The target angle $q_j^{\star}$ is what the controller commands. The passive torques are a viscous damping and a dry friction in each joint, and an armature term, which stands for the reflected inertia of the motor and of its gears, is added to the diagonal of $\boldsymbol{M}$.

Constraints are handled as in MuJoCo, by a convex optimization over accelerations rather than by a complementarity problem [73]. Let $\boldsymbol{a}_0 = \boldsymbol{M}^{-1}(\boldsymbol{\tau}_{\text{act}} + \boldsymbol{\tau}_{\text{pas}} + \boldsymbol{J}_{\text{a}}^{\top}\boldsymbol{F}_{\text{a}} - \boldsymbol{c})$ be the acceleration the system would have without constraints. The solver selects

$$\begin{gathered}\dot{\boldsymbol{v}} = \arg\min_{\boldsymbol{a}}\ \frac{1}{2}\,(\boldsymbol{a} - \boldsymbol{a}_0)^{\top}\boldsymbol{M}\,(\boldsymbol{a} - \boldsymbol{a}_0)\ +\ \sum_{i\,\in\,\mathcal{A}(\boldsymbol{a})}\ \frac{1}{2}\,d_i\,\big(\boldsymbol{J}_i\,\boldsymbol{a} - a_i^{\text{ref}}\big)^2\,,\\ a_i^{\text{ref}} = -b_i\,\boldsymbol{J}_i\boldsymbol{v} - k_i\,r_i,\end{gathered} \tag{A.3}$$

that is, the acceleration closest to the unconstrained one in the metric of the kinetic energy (Gauss's principle of least constraint), softened by a penalty on each active constraint i. The reference acceleration a_i^{ref} is that of a critically damped spring which would bring the violation r_i back to zero within a chosen time constant, the weight d_i sets how stiffly the constraint is enforced, and the set $\mathcal{A}(\boldsymbol{a})$ contains the inequality constraints that are currently pushing in their admissible direction. The problem is convex and has a unique solution, and the constraint forces $\boldsymbol{f}_{\text{c}}$ follow from it. Genesis offers a Newton and a conjugate gradient method for (A.3); this thesis

uses the latter. In the scenes of this work the problem is small, because contacts are disabled (see below): the only constraints are the mechanical limits of the six joints, which become active only when a joint reaches its stop, and the dry friction of the joints.

Time integration is semi-implicit Euler. With substep Δt,

$$\boldsymbol{v}_{k+1} = \boldsymbol{v}_k + \Delta t\, \dot{\boldsymbol{v}}_k, \qquad \boldsymbol{q}_{k+1} = \boldsymbol{q}_k \oplus \Delta t\, \boldsymbol{v}_{k+1}, \tag{A.4}$$

where the new velocity is used to advance the positions and $\oplus$ denotes ordinary addition for positions and joint angles and the composition with the rotation of axis and angle $\Delta t\, \boldsymbol{\omega}_{k+1}$ for the quaternion, which thereby stays of unit norm. The default integrator of Genesis, used here, makes one modification for stiff actuators: the joint damping $\boldsymbol{D}$ and the derivative gains $\boldsymbol{K}_v$ of the servomotors are treated implicitly, which amounts to replacing $\boldsymbol{M}$ with $\boldsymbol{M} + \Delta t\,(\boldsymbol{D} + \boldsymbol{K}_v)$ in the computation of the acceleration and keeps the servo loop stable whatever its gains. All other forces, the aerodynamic ones included, are explicit: they are evaluated on the state at the beginning of the substep and held constant across it.

Genesis has a complete collision pipeline: a broad phase that sorts axis-aligned bounding boxes to discard distant pairs, a narrow phase for convex shapes, a convex decomposition for meshes that are not convex, and contact forces with Coulomb friction that enter (A.3) as further constraints. None of it is used in this thesis, for the two reasons given in subsection 4.1.2.

A.3 Aerodynamic Model

The model below is the one credited in subsection 4.1.3, documented as it was used. Its forces enter the equation of motion as the term $\boldsymbol{J}_{\mathrm{a}}^{\top} \boldsymbol{F}_{\mathrm{a}}$ of (A.1).

Elements and local flow. The reference frame of each lifting surface sits at the quarter of its chord, with the x axis along the chord towards the trailing edge, the y axis along the span and the z axis normal to the surface. A twist of a wing half pitches this frame, and a sweep yaws it. The air is assumed at rest. If $\boldsymbol{v}_{\mathrm{cm}}$ is the velocity of the centre of mass of the drone and $\boldsymbol{R}_i$ the orientation of the frame of element i, the relative wind seen by the element and its aerodynamic angles are

$$\begin{aligned} \boldsymbol{w}_i &= -\boldsymbol{R}_i^{\top} \boldsymbol{v}_{\mathrm{cm}}, & V_i &= \|\boldsymbol{w}_i\|, \\ \alpha_i &= \operatorname{atan2}(w_{i,z},\, w_{i,x}), & \beta_i &= \arcsin \frac{w_{i,y}}{V_i}, \end{aligned} \tag{A.5}$$

the angle of attack α_i and the sideslip angle β_i. The index i is dropped in what follows.

Lift and drag coefficients. For small angles of attack a lifting surface behaves according to the classical theory of the finite wing [1]. The slope of its lift curve is lower than that of its airfoil section, C_{ℓ_α}, because the vortices shed at the tips reduce the effective incidence, and the reduction grows as the aspect ratio AR of the

surface decreases. The solver uses the formula of Helmbold, which unlike the better known lifting-line result remains accurate for the low aspect ratios of a small drone:

$$C_{L_\alpha} = C_{\ell_\alpha} \frac{AR}{2 + \sqrt{AR^2 + 4}}. \tag{A.6}$$

Lift is linear in the angle of attack measured from the zero-lift angle α_0 of the section, and drag is the sum of a parasitic part and of the drag induced by lift:

$$C_L^{\text{lin}} = f_{Re}\, C_{L_\alpha}\,(\alpha - \alpha_0), \qquad C_D^{\text{lin}} = C_{D_0} + \frac{\left(C_L^{\text{lin}}\right)^2}{\pi\, e\, AR}, \tag{A.7}$$

with $e = 0.8$ the Oswald efficiency factor. The factor $f_{Re} = \min(Re/Re_{\text{nom}}, 1)$, where $Re = \rho V \bar{c}/\mu$ is the Reynolds number of the surface computed on its mean chord $\bar{c}$, reduces the lift of a surface that operates below the nominal Reynolds number of its section, as happens to a narrow wing in slow flight. The aspect ratio in (A.6) and (A.7) is that of the complete wing, including the part of the span covered by the fuselage, with the span reduced by the factor $\cos\beta$ when the surface is swept or the drone flies with sideslip.

The linear regime ends at stall, and a controller that manoeuvres among obstacles, or a body that is badly proportioned, will cross it; a model that stops at stall would make every such event a numerical accident. Beyond stall the solver therefore switches to the coefficients of a flat plate in fully separated flow,

$$C_L^{\text{fp}} = f_{Re}\, \sin 2\alpha, \qquad C_D^{\text{fp}} = 2\, f_{Re}\, \sin^2\alpha, \tag{A.8}$$

which are defined for any incidence, reversed flow included, and give the correct limits: no lift and the drag of a plate normal to the wind at $\alpha = 90°$. The two regimes are joined by a smooth switch centred on the stall angle α_s, as is common in the flight dynamics models of small unmanned aircraft [2]:

$$\begin{gathered} \sigma(\alpha) = \frac{1}{2}\left[1 + \tanh\frac{|\alpha| - \alpha_s}{m\,\alpha_s}\right], \\ C_L = (1-\sigma)\, C_L^{\text{lin}} + \sigma\, C_L^{\text{fp}}, \qquad C_D = (1-\sigma)\, C_D^{\text{lin}} + \sigma\, C_D^{\text{fp}}, \end{gathered} \tag{A.9}$$

with $m = 0.2$ the relative width of the transition. A finite wing stalls at a larger angle than its section, because it needs more incidence to reach the same lift. The solver accounts for this by computing the stall angle of the surface from that of the section, α_s^{2D}:

$$\alpha_s = \alpha_0 + \frac{2\pi\,\delta}{C_{L_\alpha}\,(1 + 2\delta/AR)}, \qquad \delta = \alpha_s^{\text{2D}} - \alpha_0. \tag{A.10}$$

The numerator is the maximum lift coefficient that thin airfoil theory assigns to the section, and the division by C_{L_α} gives the incidence at which the finite surface reaches it; the term in brackets is an empirical correction that lowers the result for short wings. The correction is applied to the wing and to the elevator, while the rudder keeps the stall angle of its section. Figure A.2 shows the resulting curves for the wing of the reference drone.

The parameters of the section, α_0, α_s^{2D}, C_{D_0} and Re_{nom}, follow from the airfoil. The table from which they are read (subsection 4.1.3) was computed from polars

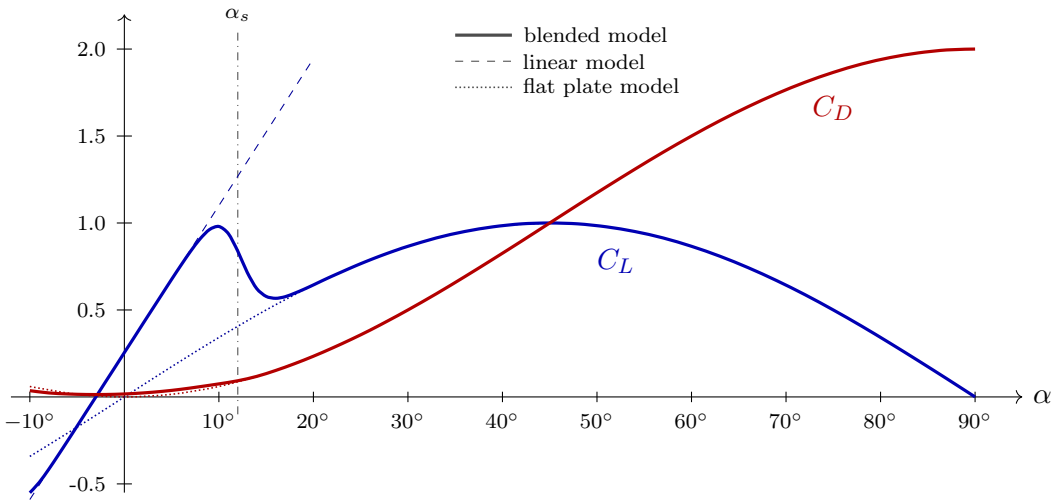


Figure A.2: Lift and drag coefficients of the aerodynamic model for the wing of the reference drone (NACA 3416 section, aspect ratio 7.5). The dashed line is the linear model (A.7) (shown for the lift only), the dotted lines are the flat plate model (A.8), and the solid lines are their blend (A.9) around the stall angle $\alpha_s = 12.0°$ of the finite wing. The maximum lift coefficient is 0.98 and the best lift to drag ratio of the isolated wing is 18.7.

obtained with XFOIL [12] over a sweep of Reynolds numbers. The slope of the section is fixed at $C_{\ell_\alpha} = 0.11$ per degree, the value that thin airfoil theory predicts to within one percent. The fuselage produces no lift and a constant drag coefficient of 0.35 referred to its own reference area.

Forces and points of application. With the dynamic pressure $\frac{1}{2}\rho V^2$, the air density $\rho = 1.225$ kg/m^3 and the area S of the element, lift and drag are

$$L = \tfrac{1}{2}\,\rho V^2 S\, C_L \cos^2\!\beta, \qquad D = \tfrac{1}{2}\,\rho V^2 S\, C_D \cos^2\!\beta, \tag{A.11}$$

where the factor $\cos^2\beta$, applied to the wing and to the elevator, expresses the classical result that only the component of the flow normal to the span of a surface loads it. The two magnitudes are assembled in wind axes and rotated into the frame of the surface through the two aerodynamic angles,

$$\boldsymbol{F} = \boldsymbol{R}_z(\beta)\,\boldsymbol{R}_y(\alpha)\,\big(D,\ 0,\ L\big)^{\top}, \tag{A.12}$$

with $\boldsymbol{R}_y$ and $\boldsymbol{R}_z$ the elementary rotations about the y and z axes of the frame. The rudder is treated as a wing turned by ninety degrees: its incidence is the sideslip angle, its coefficients are evaluated at β, and its lift acts as a side force along y. The force is applied at the *centre of pressure* of the surface, which for an attached flow lies at the quarter of the chord and moves towards the middle of the chord as the flow separates:

$$\frac{x_{\mathrm{cp}}}{\bar{c}} = \frac{1}{4} + \frac{1}{4}\,\frac{|\alpha|}{\pi/2}. \tag{A.13}$$

Since the rigid body solver receives a force and a point, the aerodynamic moments of the drone are never modelled as such: the pitching moment that trims the aircraft,

the rolling moment of a differential twist and the yawing moment of an asymmetric sweep all arise as moments of the forces (A.12) about the centre of mass, and they change with the body because the points of application do.

Interaction between the elements. Two interactions are modelled, both directed at the tail. The first is the *downwash* of the wing: a wing that produces lift deflects the flow behind it downwards, and the elevator works in that deflected flow. Its angle of attack is reduced by the downwash angle that lifting-line theory predicts at the wing,

$$\alpha_{\mathrm{t}} = \alpha - \varepsilon, \qquad \varepsilon = k_{\varepsilon}\,\frac{C_{L,\mathrm{w}}}{\pi\, AR_{\mathrm{w}}}, \tag{A.14}$$

where $C_{L,\mathrm{w}}$ is the lift coefficient that the wing half on the same side has at that same substep, AR_{w} the aspect ratio of the wing and $k_{\varepsilon} = 1$. This couples the two surfaces the way it does on a real aircraft (an increase of wing lift reduces the effectiveness of the tail) and makes the two halves of the elevator work differently when the two halves of the wing do. For this reason the kernel evaluates the elements in a fixed order: the propeller first, then the wing and the fuselage, then the tail. The second interaction is the *slipstream* of the propeller, whose induced velocity, given below, corrects the axial component of the flow seen by the elevator through a gain k_s.

Propeller. The throttle command $u \in [0, 1]$ passes through a first order lag that represents the inertia of the motor and of the propeller,

$$\tilde{u}_{k+1} = \tilde{u}_k + \frac{\Delta t}{\Delta t + 1/(2\pi f_c)}\,(u_k - \tilde{u}_k)\,, \tag{A.15}$$

with a cut-off frequency $f_c = 0.32$ Hz, and the filtered command sets the rotation speed $n = \tilde{u}\, n_{\max}$ (in revolutions per second), where $n_{\max}$ is the smaller of the no-load speed of the motor, $K_V U/60$ for a speed constant $K_V = 2300$ rpm/V and a supply voltage $U = 7.4$ V, and of the speed at which the propeller delivers its maximum static thrust $T_{\max} = 5.2$ N. The thrust of a propeller of diameter d falls as the aircraft flies faster, because the blades meet the air at a lower incidence; this is described by a thrust coefficient that decreases with the *advance ratio* J [46]:

$$J = \frac{V_a}{n\, d}, \qquad T = \rho\, n^2 d^4\, C_{T_0} \max\!\big(0,\; 1 - c_1 J - c_2 J^2\big)\,, \tag{A.16}$$

with V_a the component of the relative wind along the axis of the propeller, $C_{T_0} = 0.093$, $c_1 = 0$ and $c_2 = 2.148$, and T limited to the interval $[0, T_{\max}]$. The thrust acts along the axis, together with a reaction torque κT about it, $\kappa = 0.01$ m. The velocity induced in the slipstream follows from the momentum theory of the actuator disc of area $A = \pi d^2/4$:

$$v_{\mathrm{ind}} = \frac{1}{2}\left(-V_a + \sqrt{V_a^2 + \frac{2T}{\rho A}}\right). \tag{A.17}$$

Coupling and safeguards. At each substep the forces of all elements, with their points of application, are transformed to the world frame and accumulated on the links of the rigid body as forces and moments. Three safeguards protect the explicit coupling. The speed V that enters the coefficients is limited to 40 m/s, the force on each element is limited to 30 N, several times the weight of the drone, and values that are not finite are replaced by zero before they reach the rigid body solver. In a parallel simulation this last point is not cosmetic: the environments of a scene are advanced by the same kernels on shared arrays, and a single environment whose state diverges can otherwise spread its invalid values to the rest of the batch.

Limits of the model. The simplifications named in subsection 4.1.3 are stated here in full. The model is quasi-steady: forces depend on the instantaneous flow only, so unsteady effects such as dynamic stall or the lag of the downwash in reaching the tail are absent. Each surface carries a single lumped force, so the distribution of the load along the span, and with it the progressive stall of a twisted or swept wing, is not resolved. The relative wind (A.5) of every element is computed from the velocity of the centre of mass: the additional flow that a rotation of the body induces at a surface far from the centre of mass is neglected, and the rotations of the drone are damped only indirectly, through the change of incidence they produce. Interference is limited to the two effects described above. Finally, the air is at rest: there is no wind and no turbulence other than the noise described in section A.4.

A.4 Aerodynamic Noise

The aerodynamic noise of subsection 4.1.4 perturbs the force (A.12) of every lifting surface in magnitude and in direction before it is applied:

$$\tilde{\boldsymbol{F}} = \|\boldsymbol{F}\|\,(1+\sigma_m\, s\,\xi)\,\frac{\hat{\boldsymbol{F}}+\sigma_d\, s\,\boldsymbol{g}_\perp}{\|\hat{\boldsymbol{F}}+\sigma_d\, s\,\boldsymbol{g}_\perp\|},$$
$$s(\alpha,\beta) = 1 + 3\left(\frac{\min\!\left(\sqrt{\alpha^2+\beta^2},\,\pi\right)}{\pi}\right)^2, \tag{A.18}$$

where $\hat{\boldsymbol{F}}$ is the unit vector of $\boldsymbol{F}$, ξ a standard normal variable, $\boldsymbol{g}_\perp$ a random unit vector orthogonal to $\boldsymbol{F}$, and $\sigma_m = \sigma_d = 0.05$. The scale s grows from 1 in aligned flow to 4 in fully reversed flow. The centre of pressure is displaced at the same time by a random offset whose standard deviation is a fraction $\sigma_{\mathrm{cp}}\, s/4$ of the chord, with $\sigma_{\mathrm{cp}} = 0.01$, limited to 15% of the chord.

A.5 Collision Test and Forest Statistics

Collision. Let (x_b, y_b) be the position of the centre of a trunk in the horizontal frame centred on the drone and aligned with its heading, b the combined span of the two wing panels and ϕ the roll angle of the drone. The drone has hit the trunk

when

$$|x_b| \le r + \tau \qquad \text{and} \qquad |y_b| \le \frac{b}{2}\cos\phi + r + \tau, \tag{A.19}$$

where τ is the safety margin of section 4.2.

Survival of a blind flight. Consider a drone that flies straight and blind. It touches a trunk whenever a centre falls in the strip of width $w = b + 2r$ swept by its wing, and since the positions are independent, the probability that it is still flying at x is

$$S(x) = \exp\!\left(-\frac{w}{W}\int_0^x \lambda(\xi)\,d\xi\right), \tag{A.20}$$

which for a constant λ is an exponential law: the distance covered has a standard deviation equal to its mean.

With the profile (4.1) the survival (A.20) of the blind straight flight becomes

$$S(x) = \exp\!\left[-\frac{w}{W}\left(\lambda_0\,x + \frac{\lambda_1 - \lambda_0}{2L}\,x^2\right)\right], \tag{A.21}$$

whose quadratic term makes the risk grow with the distance already flown.

Sampling of a growing forest. The coordinate of a tree along the course is drawn from the density proportional to (4.1) by inverting its cumulative distribution, which for a linear profile has a closed form: if u is uniform in $[0, 1]$,

$$x = L\,\frac{\sqrt{\lambda_0^2 + u\,(\lambda_1^2 - \lambda_0^2)} - \lambda_0}{\lambda_1 - \lambda_0}. \tag{A.22}$$

Since the forests of a batch share one tensor and may have different numbers of trees, the shorter lists are padded with inactive trees parked outside the corridor, where they can be neither seen nor hit.

A.6 Uniform and Latin Hypercube Forests

Two forest generators were written besides the growing density generator of subsection 4.2.1. They share its corridor and its trees and differ only in the law of the positions. The first was used for internal tests, and the second was explored and then abandoned. The figures show both in the same corridor and with nearly the same number of trees as Figure 4.2, so that the three forests can be compared directly.

A.6.1 Uniform Density Forests

The simplest forest places a fixed number N of trees independently and uniformly in the rectangle $[0, L]\times[-W/2, W/2]$ (Figure A.3). The density is the same everywhere: $N/(LW)$ trees per square metre or, in the unit used throughout this thesis, $\lambda = N/L$ trees per metre of course.

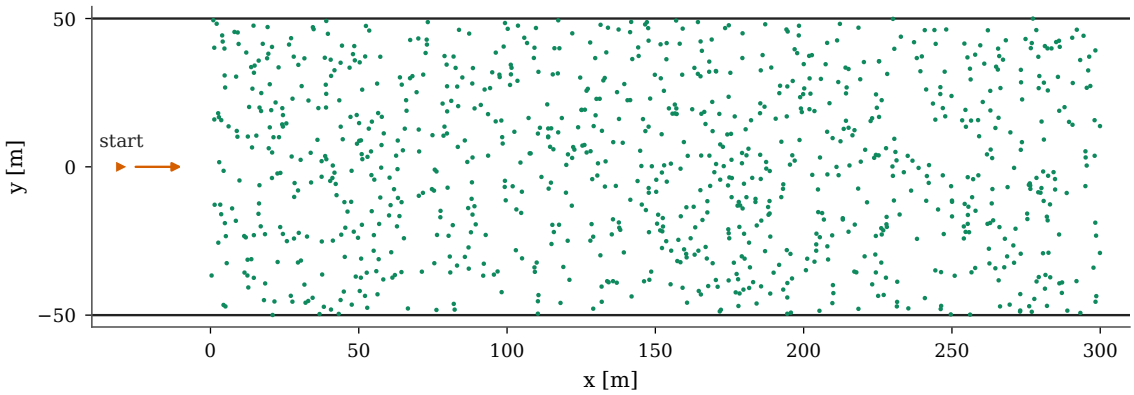


Figure A.3: A uniform density forest seen from above: 975 trees placed independently and uniformly in a corridor of 300 m by 100 m. Trunks are drawn to scale. The thickets and the clearings are what independent positions look like.

A uniform forest is statistically the same at every point of the course, and this is what makes it useful for testing. It was used to verify the depth sensor and the collision test, to debug the flight of new bodies, and to observe how a controller behaves at one given density, without the confounding effect of a forest that changes as the drone advances.

The same property makes it a poor instrument for measuring. The difficulty of a uniform forest is a single number that has to be chosen in advance, and no choice suits a population of bodies of very different ability. In a sparse forest every competent body reaches the end and the progress saturates. In a dense forest every body fails within a few metres. Between these extremes the measure is dominated by chance. The cleanest way to see this is to consider a drone that flies straight and blind. Since the positions of the trees are independent, the risk of hitting one in the next metre is the same whatever the distance already flown. The distance covered before the collision therefore follows the exponential law of (A.20), whose standard deviation equals its mean. In the forest of Figure A.3, with $b = 1.4$ m for the reference drone, that mean is 11 m, measured from the first tree. A controller that sees the trees does much better than this, but it inherits the structure of the problem. In a stationary forest the length of a flight has a large memoryless component, and many flights are needed before two bodies can be told apart.

A.6.2 Latin Hypercube Forests

Independent positions have a cost. Nothing prevents two trunks from being drawn on top of each other, or a few of them from lining up so closely that they form a wall. The effect is not marginal. For independent positions of areal density n, the probability that a tree has a neighbour closer than a distance d is $1 - e^{-n\pi d^2}$. At one tree every 25 m^2, a density that the forests of this work reach in their last part, a quarter of the trunks overlap another trunk ($d = 2r$). Almost two thirds of them are separated from their nearest neighbour by a gap narrower than the width b of the reference drone ($d = 2r + b$). Chains of such pairs form barriers several metres wide (Figure A.4, left). A flight that ends against one of them says little about the

body or about the controller. This luck of the draw is one of the sources of the noise that every comparison in this thesis has to average out.

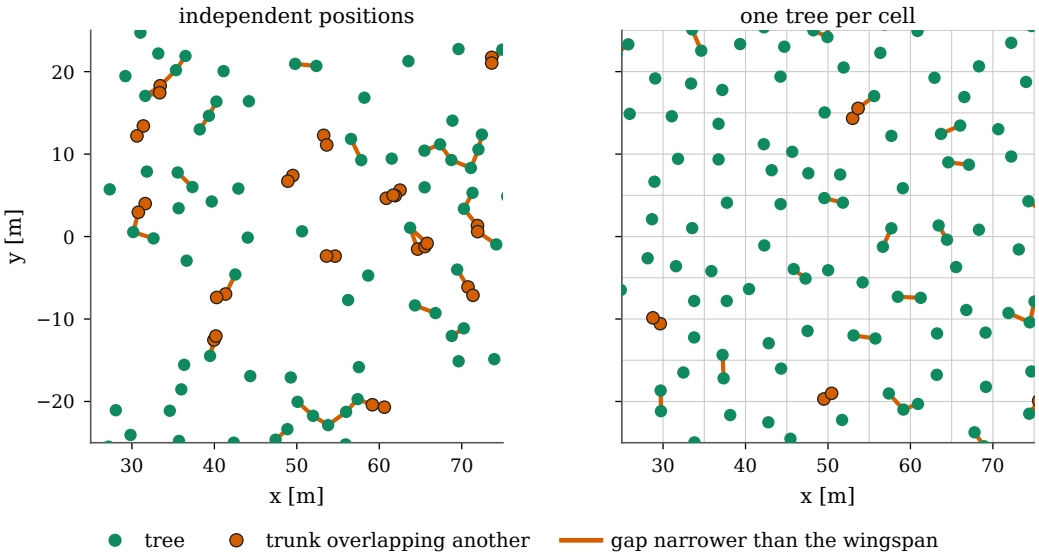


Figure A.4: Independent and stratified positions at the same density, one tree every 25 m^2, in a window of 50 m by 50 m with trunks to scale. With independent positions 25% of the trunks overlap another and 64% leave a gap narrower than the width $b = 1.4$ m of the reference drone; with one tree per cell the two figures fall to 6% and 39% (averages over 200 forests).

The stratified generator was written to remove this effect. It borrows the idea of *Latin hypercube sampling* [47], the classical device for spreading random samples evenly over a domain: the domain is divided into strata, and the samples are forced to occupy the strata evenly while remaining random inside them. Here the corridor is tiled with rectangular cells, and exactly one tree is placed in each cell, at a position uniform within it. To keep the density growing along the course, the cells shrink. The course is cut into columns whose width decreases linearly from s_0 at the entrance to s_1 at the end, rescaled so that they tile the length L exactly. Each column is then cut into equal rows whose height follows the same interpolation (Figure A.5). The name is used loosely. In a Latin hypercube proper each row and each column of the grid holds exactly one sample. Here every cell does, which is the stronger form of stratification also known as jittered sampling.

The stratification does what it was meant to do. At the same density the overlapping trunks fall from 25% to 6%, and the gaps narrower than the width b from 64% to 39% (Figure A.4, right). No region can be crowded, because at most four trees can meet around the corner shared by four cells. No region can be empty, because every cell is occupied. The number of trees is also identical in all the forests of a batch.

It was nevertheless abandoned, because what it removes is part of the task. A forest drawn in this way is tidy: the trees are almost evenly spaced and every gap is about one cell wide. The thickets and the clearings that a real forest has, and that Figure A.3 and Figure 4.2 show, are gone. A body and a controller selected in

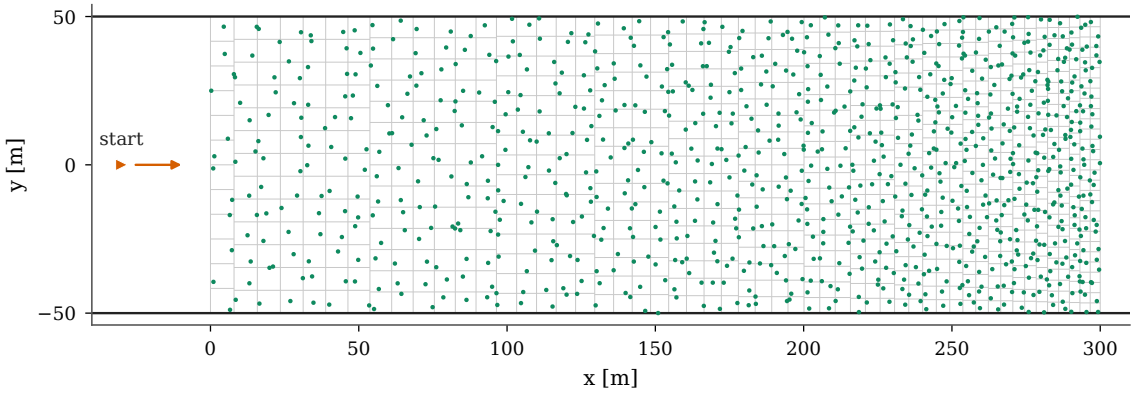


Figure A.5: A Latin hypercube forest seen from above, with its cells. The cells shrink from 8 m to 3.4 m along the course and each holds exactly one tree, for 988 trees in total, about as many as in Figure A.3 and Figure 4.2.

such a forest are selected in the presence of a regularity that no real environment offers, and a search could come to rely on it. The results of this thesis are therefore obtained in growing density forests with independent positions (subsection 4.2.1).

A.7 Mass Model and Propeller Power of the Drone

Masses. The fuselage is a foam shell 15 mm thick, and the wing and tail surfaces are solid foam panels whose thickness follows their airfoil section, all with a density of 20 kg/m^3 (reduced by a fill factor of 0.68 for the surfaces). The fixed masses are 200 g for the battery and the electronics, 30 g for the camera, 41.5 g for the motor and the propeller, 90 g for the six servomotors and 40 g for hinges and fittings.

Propeller power. The power absorbed by the propeller follows from its rotation speed n and its advance ratio J, with the same kind of law as the thrust (A.16):

$$P_{\text{prop}} = \rho\, n^3 d^5\, C_{P_0} \max\!\left(0,\ 1 - c'_1 J - c'_2 J^2\right), \tag{A.23}$$

with $C_{P_0} = 0.044$, $c'_1 = 0$ and $c'_2 = 1.104$.

List of Figures

2.1 The agent-environment interaction loop . . . 15
2.2 Actor and critic networks . . . 18
2.3 A LSTM gate . . . 21
2.4 Structure of an LSTM cell . . . 22
2.5 Covariance adaptation in CMA-ES . . . 24
2.6 Pareto dominance, non-dominated fronts and crowding distance . . . 26
2.7 Locality of the Hebbian update . . . 29

4.1 One control step of the simulation . . . 43
4.2 A growing density forest . . . 47
4.3 Parts and actuators of the drone . . . 49
4.4 Depth sectors and collision rectangle . . . 52
4.5 The standard drone . . . 55

5.1 The networks of a base controller . . . 58
5.2 The Hebbian controller . . . 68
5.3 Effect of the decay on a plastic weight . . . 71
5.4 The co-design pipeline . . . 74
5.5 The course of a run . . . 80

6.1 The inner loop on 64 random bodies . . . 85
6.2 The inner loop on the reference drone alone, generalist base . . . 87
6.3 The inner loop on the reference drone alone, specialist base . . . 88
6.4 The cumulative fronts of the sixteen runs . . . 91
6.5 The fronts read at fixed progress and at fixed cost of transport . . . 92
6.6 Hypervolume of the cumulative fronts . . . 94
6.7 A last front flown by the frozen and by the Hebbian controller . . . 95
6.8 Path of the populations of each seed in the space of the genome . . . 98
6.9 The examined bodies of a co-design run . . . 99
6.10 The two ends of the front of a co-design run . . . 100
6.11 Difference of the plastic weights on the two bodies . . . 102
6.12 Best rules of every generation on front bodies and random bodies . . . 103

A.1 Life of a Taichi kernel . . . 118
A.2 Lift and drag coefficients of the aerodynamic model . . . 122
A.3 A uniform density forest . . . 126
A.4 Independent and stratified positions at the same density . . . 127
A.5 A Latin hypercube forest . . . 128